



Predicción de la suscripción a depósitos a plazo mediante Machine Learning y Deep Learning en campañas de marketing bancario

Mauro Alexis Fernández

Máster en Data Science, Big Data & Business Analytics

Universidad Complutense Madrid

Septiembre de 2025

Contenido

1. Introducción.....	1
1.1. Contexto y motivación del trabajo.....	1
1.2. Contribución esperada.....	1
1.3. Objetivo general y objetivos específicos.....	1
1.4. Justificación de la elección del dataset.....	2
2. Comprensión del problema y del conjunto de datos.....	2
2.1. Descripción del negocio y problemática.....	2
2.2. Detalles del dataset: origen, estructura y variables.....	2
2.3. Definición del objetivo de predicción.....	3
3. Análisis Exploratorio de Datos (EDA).....	3
3.1. Distribución de la variable objetivo (desbalanceo).....	3
3.2. Análisis de variables categóricas y numéricas.....	4
3.3. Correlaciones y relaciones significativas.....	5
3.4. Segmentaciones por atributos clave.....	5
3.5. Insights preliminares.....	6
4. Preprocesamiento de Datos.....	6
4.1. Tratamiento de valores faltantes o inconsistentes.....	6
4.2. Transformaciones de variables categóricas.....	7
4.3. Transformaciones de variables numéricas.....	7
5. Ingeniería de Características.....	7
5.1. Agrupaciones o transformaciones lógicas.....	8
5.2. Generación de nuevas variables derivadas.....	8
5.3. Eliminación de variables redundantes.....	8
5.4. Encoding de variables categóricas.....	9
6. Modelado Predictivo.....	9
6.1. División del conjunto en entrenamiento y test.....	9
6.2. Selección automática o manual de features.....	9
6.3. Normalización o escalado.....	10
6.4. Balanceo de datos.....	10
6.5. Métricas de evaluación y definición de enfoque base.....	10
6.6. Modelos de Machine Learning clásicos testeados.....	11
6.7. Modelo de Deep Learning.....	12
7. Interpretabilidad y Explicabilidad de Modelos.....	12
7.1. Importancia de variables (Feature Importance).....	13
7.2. Técnicas SHAP y LIME.....	13
7.3. Análisis de sesgos y robustez.....	13
8. Evaluación final y comparación de modelos.....	14
8.1. Comparación global de desempeño y selección del modelo óptimo.....	14
8.2. Optimización del modelo XGBoost.....	15
9. Productivización del modelo.....	16



9.1. Desarrollo de un pipeline para predecir nuevos casos.....	16
9.2. Diseño de una interfaz simple y un chatbot RAG.....	16
10. Visualización de resultados.....	17
10.1 Interfaz Web y Frontend con Chatbot.....	17
10.1.1 Interacción directa con el modelo predictivo.....	17
10.1.2 Chatbot explicativo con RAG (Retrieval-Augmented Generation).....	18
10.2 Simulación de escenarios: segmentaciones y decisiones de campaña.....	18
11. Conclusiones y líneas futuras.....	19
11.1. Principales hallazgos técnicos.....	19
11.2. Principales hallazgos de negocio.....	19
11.3. Limitaciones.....	19
11.4. Líneas futuras de trabajo.....	20
ANEXOS.....	21
a. Distribución de variable objetivo.....	21
b. EDA variables numéricas.....	21
c. EDA variables categóricas.....	24
d. Gráficos complementarios del EDA.....	28
e. Segmentación por atributo clave.....	30
f. Métricas extendidas de modelos.....	33
g. Explicabilidad e interpretabilidad del modelo final tuneado.....	37
h. Capturas de interfaz productiva.....	39
i. Instrucciones para ejecución del modelo productivo final.....	42
Luego y en relación con las instrucciones para la ejecución de la aplicación productiva, se detallan a continuación los pasos sucesivos, a saber:.....	43
j. Bibliografía.....	44
k. Código completo del proyecto.....	45
k.1) Archivo de modelado en Jupyter Notebook: TFM_Fernández_Mauro_Alexis_MODELO.....	45
k.2) Archivo de aplicación backend en VisualStudioCode: app.py.....	89
k.3) Archivo de generación de índice RAG en VisualStudioCode: build_rag_index.py. 91	
k.4) Archivo de chatbot con recuperación aumentada de información (RAG) en VisualStudioCode: chat_rag_local.py.....	93
k.5) Archivo con clase de selección de características en VisualStudioCode: feature_selector.py.....	95
k.6) Archivo con clase de escalado de variables en VisualStudioCode: manual_scaler.py.....	96
k.7) Archivo con clase de codificación One-Hot en VisualStudioCode: onehot_transformer.py.....	97
k.8) Archivo con clase de procesamiento manuales en VisualStudioCode: preprocessing.py.....	97
k.9) Archivo de interfaz de usuario (frontend) en VisualStudioCode: chat.html.....	99

1. Introducción

1.1. Contexto y motivación del trabajo

En un entorno bancario cada vez más competitivo y digitalizado, comprender el comportamiento de los clientes y anticiparse a sus decisiones se ha convertido en una necesidad estratégica para las instituciones financieras. En este marco, el uso de técnicas de inteligencia artificial, en particular el aprendizaje automático (Machine Learning) o el aprendizaje profundo (Deep Learning), ofrecen oportunidades valiosas para optimizar las campañas de marketing, segmentar de forma más precisa a los usuarios y mejorar las tasas de conversión.

Este trabajo se centra en el dataset [Bank Marketing](#), un caso real proporcionado por una institución bancaria portuguesa, que llevó a cabo campañas de marketing directo basadas en llamadas telefónicas para promover la contratación de depósitos a plazo fijo. Estas campañas, implementadas entre 2008 y 2010, constituyen una fuente rica de información tanto por su volumen como por la variedad de atributos sociodemográficos, económicos y de interacción que recoge el conjunto de datos.

1.2. Contribución esperada

El presente trabajo tiene como propósito desarrollar, evaluar y comparar distintos modelos de clasificación capaces de predecir la probabilidad de que un cliente suscriba un depósito a plazo, en el contexto de campañas telefónicas. Además del rendimiento predictivo, se busca garantizar la interpretabilidad de los modelos, considerando su eventual integración en sistemas reales de apoyo a la toma de decisiones bancarias.

La contribución principal de este trabajo reside en la combinación y comparación de modelos clásicos de Machine Learning con técnicas de Deep Learning, junto con un análisis exhaustivo de las variables más influyentes y la explicación local y global de las decisiones del modelo. Se propone también un enfoque de productivización del modelo final mediante una API de predicción y un chatbot basado en RAG (Retrieval-Augmented Generation) que permita la interacción y explicación de la solución a partir de herramientas de análisis textual basados en modelos de lenguaje (LLM).

1.3. Objetivo general y objetivos específicos

Objetivo general: Desarrollar un sistema de clasificación que permita predecir la suscripción a depósitos a plazo fijo por parte de clientes bancarios previamente contactados, utilizando técnicas de Machine Learning o Deep Learning sobre datos recolectados desde campañas de marketing telefónico.

Objetivos específicos:

- Comprender la estructura y características del conjunto de datos.
- Realizar un análisis exploratorio que permita extraer patrones preliminares relevantes.
- Aplicar técnicas de preprocesamiento e ingeniería de características adecuadas al problema.
- Evaluar y comparar el rendimiento de modelos clásicos de clasificación y un modelo de Deep Learning (red neuronal multicapa).
- Seleccionar el modelo con mejor equilibrio entre precisión, robustez e interpretabilidad para este proyecto en particular.

- Desarrollar un pipeline productivo reproducible y una interfaz web para la predicción y explicación de resultados, incluyendo un componente conversacional basado en RAG.

1.4. Justificación de la elección del dataset

El conjunto de datos utilizado proviene del repositorio de aprendizaje automático de la Universidad de California en Irvine (UCI) y ha sido ampliamente utilizado en investigaciones académicas. Su elección se justifica por varias razones:

- **Realismo y desafío:** Representa un caso realista y desafiante, con una variable objetivo desbalanceada (la mayoría de clientes NO suscriben depósitos a plazo fijo).
- **Diversidad de variables:** Contiene tanto variables numéricas como categóricas, lo que permite aplicar una variedad de técnicas de preprocesamiento e ingeniería de características.
- **Flexibilidad para el modelado:** Ofrece la posibilidad de evaluar distintos enfoques de modelado, desde métodos lineales hasta técnicas más complejas.
- **Énfasis en explicabilidad:** Permite explorar aspectos de explicabilidad e interpretabilidad, fundamentales en contextos financieros, comerciales y del marketing.

2. Comprensión del problema y del conjunto de datos

2.1. Descripción del negocio y problemática

Las entidades bancarias recurren frecuentemente a campañas de marketing directo para ofrecer productos financieros específicos, como los depósitos a plazo. Estas campañas buscan maximizar la tasa de conversión de clientes, optimizando tanto los recursos invertidos como la rentabilidad esperada. En el caso particular de este estudio, se implementaron una serie de campañas de llamadas telefónicas a clientes actuales y potenciales con el fin de promocionar depósitos a plazo fijo.

El desafío central radica en la necesidad de identificar, de manera anticipada, cuáles clientes tienen mayor probabilidad de aceptar la oferta. Un enfoque tradicional basado en la intuición de los agentes o en reglas manuales resulta insuficiente para capturar la complejidad del comportamiento del cliente. En este contexto, los modelos predictivos de clasificación permiten anticipar la respuesta del cliente y segmentar la base de contactos, mejorando la eficacia de la campaña y reduciendo costos operativos.

2.2. Detalles del dataset: origen, estructura y variables

El conjunto de datos utilizado en este trabajo proviene del **UCI Machine Learning Repository** y corresponde a campañas de marketing directo realizadas por un banco portugués entre 2008 y 2010. Estas campañas consistieron en llamadas telefónicas, que en muchos casos implicaron múltiples contactos con un mismo cliente, con el objetivo de promocionar depósitos a plazo fijo.

El dataset cuenta con **41.188 registros** y **20 variables independientes** que describen diferentes aspectos de cada interacción entre el agente comercial y el cliente, además de una **variable objetivo** (y) que indica si el cliente aceptó o no la oferta (yes/no).

Las variables abarcan:

- **Información sociodemográfica:** age (edad), job (ocupación), marital (estado civil) y education (nivel educativo).

- **Datos financieros:** default (existencia de crédito en incumplimiento), housing (tenencia de préstamos hipotecarios) o loan (préstamos personales).
- **Variables de interacción de campaña:** contact (canal de comunicación), campaign (número de contactos en la campaña actual), previous (número de contactos en campañas previas), duration (duración de la última llamada), day_of_week (día) y month (mes) del contacto, pdays (días desde el último contacto) y poutcome (resultado de campañas anteriores).
- **Indicadores socioeconómicos:** emp.var.rate (tasa de variación del empleo registrado trimestral), cons.price.idx (índice mensual de precios al consumidor), cons.conf.idx (índice mensual de confianza del consumidor), euribor3m (tasa *Euribor* a tres meses) y nr.employed (número de personas empleadas).

En conjunto, el dataset combina **10 variables categóricas, 10 numéricas y 1 binaria**, lo que plantea retos tanto en el preprocesamiento como en el modelado. Además, la variable objetivo presenta un claro **desbalance de clases**: sólo alrededor del **11%** de los clientes aceptó la oferta, lo que obliga a aplicar técnicas específicas para tratar este tipo de problemas y a utilizar métricas de evaluación adecuadas más allá de la simple exactitud (*accuracy*).

2.3. Definición del objetivo de predicción

El objetivo principal del trabajo es construir un modelo de clasificación binaria que prediga la **probabilidad de que un cliente acepte un depósito a plazo**, dadas sus características personales, su historial de interacción con la campaña y el contexto socio-económico del momento. Formalmente, se busca predecir la variable “**y**” (valor ‘**yes**’ o ‘**no**’) en función de los atributos disponibles.

Más allá de la predicción, también se persigue entender **qué variables influyen más en la decisión del cliente**, de manera que los resultados sean explicables y útiles desde el punto de vista del negocio. Esto habilita el desarrollo de estrategias más personalizadas y eficientes, donde los recursos puedan ser dirigidos hacia clientes con mayor probabilidad de conversión.

3. Análisis Exploratorio de Datos (EDA)

El análisis exploratorio de datos (EDA) constituye una etapa fundamental en todo proyecto de ciencia de datos. Permite comprender la estructura del conjunto de datos, identificar patrones, detectar valores atípicos o inconsistencias, y formular hipótesis sobre las relaciones entre variables. En este trabajo se llevó a cabo un EDA exhaustivo que abordó tanto las variables independientes como la variable objetivo, como las relaciones entre ellas.

3.1. Distribución de la variable objetivo (desbalanceo)

La variable objetivo (**y**) indica si el cliente aceptó (yes) o no (no) la propuesta del depósito a plazo. Al analizar su distribución, se observa un marcado **desbalance**: solo un **11,3%** (4.639 registros) de los casos corresponden a una respuesta positiva, mientras que el **88,7%** restante (36.537 registros) indica una respuesta negativa. Este desequilibrio implica que, si el modelo se evaluara únicamente con la métrica de **exactitud** (*accuracy*), podría obtener un valor alto simplemente prediciendo siempre la clase mayoritaria (“no”), sin realmente identificar de forma efectiva a los clientes que sí aceptarían la oferta.

Por este motivo, se utilizaron métricas adicionales como **recall** (sensibilidad), **precisión** (*precision*), **F1-score** y el **área bajo la curva ROC (AUC)**, que ofrecen una visión más completa del rendimiento del modelo en escenarios con clases desbalanceadas. Asimismo, se consideró el uso de técnicas de

re-muestreo (*resampling*), como la generación de ejemplos adicionales de la clase minoritaria, con el fin de mejorar la capacidad del modelo para reconocer patrones asociados a clientes potenciales y, en consecuencia, aumentar su efectividad en la predicción de nuevos casos.

Ver [Anexo A: Distribución de variable objetivo](#)

3.2. Análisis de variables categóricas y numéricas

Las variables **categóricas** fueron analizadas mediante tablas de frecuencia y gráficos de barras, evaluando su distribución general y su relación con la variable objetivo. Algunas observaciones relevantes incluyen:

- **poutcome (Resultado de campañas anteriores):** La mayoría de los clientes no ha participado en campañas previas (*nonexistent*). Las categorías *failure* y *success* tienen una representación significativamente menor.
- **job (Empleos/ocupaciones):** La mayoría de los clientes tienen ocupaciones como *admin.*, *blue-collar* y *technician*. También se observan categorías como *student*, *unemployed* y *unknown*, que podrían requerir un tratamiento especial.
- **day_of_week (Día del contacto):** Distribución relativamente equilibrada entre los días de la semana, aunque martes y miércoles presentan una ligera mayor frecuencia.
- **month (Mes del contacto):** Se observa una fuerte concentración en los meses de mayo, junio, julio y agosto. Esto sugiere una estacionalidad en la ejecución de las campañas, lo cual podría ser relevante considerar.
- **contact (Tipo de contacto):** La mayoría de los contactos se realizó vía celular. La categoría *telephone* tiene baja frecuencia. A pesar del desbalance, la variable puede aportar información útil.
- **loan, housing, default:** En los tres casos, predomina la categoría *no*. La variable *default* tiene una frecuencia muy baja en la categoría *yes*, lo que podría reducir su poder predictivo.
- **education (nivel educativo):** Existen múltiples niveles educativos, con predominancia de *university.degree*, *high.school* y *basic.9y*. Algunas categorías como *illiterate* y *unknown* tienen muy poca representación. Será conveniente agrupar niveles en categorías más amplias (ej.: básico, medio, superior) para reducir la cantidad de valores y mejorar interpretabilidad.
- **marital (estado civil):** Distribución clara entre *married*, *single* y *divorced*, siendo *married* la más frecuente.

Ver [Anexo C: EDA variables categóricas](#)

En cuanto a las variables **numéricas**:

- **age (edad):** La distribución varía entre 17 y 98 años, con una media cercana a los 40 años y una desviación estándar de aproximadamente 10. Esto refleja una población mayoritariamente adulta y heterogénea. Podría considerarse agrupar por rangos etarios.
- **campaign (número de contactos en la campaña actual):** Presenta una fuerte asimetría. El valor máximo es 56, mientras que el percentil 75 es apenas 3, lo que sugiere la presencia de *outliers* (clientes contactados muchas veces).
- **duration (duración de la última llamada):** La variable presenta una distribución fuertemente asimétrica hacia la derecha, con una gran concentración de observaciones en valores bajos (cerca de cero) y una larga cola de registros que se extiende hasta los 4918 segundos.
- **pdays (días desde el último contacto):** Se observa que los percentiles 25, 50 y 75 coinciden en 999, lo cual indica que este valor está siendo utilizado como código para "no contactado previamente". Por su naturaleza, debería ser recodificado o transformado en una variable categórica o binaria.

- **previous (número de contactos previos):** Tanto la mediana como el tercer cuartil son cero, lo que indica que la mayoría de los clientes no tuvo contactos previos. Esta variable presenta una alta concentración de ceros, por lo que podría analizarse su distribución o convertirla en una variable binaria.
- **emp.var.rate, cons.price.idx, euribor3m, nr.employed (Variables macroeconómicas):** Muestran una baja dispersión relativa y valores bastante acotados, lo que refleja su estabilidad como indicadores de contexto económico. Si bien su variabilidad es limitada, podrían aportar valor al modelo al capturar el entorno macro durante cada campaña.

Ver [Anexo B: EDA variables numéricas](#)

3.3. Correlaciones y relaciones significativas

Se calcularon matrices de correlación entre variables numéricas, complementadas con visualizaciones como mapas de calor (**heatmaps**). Si bien no se observaron correlaciones lineales altas entre las variables independientes, algunas relaciones no lineales fueron evidentes al graficar la distribución de “y” respecto de variables como “**duration**”, “**month**” y “**poutcome**”.

En el caso de las variables categóricas, se aplicaron análisis de proporciones condicionales para evaluar su dependencia con la variable objetivo. Variables como **poutcome** (resultado de campañas anteriores) y **education** resultaron particularmente influyentes en la suscripción de depósitos.

Ver [Anexo D: Gráficos complementarios del EDA](#)

3.4. Segmentaciones por atributos clave

Se realizaron segmentaciones de la población de clientes en función de cómo se relacionaban cada una de las variables con la variable objetivo “y”. Este análisis permitió evidenciar si algunos valores/categorías específicos de estas variables se encuentran más representados proporcionalmente en el éxito o no en la suscripción a plazo fijo.

Uno a uno se logró identificar y comenzar a establecer que algunas de las variables originales del dataset (y particularmente, algunas de las categorías/valores dentro de estas) encuentran una representación proporcional mayor entre resultados positivos o negativos en la suscripción de plazos fijos. Se destacan:

- La **edad** presenta una distribución sesgada hacia edades adultas, con un rango entre los 17 y 98 años. La correlación con la variable objetivo indica que existe una leve tendencia a que las personas de mayor edad tienden a ser más propensos a suscribir al plazo fijo.
- La **duración de la última llamada** muestra una fuerte correlación con la variable objetivo, sugiriendo que aquellos contactos con clientes que posean una duración más prolongada son un fuerte indicador de potencial éxito en la suscripción al plazo fijo.
- Otras variables como **número de contactos durante la campaña** y **número de días desde el último contacto** también presentan valores extremos y asimetrías que se abordaron durante el preprocesamiento.
- La tasa de aceptación varía notablemente según el **nivel educativo** y el **tipo de empleo**. Por ejemplo, clientes que posean altos niveles educativos o sean jubilados muestran tasas de conversión superiores al promedio, quizás debido a mayor disponibilidad de recursos y tiempo, junto con una mayor confianza en el sector bancario tradicional. Por el contrario, empleados de trabajos manuales/*blue-collar* y con niveles educativos menores, poseen menores tasas comparativas de conversión, en este caso quizás debido a menores ingresos y/o desconfianza hacia instituciones financieras.

- Por su parte, variables como **estado civil** o los **resultados en campañas previas** presentan también diferencias marcadas, demostrando que (como tendencia) personas solteras y con resultados de suscripción previa de depósitos a plazo fijo, respectivamente, favorecen a las probabilidades en la suscripción de un nuevo plazo fijo.

Ver [Anexo E: Segmentación por atributo clave](#)

3.5. Insights preliminares

Del EDA surgieron varias observaciones clave que guiaron las decisiones de modelado, entre las que se destacan:

1. El desbalance de clases en la variable objetivo (mayoría marcada de valor "NO") obliga a una cuidadosa selección de métricas y estrategias de compensación para evitar errores de predicción.
2. Existen diferencias sustanciales en la tasa de éxito según variables socio-económicas, demográficas, laborales y de comportamiento previo.
3. Segmentos definidos por **nivel educativo alto, sin préstamos vigentes y resultados previos exitosos** mostraron tasas de conversión significativamente superiores.
4. Algunas de las variables presentan una elevada presencia de valores *unknown* (por ejemplo, **education** o **job**) o valores anómalos/ausentes implícitos (como **pdays** que presenta una alta proporción del valor 999 el cual es una forma que usaron en la entidad bancaria para marcar la ausencia de contacto) lo cual requiere de tratamientos específicos para mejorar la calidad de los datos y así mejorar las capacidades predictivas.
5. Otras variables numéricas como **age** se beneficiarán de su agrupamiento en conjuntos etarios que permitan mejorar su manejo.
6. Las variables del contexto socio-económico pueden llegar a presentar problemas de alta-correlación lo cual puede llevar a cierta redundancia informacional. Se analizará la necesidad de conservar o seleccionar todos o algunas de ellas.

Estos hallazgos fueron fundamentales para orientar el preprocesamiento, la ingeniería de características y la ulterior selección de modelos que se abordarán en secciones posteriores.

4. Preprocesamiento de Datos

El preprocesamiento de datos constituye una fase crítica para garantizar la calidad y pertinencia del conjunto de datos que se usarán para el entrenamiento del modelo predictivo. A partir de lo descubierto durante el análisis exploratorio, se aplicaron una serie de transformaciones destinadas a preparar los datos para su uso en modelos de Machine Learning y Deep Learning. Este proceso incluyó tanto procedimientos automáticos como decisiones manuales basadas en las características de los datos previamente descritas.

4.1. Tratamiento de valores faltantes o inconsistentes

El dataset original no contiene valores nulos explícitos, pero algunas variables incluyen la categoría "unknown", que podría interpretarse como una forma implícita de dato faltante.

En las variables **job** (ocupación) y **education** (nivel educativo) se decidió imputar estos valores desconocidos utilizando la información cruzada entre ambas. La hipótesis a considerar es que el "job" se ve influenciado por la "education" de una persona; por lo tanto, es factible inferir ocupación basándose en el nivel educativo de la persona y viceversa. Por ejemplo:

- Si una persona tiene un trabajo de tipo *management* pero su nivel educativo aparece como unknown, se le asignó el nivel “universitario”.
- Si el trabajo figura como unknown pero el nivel educativo corresponde a 4 años de educación básica, se le asignó la ocupación “manual/blue-collar”.

En cambio, para variables como **marital** (estado civil) y **default** (crédito en incumplimiento), se decidió mantener "unknown" como una categoría válida. Esto se debe a que su presencia podría contener información relevante, como evasión de respuesta, omisión o falta de contacto suficiente, que a su vez podrían estar asociados con un comportamiento específico del cliente.

No se identificaron valores numéricos inconsistentes. Se revisaron posibles valores atípicos (*outliers*) mediante visualización y análisis estadístico. Si los valores eran poco frecuentes pero plausibles —por ejemplo, una edad de 98 años en un país con alta longevidad como Portugal— se conservaron, ya que pueden aportar información valiosa al modelo.

4.2. Transformaciones de variables categóricas

Para las variables que representan el **día de la semana** y el **mes del contacto**, se reemplazaron sus valores en formato textual (por ejemplo, "mon" o "tue" para lunes y martes, y "may" o "aug" para mayo y agosto) por su equivalente numérico. Así, por ejemplo:

- "mon" se transformó en 1 y "tue" en 2 para los días.
- "jan" se transformó en 1 y "feb" en 2 para los meses.

Esta transformación permite reflejar el orden natural que existe entre días y meses, y al mismo tiempo convierte datos en texto en valores numéricos más manejables para la mayoría de los modelos de machine learning utilizados posteriormente.

4.3. Transformaciones de variables numéricas

Se aplicaron transformaciones específicas a variables numéricas con distribuciones muy asimétricas o con valores extremos. En particular:

- **pdays**: esta variable toma el valor 999 cuando no hubo contacto previo con el cliente. Para reflejar mejor su significado, se recodificó separando explícitamente los casos con y sin historial previo de contacto.
- **age**: con el fin de facilitar la interpretación posterior de los resultados, la edad numérica se agrupó en rangos etarios (*binning*), asignando categorías como young (jóvenes), lower middle aged (adultos jóvenes), middle aged (adultos medios) y senior (adultos mayores).

Estas transformaciones ayudan a que el modelo interprete la información de manera más coherente, evitando que valores especiales o escalas amplias distorsionen el aprendizaje.

5. Ingeniería de Características

La ingeniería de características constituye una de las etapas más relevantes en todo proyecto de aprendizaje automático, ya que permite transformar el conocimiento del dominio en variables que capturen mejor la estructura subyacente del problema; y así mejorar la precisión, como también la interpretabilidad y eficiencia del modelo. En este trabajo y luego de realizado el preprocesamiento, se aplicaron técnicas manuales y procedimientos automáticos para enriquecer y refinar la representación de los datos (antes del modelado).

5.1. Agrupaciones o transformaciones lógicas

Con el objetivo de reducir la dispersión de categorías y mejorar la capacidad de los modelos para generalizar frente a nuevos datos, se realizaron agrupaciones en algunas variables del conjunto de datos:

- **job_grouped**: las ocupaciones individuales se agruparon en categorías más amplias:
 - **White-collar**: admin., technician, management.
 - **Blue-collar**: blue-collar, services, housemaid.
 - **Self-employed**: self-employed, entrepreneur.
 - **Non-active**: student, unemployed, retired.
 - **Other**: unknown.
- **education_grouped**: los niveles educativos se consolidaron en cuatro categorías:
 - **Basic**: basic.4y, basic.6y, basic.9y.
 - **Middle**: high.school, professional.course.
 - **Superior**: university.degree.
 - **Other**: unknown, illiterate.

5.2. Generación de nuevas variables derivadas

Se incorporaron variables sintéticas creadas a partir de combinaciones lógicas o transformaciones de las variables originales, con el fin de capturar relaciones adicionales entre los datos. Algunos ejemplos son:

- **contacted_previously**: indicador binario que toma el valor 1 si el cliente fue contactado al menos una vez en el pasado, y 0 en caso contrario. Esto permite reflejar de manera directa si existió historial previo de contacto.
- **life_stage**: variable combinada que integra el grupo etario y el estado civil del cliente, generando categorías como "middle aged & married" o "young & single".
- **socio-economic**: variable que combina ocupación y nivel educativo, generando un perfil socioeconómico general. Ejemplos: "admin. & university.degree" o "technician & professional.course". Este perfil puede ser útil tanto para la interpretación de resultados como para la segmentación de clientes en campañas.

Estas nuevas variables ayudaron a capturar patrones no evidentes en los datos originales, lo que contribuyó a mejorar el rendimiento de los modelos en las etapas posteriores.

5.3. Eliminación de variables redundantes

Durante el proceso de análisis y transformación, se identificó que la variable "**nr.employed**" presentaba alta redundancia debido a su elevada correlación con otras variables macroeconómicas del dataset, como "**emp.var.rate**" y "**euribor3m**". Esta situación genera problemas de multicolinealidad que pueden afectar negativamente al modelo predictivo. En consecuencia, se decidió eliminar la variable **nr.employed**.

5.4. Encoding de variables categóricas

En esta etapa, después de limpiar y transformar los datos, el dataset aún incluye variables categóricas, que son aquellas que representan categorías o grupos, ya sea con un orden (ordinales) o sin él (nominales). Para que los modelos predictivos puedan utilizarlas, estas variables se convirtieron en un formato numérico mediante una técnica llamada **One-Hot Encoding**.

Esta técnica transforma cada categoría en una columna separada con valores 1 o 0, indicando si esa categoría está presente o no en cada registro. De esta manera, se evita que el modelo interprete un orden o relación que no existe entre las categorías, lo que ayuda a que el aprendizaje sea más preciso.

6. Modelado Predictivo

El modelado predictivo constituye el núcleo del presente trabajo. A partir del conjunto de datos procesado, se entrenaron y evaluaron múltiples modelos de clasificación con el objetivo de predecir la suscripción a depósitos a plazo. Se combinaron enfoques clásicos de diversos modelos de Machine Learning, junto con un modelo de Deep Learning (red neuronal multicapa), permitiendo comparar su rendimiento y capacidad de generalización predictiva.

6.1. División del conjunto en entrenamiento y test

Para poder medir con precisión qué tan bien funciona el modelo antes de usarlo en la práctica, el conjunto de datos se separó en dos partes:

- **Datos de entrenamiento** (80%): utilizados para que el modelo “aprenda” a reconocer patrones entre los datos.
- **Datos de prueba** (20%): reservados para evaluar el desempeño del modelo con información que no vio durante su entrenamiento.

Esta separación se hizo cuidando que ambas partes mantuvieran la misma proporción de casos positivos y negativos que en los datos originales, de manera que la evaluación sea representativa de la realidad.

Trabajar de esta forma garantiza que la medición del modelo sea objetiva y que sus resultados no estén “inflados” por haber visto previamente la información.

6.2. Selección automática o manual de features

Para optimizar el conjunto de datos que utiliza el modelo, se aplicó un proceso sistemático llamado **“Recursive Feature Elimination with Cross-Validation” (o RFECV)**, utilizando un clasificador **XGBoost** como estimador base.

En términos simples, este método evalúa la importancia de cada variable y, de forma progresiva, elimina la menos relevante en cada iteración. Después de cada eliminación, se vuelve a entrenar y probar el modelo para comprobar si el rendimiento mejora o empeora.

La validación de cada paso se realizó mediante *validación cruzada estratificada*, que consiste en probar el modelo varias veces con diferentes fragmentos de los datos, garantizando que los resultados sean consistentes y no dependan de una única división de la información.

Como criterio para medir la calidad del modelo se utilizó el **F1-score**, una métrica que combina precisión y sensibilidad, especialmente útil en este caso porque la proporción de clientes que aceptan la oferta es mucho menor que la de los que la rechazan.

Este enfoque permitió identificar un subconjunto de 108 variables clave para la clasificación (de un total inicial de 162 luego de aplicar el encoding de las variables categóricas), reduciendo la

complejidad del modelo, mejorando su interpretabilidad y disminuyendo el riesgo de que aprenda patrones irrelevantes.

6.3. Normalización o escalado

En el conjunto de datos, algunas variables numéricas tienen rangos muy diferentes. Por ejemplo, la edad de los clientes suele ir de 18 a 98 años, mientras que el número de contactos realizados en una campaña puede ser de 1 a más de 20. Si se usan estos valores tal cual, las variables con números más grandes podrían influir desproporcionadamente en el modelo, incluso aunque no sean las más importantes.

Para evitarlo, se aplicó un proceso llamado **escalado**, que ajusta los valores para que todas las variables numéricas se midan en la misma “escala” y tengan un peso comparable en el aprendizaje del modelo. En este proyecto se utilizó la técnica **MinMaxScaler**, que transforma los datos para que todos los valores queden dentro de un rango entre 0 y 1.

Este ajuste no cambia la información original, pero sí ayuda a que el modelo sea más estable, aprenda más rápido y no se vea sesgado hacia variables que tienen números más grandes solo por su naturaleza de medición.

6.4. Balanceo de datos

En los datos originales, la gran mayoría de los clientes no contrata el depósito, mientras que solo un pequeño porcentaje sí lo hace. Esta desproporción puede hacer que el modelo “se acostumbre” a predecir casi siempre que un cliente no contratará el producto, pasando por alto oportunidades reales.

Para resolver este problema, se aplicó una técnica llamada **SMOTE** (*Synthetic Minority Over-sampling Technique*). En términos simples, este método crea nuevos ejemplos ficticios de clientes que sí contrataron el depósito, basándose en clientes reales con características similares.

El resultado es un conjunto de datos de entrenamiento más equilibrado, que permite al modelo aprender de forma más justa sobre ambos grupos de clientes. Esto aumenta su capacidad para reconocer patrones que indiquen una posible contratación y, por lo tanto, mejora la identificación de oportunidades comerciales.

6.5. Métricas de evaluación y definición de enfoque base

Dado que la variable objetivo está desbalanceada, es decir, hay muchas más respuestas negativas que positivas, no es suficiente evaluar el desempeño del modelo solo con la precisión general. Por eso, se eligieron métricas que reflejan mejor la calidad del modelo en este contexto:

- **Accuracy:** medir el porcentaje total de aciertos, pero puede ser engañosa cuando existe mucho desbalance de los datos (como es nuestro caso).
- **Recall (Sensibilidad):** mide qué tan bien el modelo detecta a los posibles suscriptores reales, lo cual es clave para nuestro objetivo.
- **Precisión:** indica qué porcentaje de las predicciones positivas son correctas.
- **F1-Score:** combina precisión y recall en una sola medida que da un panorama equilibrado del rendimiento.
- **AUC-ROC:** evalúa la capacidad del modelo para diferenciar entre suscriptores y no suscriptores en distintos niveles de decisión.

Estas métricas se calcularon tanto en un conjunto de datos reservado para prueba como mediante validación cruzada estratificada, lo que garantiza una evaluación robusta y confiable.

Como referencia inicial, se definió un modelo base o “baseline” muy simple, que siempre predice la clase mayoritaria (es decir, que nadie se suscribe). Este modelo se implementó con la herramienta *DummyClassifier* de Python y sirve para establecer un punto mínimo de comparación. Así, los modelos más complejos deben superar este desempeño básico para considerarse efectivos.

6.6. Modelos de Machine Learning clásicos testeados

Para predecir la probabilidad de que un cliente suscriba un depósito, se evaluaron varios modelos clásicos de Machine Learning. Cada uno tiene características y enfoques diferentes, que influyen en su precisión, facilidad de interpretación y capacidad para manejar el desbalance en los datos. A continuación, se describen brevemente los modelos probados y su desempeño general.

- **Regresión logística:** Este modelo establece relaciones lineales entre las variables y la probabilidad de que un cliente responda positivamente. Fue usado inicialmente y luego mejorado con técnicas que evitan el sobreajuste. Su simplicidad y fácil interpretación son valiosas, aunque su rendimiento fue superado por modelos más complejos.
- **Árboles de decisión:** Estos modelos dividen los datos en grupos según reglas simples y fáciles de entender. Aunque son muy interpretables, tienden a ajustarse demasiado a los datos de entrenamiento, perdiendo precisión al aplicarse a nuevos datos.
- **Random Forest:** Esta técnica combina muchos árboles de decisión, cada uno entrenado con diferentes partes del conjunto de datos, lo que reduce errores y mejora la estabilidad. Ajustando parámetros como la cantidad de árboles y su profundidad, Random Forest fue uno de los modelos más robustos y equilibrados en rendimiento.
- **XGBoost:** Un modelo basado en *boosting* (técnica que combina varios modelos simples de forma secuencial) que mejora progresivamente las predicciones corrigiendo errores anteriores. Es eficiente para manejar el desbalance de clases y **alcanzó los mejores resultados en términos de precisión y capacidad de distinguir suscriptores reales**. Se optimizó cuidadosamente mediante búsqueda de parámetros y validación cruzada.
- **K-Nearest Neighbors (KNN):** Este modelo clasifica a un cliente según la cercanía a otros con características similares. Sin embargo, debido a la gran cantidad de variables y el desbalance en los datos, su desempeño fue inferior y se decidió no usarlo en etapas avanzadas.
- **Naive Bayes:** Basado en probabilidades simples y asumiendo independencia entre variables, es rápido y fácil de entrenar. Sin embargo, esa suposición rara vez se cumple en la realidad, lo que afectó su precisión y generó muchos falsos positivos. Fue útil para comparación, pero quedó detrás de modelos como Random Forest y XGBoost.

Comparación de modelos:

La evaluación se realizó usando datos reservados y validación cruzada, priorizando un buen equilibrio entre detección de suscriptores (recall), precisión y capacidad de discriminación (AUC).

- Mejor modelo general: **XGBoost**, con ajuste fino de sus parámetros y umbrales.
- Mayor interpretabilidad: **Regresión logística** y **Árboles de decisión**.
- Buen equilibrio entre rendimiento y explicabilidad: **Random Forest**.

Los resultados detallados y gráficos comparativos se presentan en la Sección 8 del presente informe. Asimismo, vea los resultados graficados en [Anexo F: Métricas extendidas de modelos](#)

6.7. Modelo de Deep Learning

Para comparar distintos enfoques y rendimientos, se implementó un modelo basado en una red neuronal multicapa (MLP). Para esto, se utilizó Keras y TensorFlow, que son herramientas de

software que facilitan la creación y entrenamiento de redes neuronales, haciendo el proceso más sencillo y eficiente.

Esta red está compuesta por:

- Una capa de entrada adaptada al número de variables después del preprocesamiento.
- Dos capas ocultas con varias unidades que aplican funciones que ayudan a la red a aprender patrones complejos en los datos.
- Una capa de salida que entrega una probabilidad para la clasificación entre las dos clases (suscriptor o no).

Para entrenar la red, se utilizó una función que mide el error en problemas de clasificación binaria y un optimizador llamado Adam que ajusta los parámetros para mejorar el desempeño.

Optimización y prevención de sobreajuste:

Para evitar que la red “memorice” los datos de entrenamiento y pierda capacidad para funcionar bien con nuevos casos, se aplicaron técnicas como:

- **Dropout:** apaga aleatoriamente algunas conexiones durante el entrenamiento para que la red no dependa demasiado de ciertas neuronas/unidades de procesamiento.
- **Early Stopping:** detiene el entrenamiento cuando no se observa mejora en la validación para evitar sobreentrenamiento.
- **Batch Normalization:** estabiliza y acelera el proceso de entrenamiento.

Asimismo, se probaron distintas configuraciones de tamaño de grupos de datos (batch size) y cantidad de ciclos de entrenamiento (epochs), seleccionando la combinación que dio mejor rendimiento en datos no vistos.

Resultados y comparación:

El modelo de red neuronal logró un desempeño competitivo, especialmente en la capacidad para identificar correctamente suscriptores (recall) y en la medida de discriminación general (AUC). Sin embargo, fue superado ligeramente por XGBoost en estabilidad y precisión general.

7. Interpretabilidad y Explicabilidad de Modelos

En el ámbito bancario, no alcanza con que un modelo prediga bien: también es clave entender **cómo** y **por qué** llega a cada decisión. Esto genera confianza, ayuda a cumplir con regulaciones sobre decisiones automatizadas y permite detectar posibles sesgos o errores antes de que afecten al negocio.

En este proyecto se aplicaron técnicas para explicar el modelo desde dos perspectivas:

- **Global:** entender la lógica general del modelo de forma integral y total.
- **Local:** explicar casos concretos y por qué se tomó una decisión específica.

7.1. Importancia de variables (Feature Importance)

Modelos como Random Forest y XGBoost permiten calcular qué variables tienen más peso en las predicciones. Este “peso” se mide según cuánto mejora la precisión del modelo cada vez que la variable se usa para dividir la información.

Las variables más influyentes fueron:

- **poutcome** (resultado de campañas anteriores)
- Variables económicas externas como **euribor3m**, **cons.conf.idx** y **cons.price.idx**
- **month** (mes del contacto)
- **duration** (duración de la llamada)
- **job_grouped** y **education_grouped** (categorías de ocupación y educación)
- **campaign_intensity** (intensidad de la campaña, variable derivada)
- **contact** (canal de comunicación)

Asimismo estos hallazgos permiten identificar oportunidades de optimización para campañas futuras:

- El perfil sociodemográfico del cliente tiene un impacto moderado, lo cual sugiere que **la disposición a contratar el producto depende más del contexto e interacción durante la campaña** que del perfil estático del cliente.
- Evitar campañas demasiado insistentes (controlar campaign y pdays).
- Lanzar campañas en momentos macroeconómicamente favorables (euribor3m, cons.conf.idx).

Este análisis refuerza la importancia de combinar información contextual, del cliente y del entorno económico para mejorar la eficacia de las campañas de marketing predictivo. Para observar el gráfico correspondiente, véase [Anexo G: Explicabilidad e interpretabilidad del modelo final tuneado](#)

7.2. Técnicas SHAP y LIME

Asimismo, se usaron dos métodos complementarios:

- **SHAP**: basado en la teoría de juegos, calcula cuánto aporta cada variable de forma global en el proceso de predicción. Por ejemplo, muestra si “duration” aumentó o redujo la probabilidad de que los clientes acepten la oferta y en qué porcentaje.
- **LIME**: crea un modelo simple alrededor de un caso específico (es decir, la predicción sobre un cliente puntual) para explicar localmente la decisión, facilitando que una persona entienda cuáles variables incidieron positiva o negativamente para que ese cliente puntual suscriba o no al depósito.

Se destaca que, SHAP ofrece una visión sistemática y cuantitativa de todas las predicciones (global), mientras que LIME ayuda a validar casos puntuales y detectar inconsistencias (local). Se pueden observar los gráficos correspondientes en [Anexo G: Explicabilidad e interpretabilidad del modelo final tuneado](#)

7.3. Análisis de sesgos y robustez

Se evaluaron posibles sesgos y la capacidad del modelo para adaptarse a cambios en los datos:

- No se usaron variables sensibles como género, aunque se monitoreó que variables como ocupación o educación no generaran sesgos indirectos.
- Se verificó que los modelos manejen bien el desbalance entre clientes que aceptan y los que no. Random Forest y XGBoost fueron los más estables tras ajustar umbrales de decisión.
- Las explicaciones de SHAP y LIME coincidieron, lo que refuerza la confianza en las conclusiones.

En resumen: estas técnicas permitieron comprobar que el sistema predictivo es coherente, justo y explicable, facilitando que los resultados puedan comunicarse claramente a decisores de negocio y áreas regulatorias.

8. Evaluación final y comparación de modelos

8.1. Comparación global de desempeño y selección del modelo óptimo

Tras el entrenamiento de diversos modelos de clasificación y el análisis de su desempeño en términos de rendimiento y explicabilidad, se procedió a la **evaluación global de resultados**. Esta etapa fue clave para **seleccionar el modelo final a utilizar**, considerando no solo métricas cuantitativas, sino también criterios de interpretabilidad, robustez y viabilidad de implementación o uso en un entorno real.

Se analizaron múltiples métricas (precisión, recall, F1-score y AUC) usando datos no vistos durante el entrenamiento, lo que garantiza una evaluación realista.

<u>Modelo</u>	<u>Precisión</u>	<u>Recall</u>	<u>F1-Score</u>	<u>AUC</u>	<u>Observaciones clave</u>
XGBoost	0.48	0.88	0.62	0.945	Mejor balance global, especialmente para identificar clientes que si aceptan la oferta.
Random Forest	0.60	0.56	0.58	0.941	Muy sólido y más simple de explicar que XGBoost.
Regresión Logística	0.45	0.85	0.59	0.933	Fácil de interpretar pero con menor precisión.
Red Neuronal (MLP)	0.44	0.86	0.58	0.932	Buen rendimiento, pero más complejo de explicar.
Árbol de Decisión	0.51	0.57	0.54	0.750	Transparente, pero con menor rendimiento.
Naive Bayes	0.23	0.76	0.35	0.777	Débil precisión; útil como referencia y comparación.
KNN	0.30	0.55	0.39	0.736	Menor rendimiento y alto costo computacional.

Selección final

Se eligió **XGBoost** como modelo principal por:

- Mayor capacidad para detectar clientes que sí suscriben (recall alto).
- Buen equilibrio con la precisión, evitando demasiados falsos positivos.
- Facilidad para aplicar técnicas avanzadas de interpretación como SHAP.
- Robustez ante el desbalance natural de clientes interesados vs. no interesados.

8.2. Optimización del modelo XGBoost

Tras identificar a **XGBoost** como el modelo más efectivo, se realizó un ajuste fino de sus parámetros (hiperparámetros) para maximizar su rendimiento y adaptarlo mejor a los datos. Para ello, se utilizó **GridSearchCV**, una técnica que prueba múltiples combinaciones y selecciona la que ofrece el mejor resultado.

Los ajustes más relevantes fueron:

- **Profundidad moderada de los árboles:** evita que el modelo se sobreajuste y capture solo patrones útiles.
- **Tasa de aprendizaje baja:** permite un aprendizaje más gradual y estable.
- **Mayor número de árboles:** mejora la capacidad de generalización.
- **Muestreo parcial de datos y variables:** reduce sobreajuste y mejora robustez.
- **Ajuste de peso para la clase minoritaria:** mejora la detección de clientes que aceptan la oferta (recall).

Resultados clave finales del modelo optimizado:

- **AUC = 0.95** → Excelente capacidad para distinguir entre clientes que aceptan o no la oferta.
- **Recall = 81%** en clase "Yes" → Alta capacidad para encontrar clientes interesados.
- **Precision = 54%** → Aceptable en campañas donde es más importante captar potenciales que evitar contactos innecesarios.
- **F1 Score = 0.65** → Buen equilibrio entre precisión y recall.
- **Accuracy = 90%** → Alta, aunque influida por el desbalance de clases.

Conclusión: El XGBoost optimizado es una herramienta robusta y precisa para campañas de marketing, maximizando la detección de clientes con alta probabilidad de aceptación, incluso a costa de algunos falsos positivos. Se incluyen los gráficos asociados a este modelo final en [Anexo G: Explicabilidad e interpretabilidad del modelo final tuneado](#)

9. Productivización del modelo

El desarrollo de modelos predictivos no concluye con la evaluación de su rendimiento, sino que requiere su posterior integración en sistemas reales de decisión. La **productivización del modelo** implica preparar los artefactos generados durante el modelado para su uso en entornos operativos, garantizando reproducibilidad, escalabilidad y facilidad de uso por parte de usuarios finales o sistemas automatizados que agreguen valor e información a las decisiones cotidianas de la organización.

9.1. Desarrollo de un pipeline para predecir nuevos casos

Se implementó un pipeline completo y modular en Python, apoyado en la librería **scikit-learn**, que encapsula todas las etapas necesarias para obtener una predicción (replicando los pasos efectuados en la etapa de modelado documentada en el Jupyter Notebook).

El diseño se estructuró en **clases personalizadas** y componentes distribuidos en diferentes archivos .py, que luego son invocados por el archivo principal app.py:

- **Preprocesamiento manual:** mediante la clase PreprocessingTransformer, que aplica reglas definidas (agrupaciones lógicas, limpieza de valores y generación de variables derivadas).
- **Codificación y escalado:** mediante clases OneHotEncoder para las variables categóricas y ManualScaler para las numéricas.
- **Selección de variables:** implementada con una clase FeatureSelector personalizada, que reduce la dimensionalidad tras la codificación.
- **Modelo final:** un clasificador **XGBoost** entrenado para realizar predicciones probabilísticas y guardado como archivo .pkl.

El pipeline recibe registros en el mismo formato de las variables originales, los procesa internamente y devuelve como salida la probabilidad estimada de que un cliente suscriba un depósito a plazo.

Este esquema modular permite:

- Sustituir o actualizar componentes de forma independiente (por ejemplo, reemplazar el modelo o modificar el esquema de codificación).
- Reutilizar clases y funciones en diferentes entornos, tanto en notebooks como en la aplicación backend.

Se optó por un **enfoque híbrido** debido a que:

- Los objetos principales (PreprocessingTransformer, FeatureSelector, etc.) fueron definidos en scripts dedicados (preprocessing.py, feature_selector.py, entre otros).
- El modelo entrenado sí fue serializado, lo que garantiza **reproducibilidad, portabilidad y mantenibilidad**.
- El archivo app.py integra estos componentes, permitiendo la carga y ejecución del pipeline de forma eficiente en el entorno productivo.

Se incluye el código asociado en [Anexo K: Código completo del proyecto](#)

9.2. Diseño de una interfaz simple y un chatbot RAG

Para permitir la interacción de usuarios no técnicos con el modelo, se desarrollaron dos mecanismos complementarios:

Interfaz Web (Flask):

Se construyó una aplicación web ligera con *Flask* que incluye:

- **Endpoint /predict:** recibe datos en formato JSON, procesa internamente el pipeline y devuelve la probabilidad de suscripción de ese cliente.
- **Dashboard básico:** permite ingresar manualmente los datos de un cliente y obtener la predicción en pantalla.

Chatbot con sistema RAG (Retrieval-Augmented Generation):

Se integró un **componente conversacional** que permite responder preguntas sobre:

- El funcionamiento del modelo y su lógica interna.
- Las predicciones para clientes específicos.
- El preprocesamiento y las variables utilizadas.
- Aspectos generales del TFM (modelo, scripts, decisiones técnicas).

El sistema combina:

- Un **modelo de embeddings multilingüe** (paraphrase-multilingual-MiniLM-L12-v2), para transformar tanto consultas como documentos en vectores semánticos.
- Un **modelo de QA en español** (mrm8488/bert-base-spanish-wwm-cased-finetuned-spa-squad2-es), encargado de generar respuestas basadas en la documentación del proyecto (es decir, obtiene la información y contextos a partir de este preciso informe que se está leyendo aquí).
- Un **índice FAISS**, que optimiza la recuperación de fragmentos relevantes para cada consulta.

Este enfoque híbrido asegura que el chatbot **entienda el contexto y brinde explicaciones claras**, lo que mejora la transparencia y accesibilidad del modelo. Se trata de una propuesta innovadora dentro de la ciencia de datos aplicada al negocio bancario, ya que no solo ofrece predicciones, sino también la capacidad de **explicarlas en lenguaje natural**; es decir, para usuarios no técnicos y más orientados al negocio.

Véase [Anexo K: Código completo del proyecto](#) y [Anexo H: Capturas de interfaz productiva](#)

10. Visualización de resultados

La visualización y la interacción con el modelo son componentes esenciales de todo sistema de analítica aplicada. En este proyecto, se optó por desarrollar una **interfaz web propia con capacidades conversacionales**, en lugar de un dashboard tradicional (como podría ser Power BI, Tableau o Streamlit).

10.1 Interfaz Web y Frontend con Chatbot

La aplicación se implementó en **Flask** y ofrece dos funcionalidades principales, accesibles desde un frontend HTML/JS:

10.1.1 Interacción directa con el modelo predictivo

- Los usuarios pueden ingresar los datos de un cliente a través de un formulario web.
- El backend recibe los datos en JSON mediante el endpoint /predict (app.py).
- El pipeline productivo procesa los registros con los siguientes pasos:
 - Preprocesamiento de variables.
 - Codificación y escalado de columnas con OneHotEncoderTransformer y ManualScaler.
 - Selección de variables con FeatureSelector.
 - Predicción probabilística con el modelo XGBoost entrenado.
- La respuesta incluye la predicción y su probabilidad para los datos registrados en la pantalla (el cual posee una barra con coloración del rojo al verde según más cerca del NO o SÍ para suscribir al depósito a plazo se encuentre la predicción).

Véase [Anexo H: Capturas de interfaz productiva](#)

10.1.2 Chatbot explicativo con RAG (Retrieval-Augmented Generation)

- Luego de la etapa de predicción, se integra un componente conversacional que combina:
 - **Índice semántico** construido con *sentence-transformers* y FAISS (`build_rag_index.py`).
 - **Modelo de Question Answering** en español basado en BERT (`mrm8488/bert-base-spanish-wwm-cased-finetuned-spa-squad2-es`), que devuelve respuestas concisas a partir del índice semántico.
- El endpoint `/rag_chat` (`chat_rag_local.py`) permite al usuario consultar (en el mismo espacio del chatbot) sobre, por ejemplo:
 - Funcionamiento general del modelo.
 - Predicciones individuales y su interpretación.
 - Variables utilizadas y su importancia.
 - Desarrollo del TFM y técnicas aplicadas.
- El sistema, por ende, genera **respuestas fundamentadas** combinando la información del índice y generación de texto, mejorando la transparencia y accesibilidad del modelo.

Véase [Anexo H: Capturas de interfaz productiva](#)

10.2 Simulación de escenarios: segmentaciones y decisiones de campaña

Se realizaron simulaciones controladas para explorar cómo responde el modelo ante distintos perfiles de cliente:

- **Identificación de segmentos prioritarios:** por ejemplo, clientes adultos con estudios superiores, sin préstamos vigentes y con historial positivo de contacto previo.
- **Análisis de atributos modificables:** canal de contacto (*cellular vs telephone*), mes de contacto, número de interacciones anteriores.
- **Evaluación de estrategias de campaña:** simulando cambios en tipo de contacto o frecuencia, se estimó el efecto esperado en la tasa de conversión.

Estas simulaciones proporcionan información valiosa para la planificación de campañas, orientando recursos hacia los clientes más propensos a contratar el producto.

Véase [Anexo H: Capturas de interfaz productiva](#)

11. Conclusiones y líneas futuras

Este trabajo permitió desarrollar un sistema integral que combina **aprendizaje automático** y **técnicas de recuperación aumentada de información (RAG)** para anticipar la suscripción a depósitos a plazo en campañas de marketing bancario. La solución no se limita a generar predicciones, sino que además ofrece un **chatbot explicativo** que permite interpretar y comunicar los resultados de forma clara para usuarios no técnicos. Esta integración constituye una **contribución diferenciadora** dentro del ámbito académico y práctico, ya que aporta no solo

capacidad predictiva, sino también **transparencia, accesibilidad y valor comunicacional** para la toma de decisiones.

11.1. Principales hallazgos técnicos

- **Mejor modelo:** XGBoost se identificó como el algoritmo con mayor rendimiento global (recall, F1-score y AUC), superando tanto a modelos clásicos como a redes neuronales.
- **Pipeline modular y reproducible:** con preprocesamiento estructurado, codificación y selección de características automatizadas, lo que facilita mantenimiento y escalabilidad.
- **Interpretabilidad:** uso de SHAP y LIME para explicar decisiones a nivel global y local.
- **Accesibilidad:** desarrollo de una interfaz web y un **chatbot explicativo con RAG**, que convierte el modelo en una herramienta interactiva y pedagógica, capaz de presentar las predicciones como así también responder preguntas tanto técnicas como de negocio.

11.2. Principales hallazgos de negocio

- **Segmentos prioritarios:** identificación de perfiles con mayor propensión a contratar, como adultos con educación superior, sin préstamos vigentes y con historial positivo en campañas previas.
- **Variables clave:** confirmación de la relevancia de *duration*, *poutcome*, *month*, *contact*, *job_grouped* y las variables macroeconómicas (*euribor3m*, *cons.conf.idx* y *cons.price.idx*).
- **Valor práctico:** el trabajo demuestra que un enfoque basado en datos, complementado con un sistema explicativo, mejora la eficiencia de campañas y posibilita decisiones más informadas en entornos bancarios reales. En particular, el banco puede:
 - **Priorizar recursos** en clientes con alta probabilidad de aceptación.
 - **Reducir costos operativos**, al disminuir las llamadas innecesarias a clientes con baja propensión de contratación.
 - **Optimizar el timing de las campañas**, considerando tanto el contexto macroeconómico como la estacionalidad de los contactos.

En suma, el sistema propuesto no solo predice la probabilidad de conversión, sino que además entrega **insights accionables para la gestión comercial**, facilitando una asignación más eficiente de recursos y un incremento potencial de la rentabilidad de las campañas.

11.3. Limitaciones

- **Datos históricos acotados:** el dataset se basa en campañas antiguas (2008–2010), lo que puede reducir la aplicabilidad inmediata sin reentrenamiento con datos más actualizados.
- **Desbalance de clases:** pese a estrategias de mitigación, la baja proporción de respuestas positivas limita la sensibilidad del sistema.
- **Aspectos regulatorios:** no se abordaron temas de privacidad y cumplimiento normativo que serían esenciales en una implementación bancaria real.

11.4. Líneas futuras de trabajo

- **Recolección de nuevos datos:** incorporar información más reciente y enriquecida, incluyendo variables de comportamiento digital y socioeconómicas externas.
- **Aprendizaje continuo (MLOps):** implementar un *feedback loop* que capture resultados reales de campañas y actualice periódicamente el modelo.
- **Despliegue y monitoreo productivo:** llevar el sistema a la nube o infraestructura del banco, con alertas ante *data drift* (*desviaciones con los datos usados para entrenar el modelo que puedan indicar la necesidad de ajustar o incluso reentrenar el modelo*).

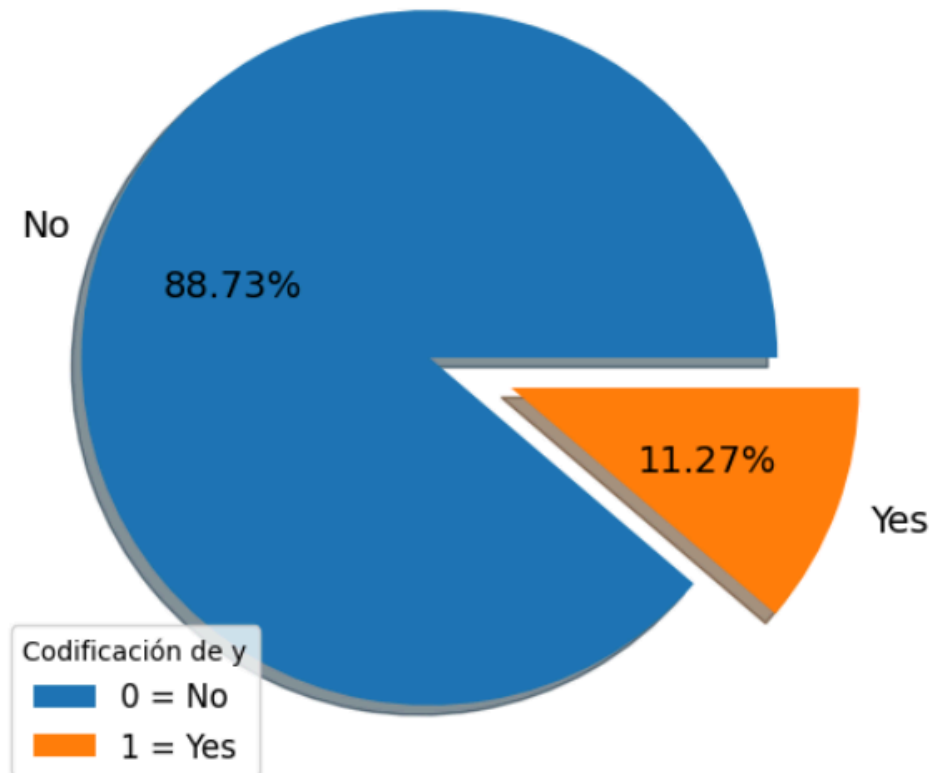
- **Ampliación del chatbot RAG:** incorporar respuestas sobre campañas históricas, segmentos óptimos y recomendaciones personalizadas para equipos comerciales; potenciando las capacidades semánticas y (como resultado) interactivas de la solución para con los usuarios finales. En este punto sería ideal el reemplazo de un modelo de preguntas-respuestas (como el actual) por un modelo generativo entrenado y ajustado específicamente para el contexto y dominio de nuestra entidad bancaria.

Con estas mejoras, la solución evolucionaría hacia un **sistema predictivo y explicativo inteligente**, adaptable a cambios del entorno, con capacidad de aprendizaje continuo y alineado con los objetivos estratégicos de las instituciones financieras en un contexto cada vez más digitalizado.

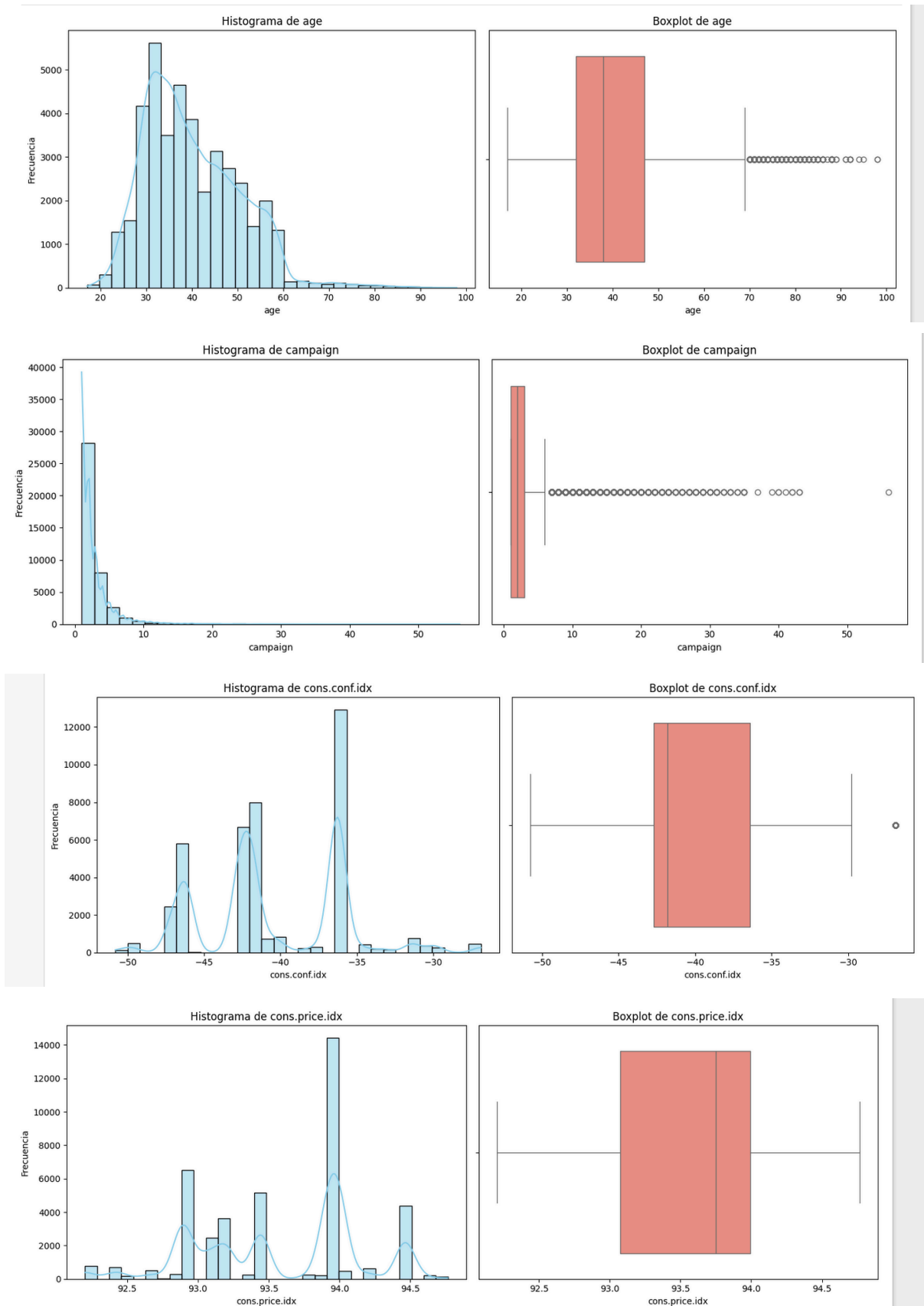
ANEXOS

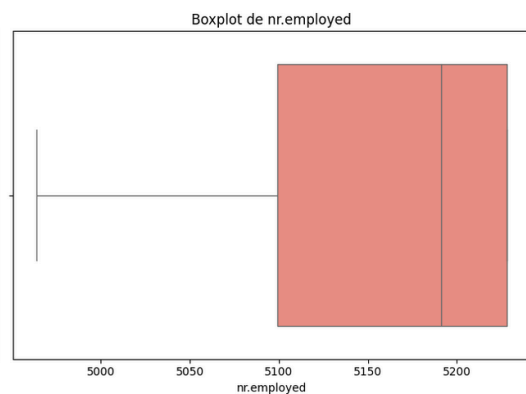
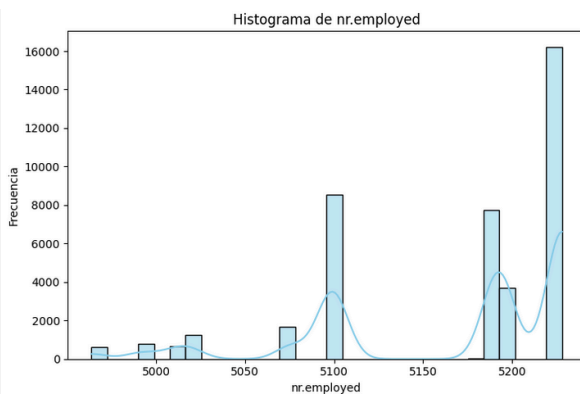
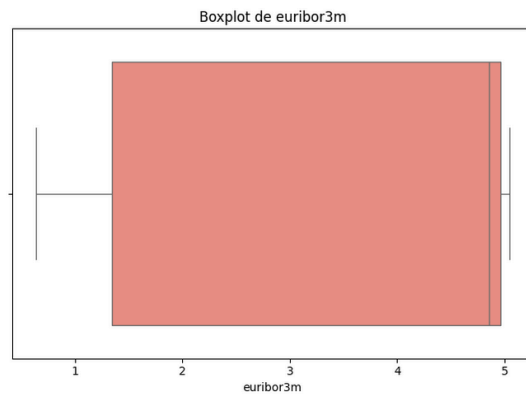
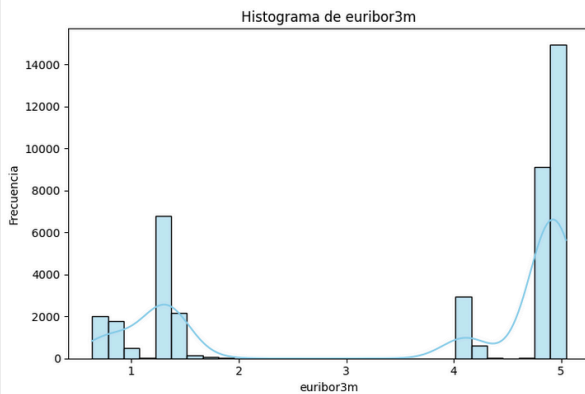
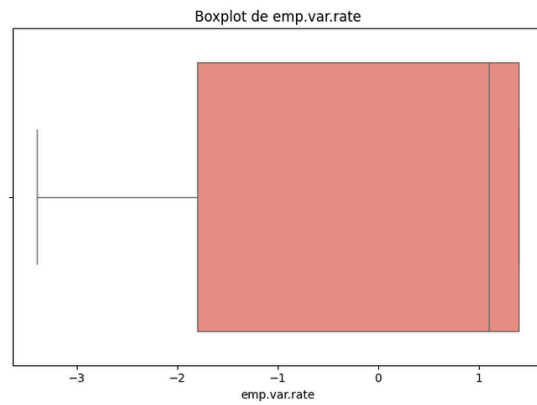
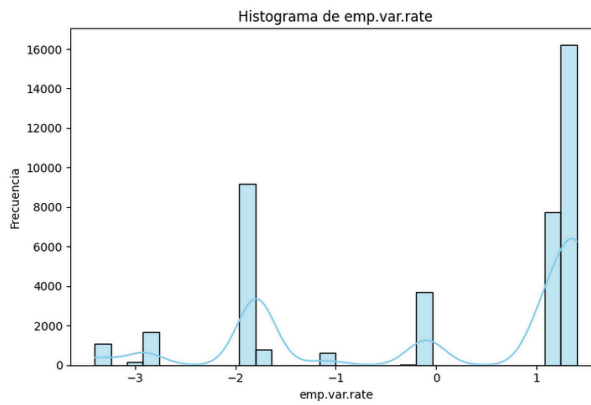
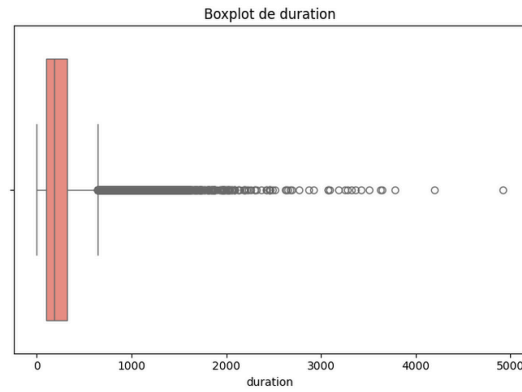
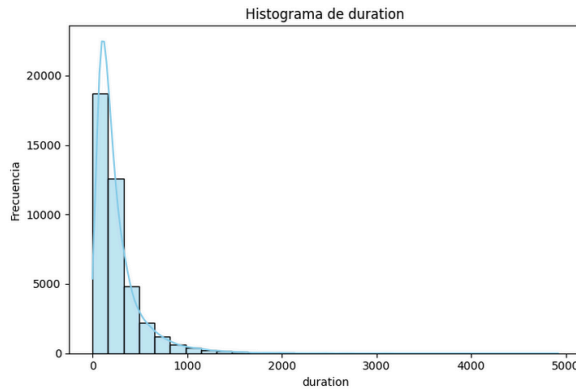
a. Distribución de variable objetivo

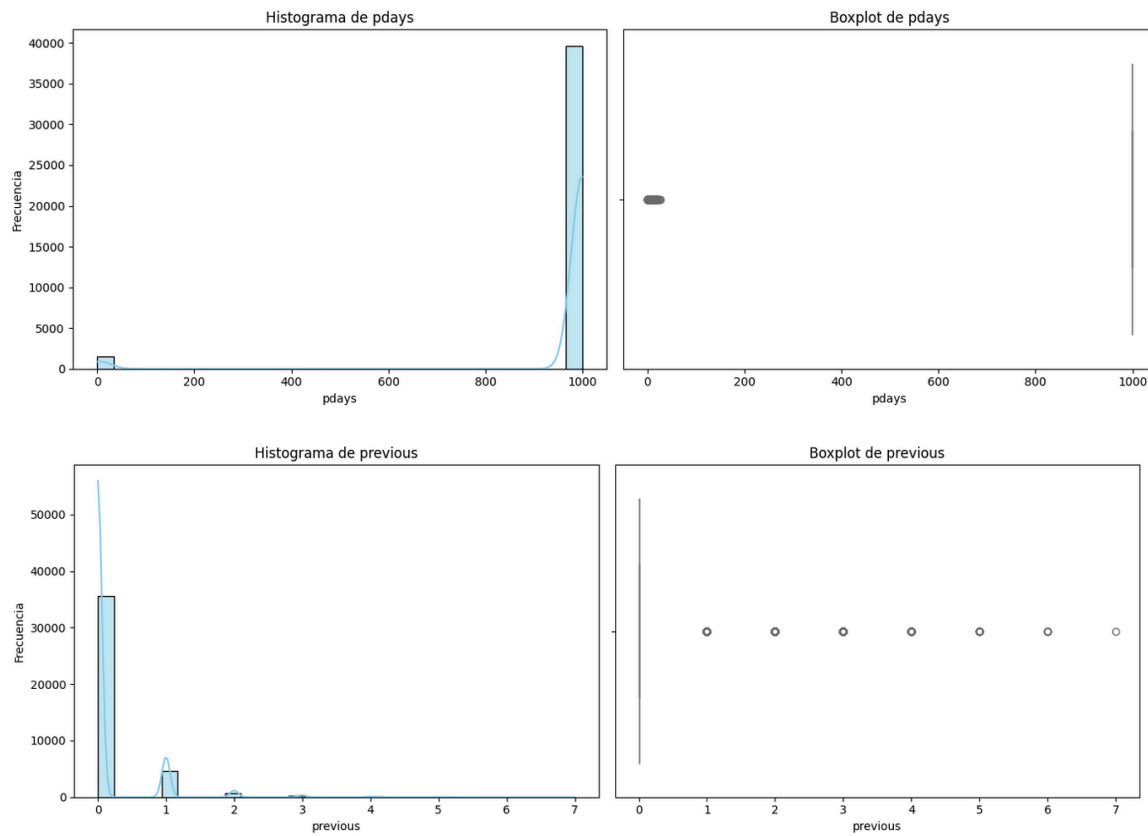
Distribución de la variable objetivo "y"



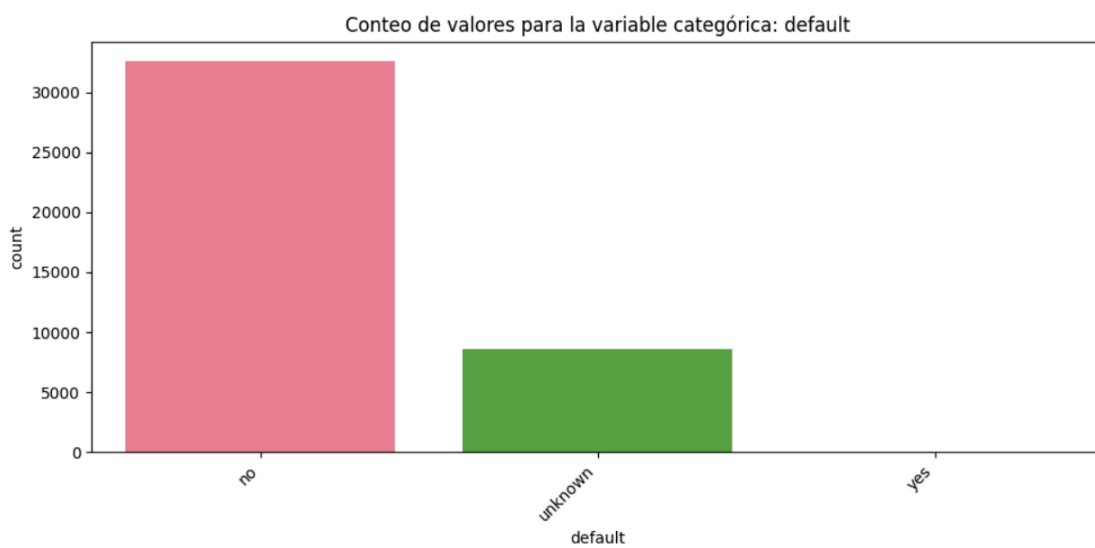
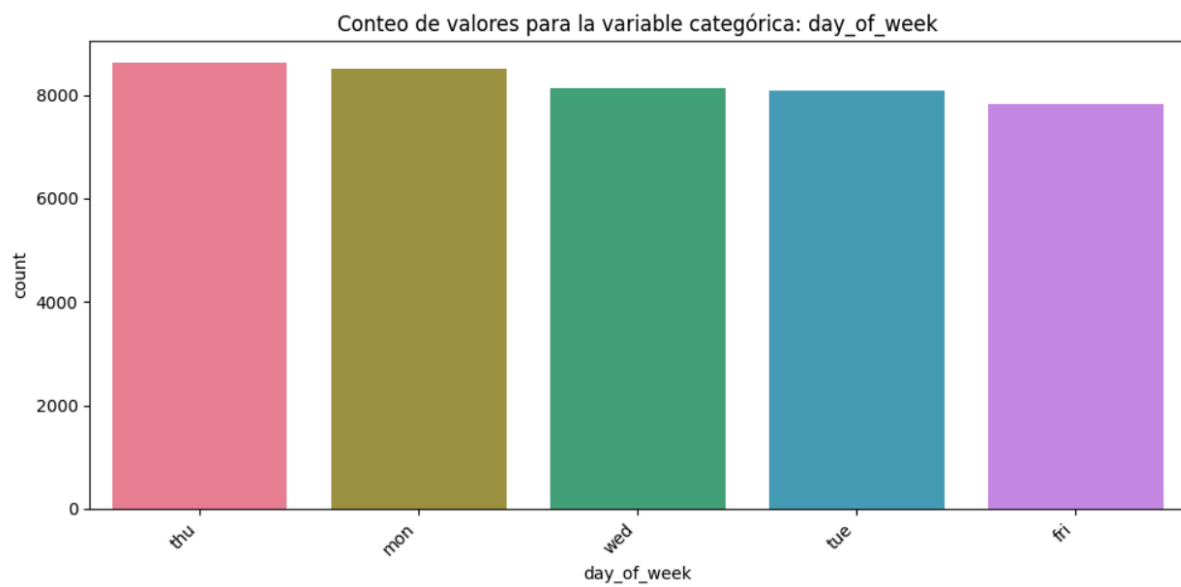
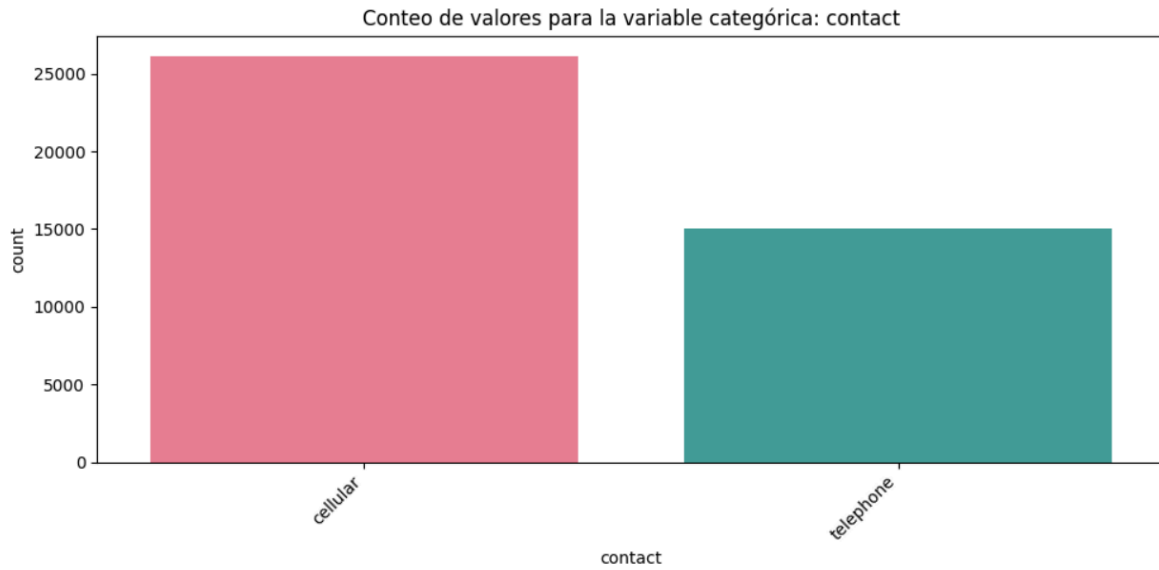
b. EDA variables numéricas

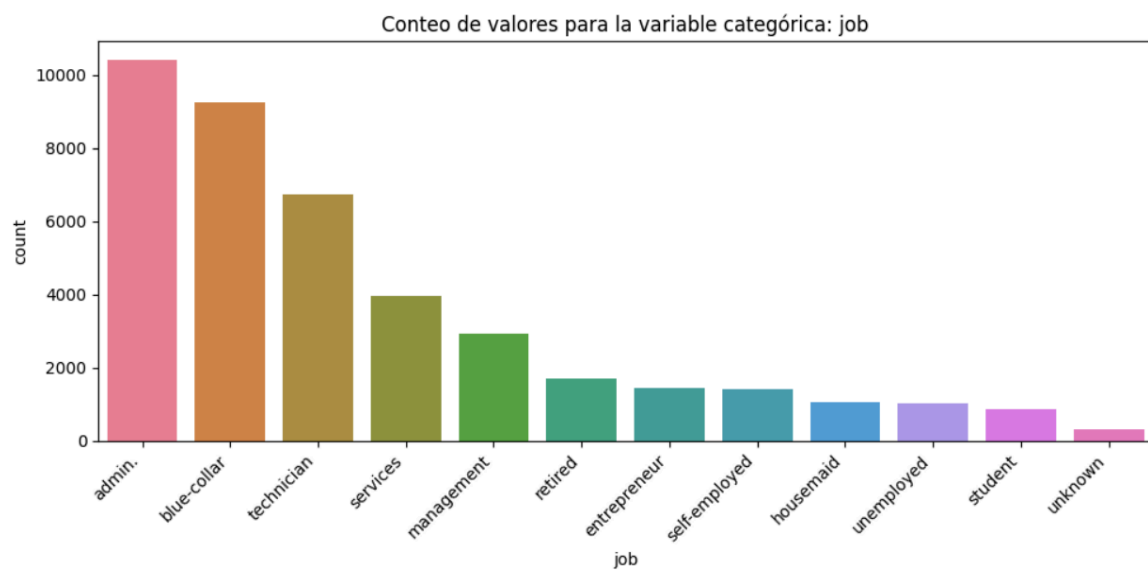
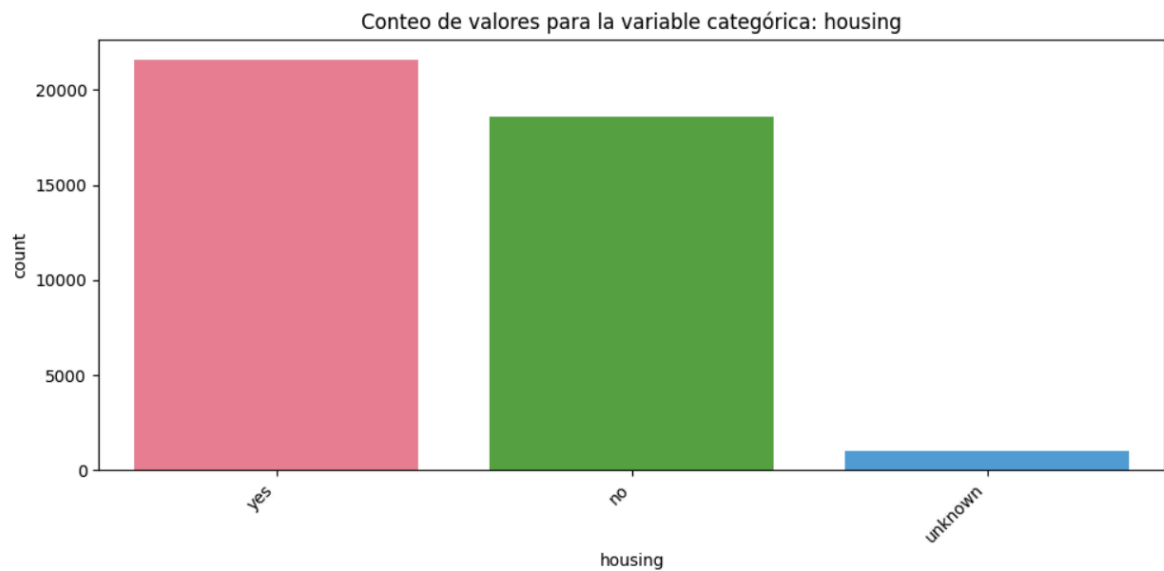
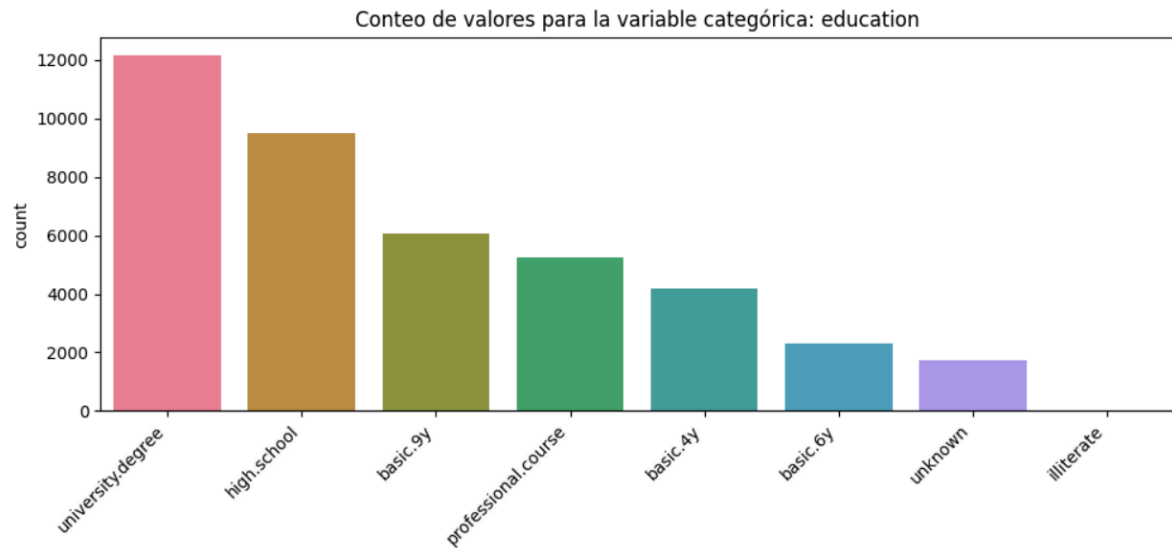


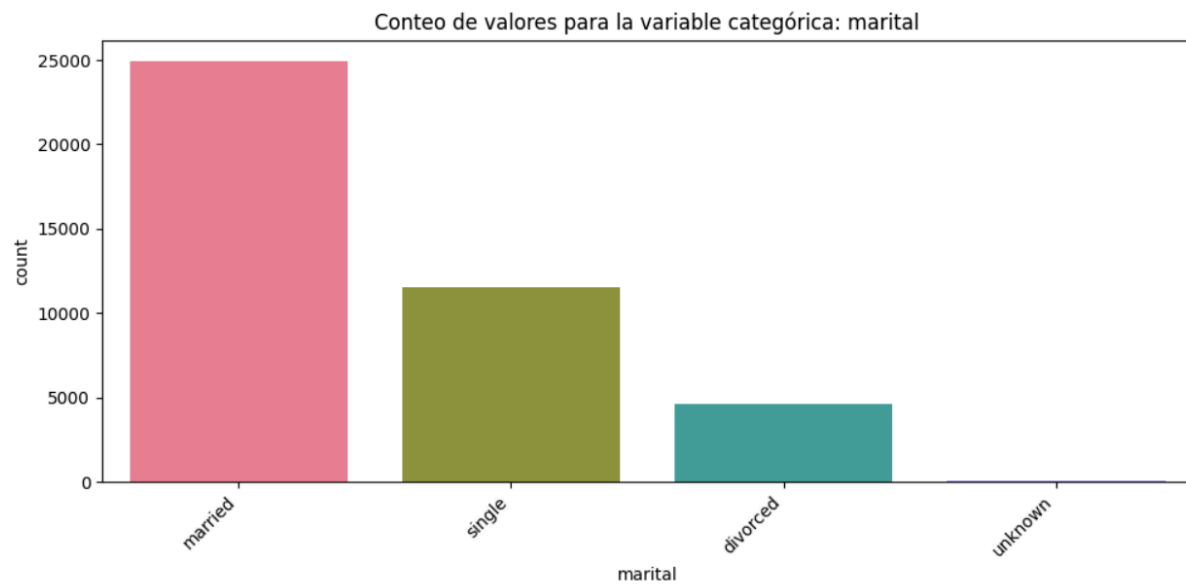
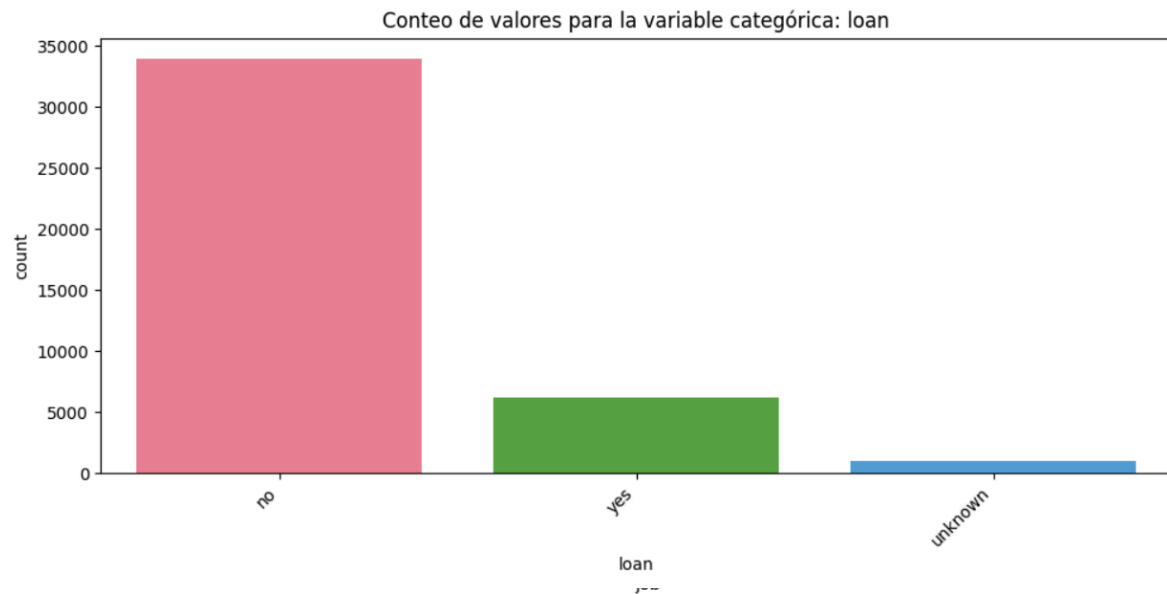


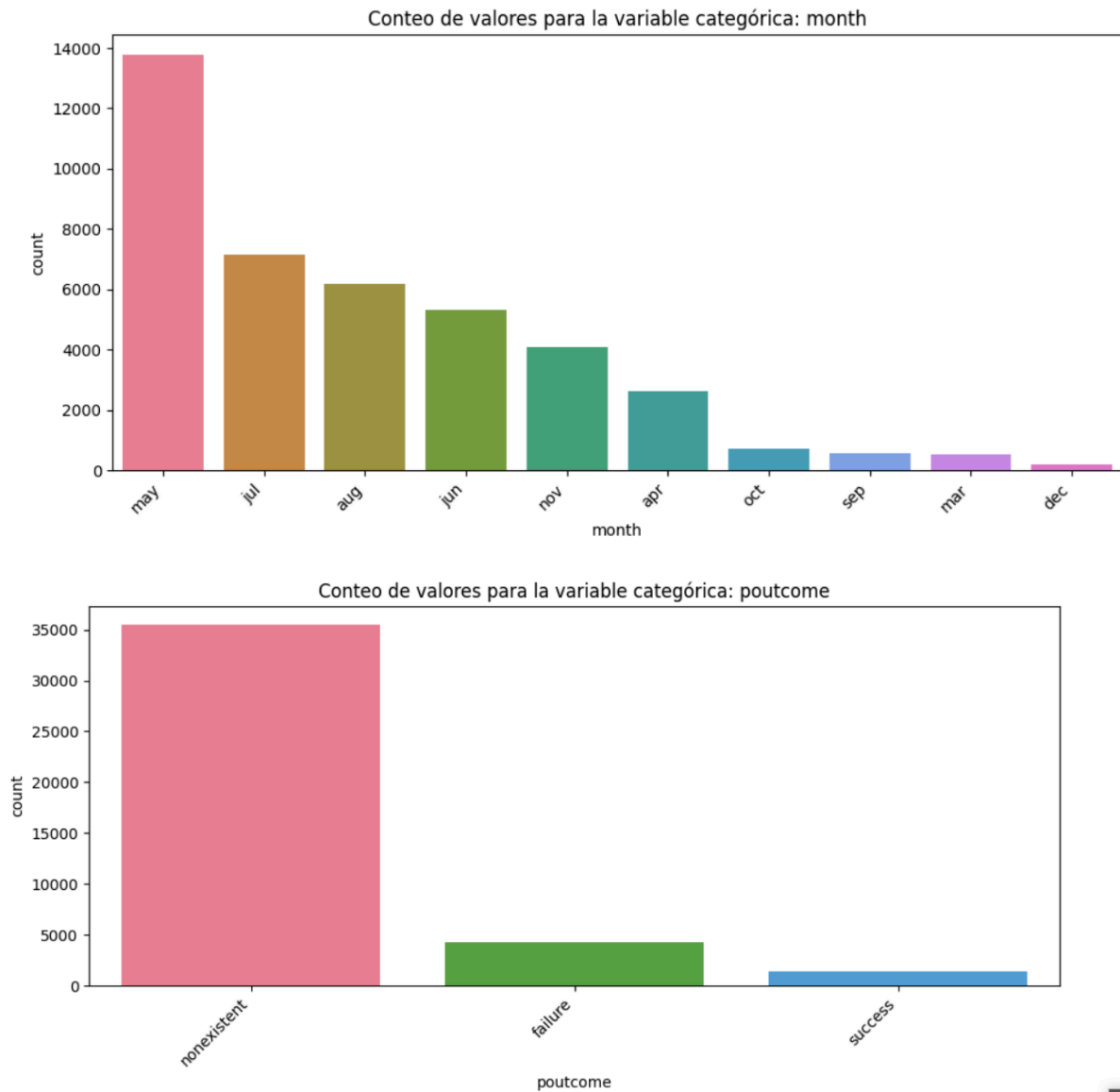


c. EDA variables categóricas

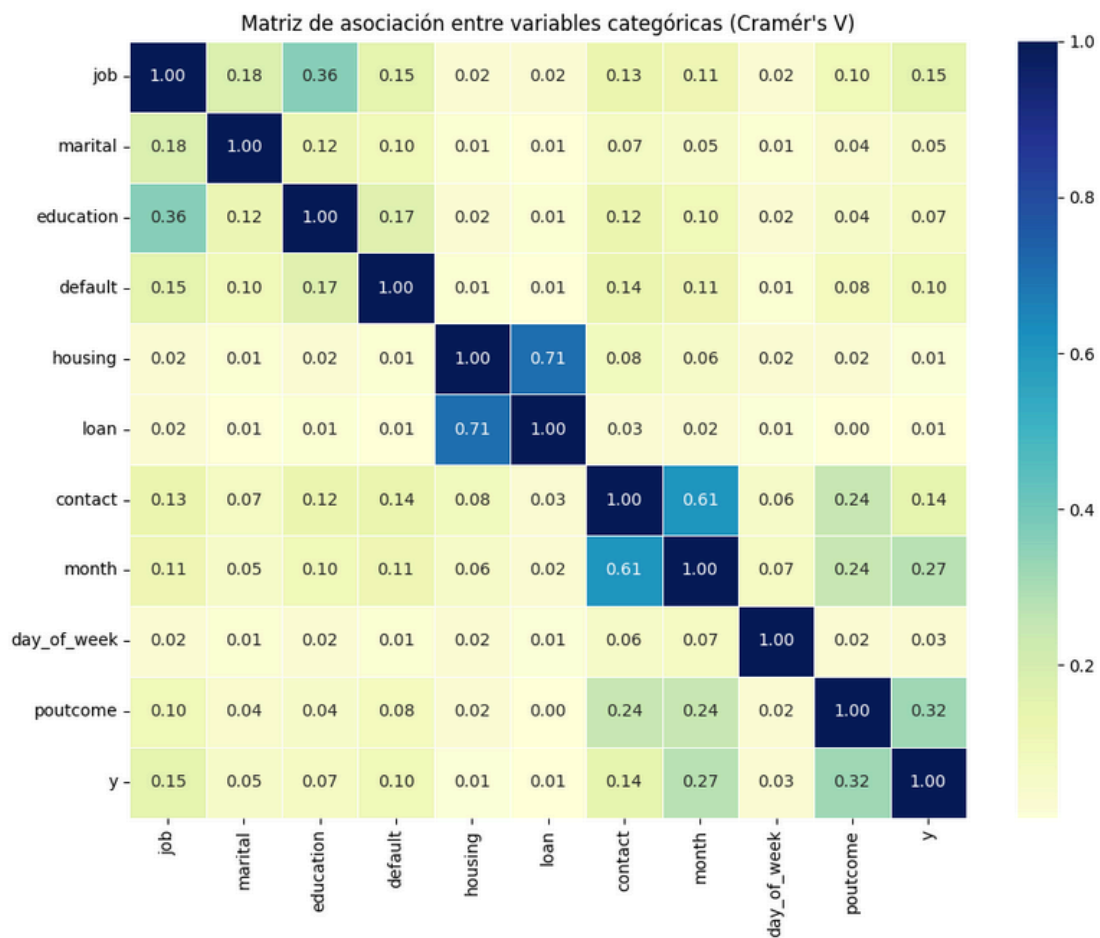
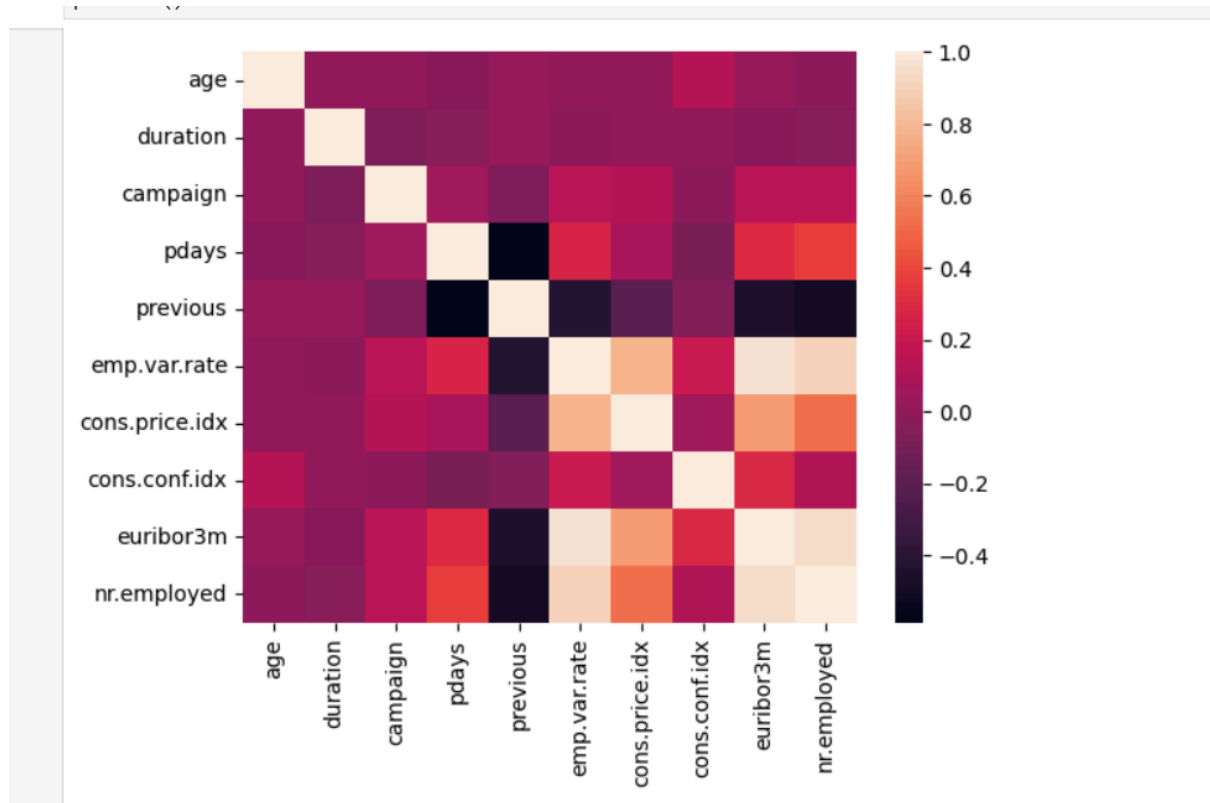


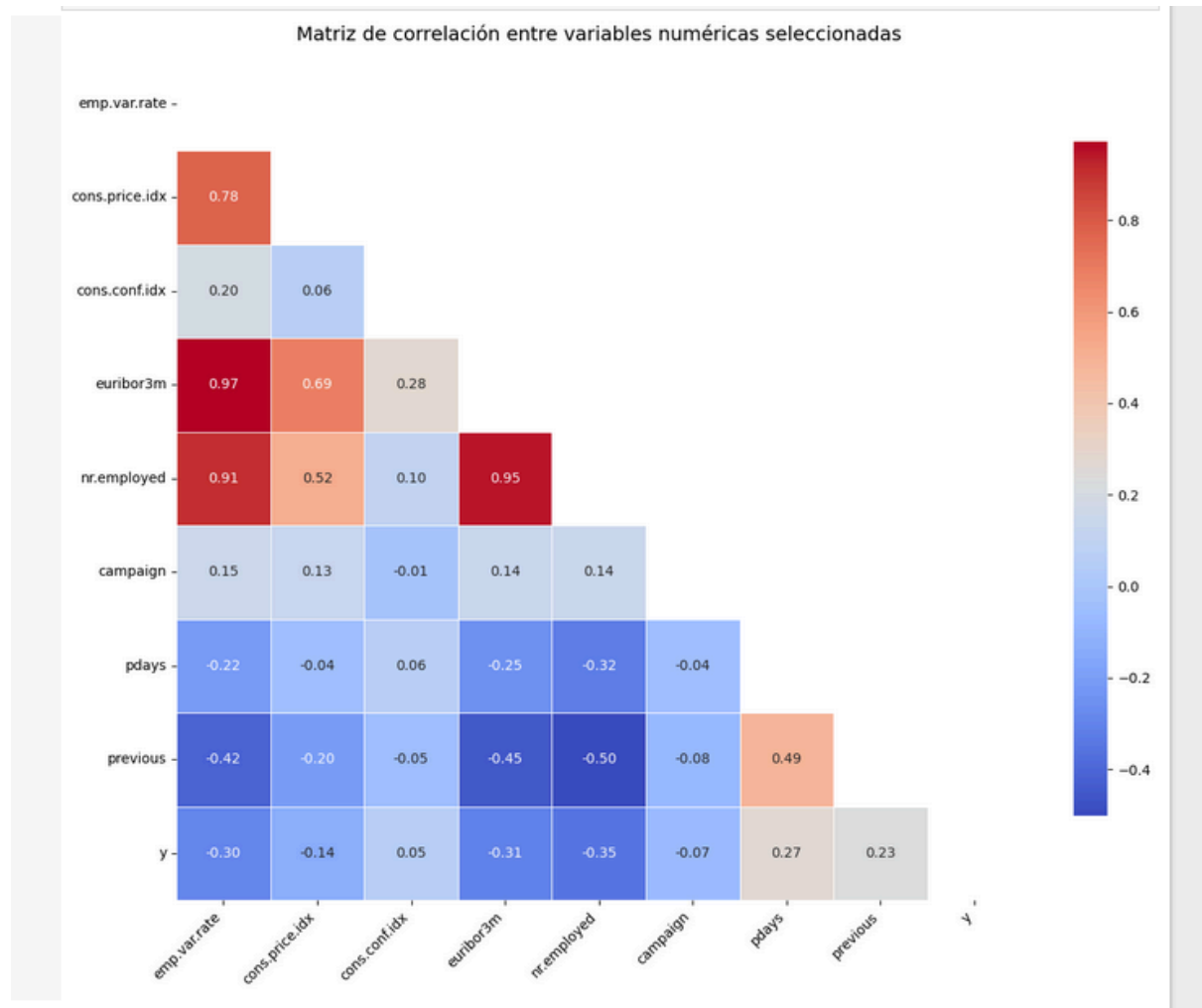




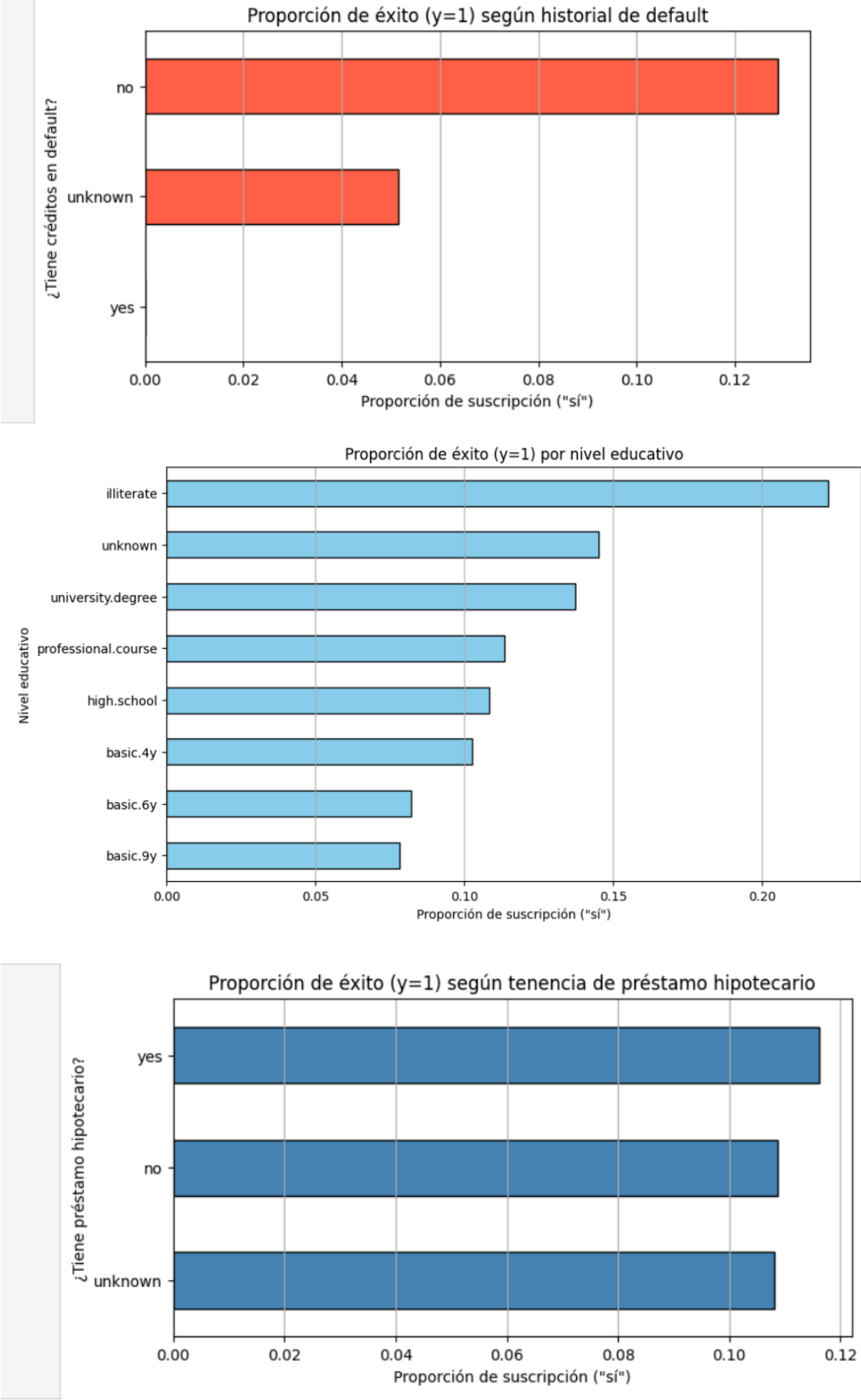


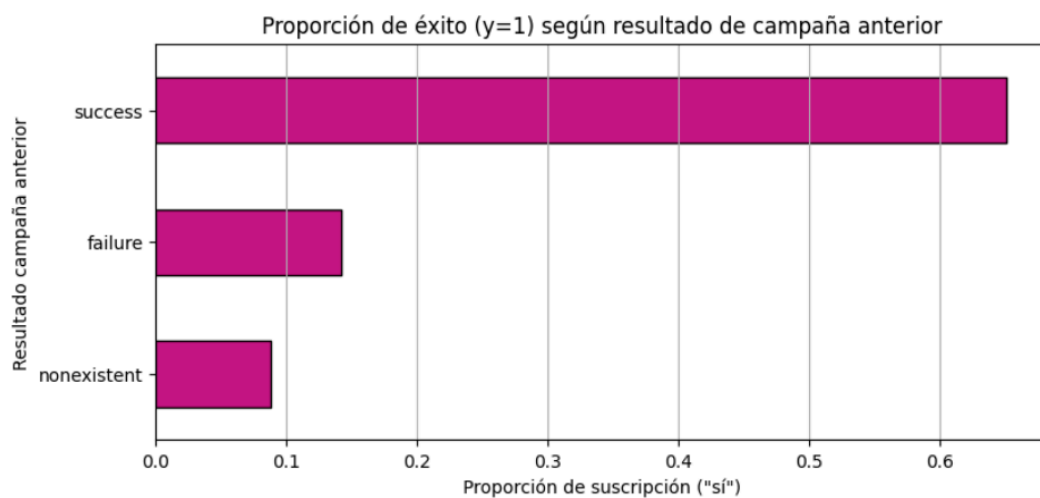
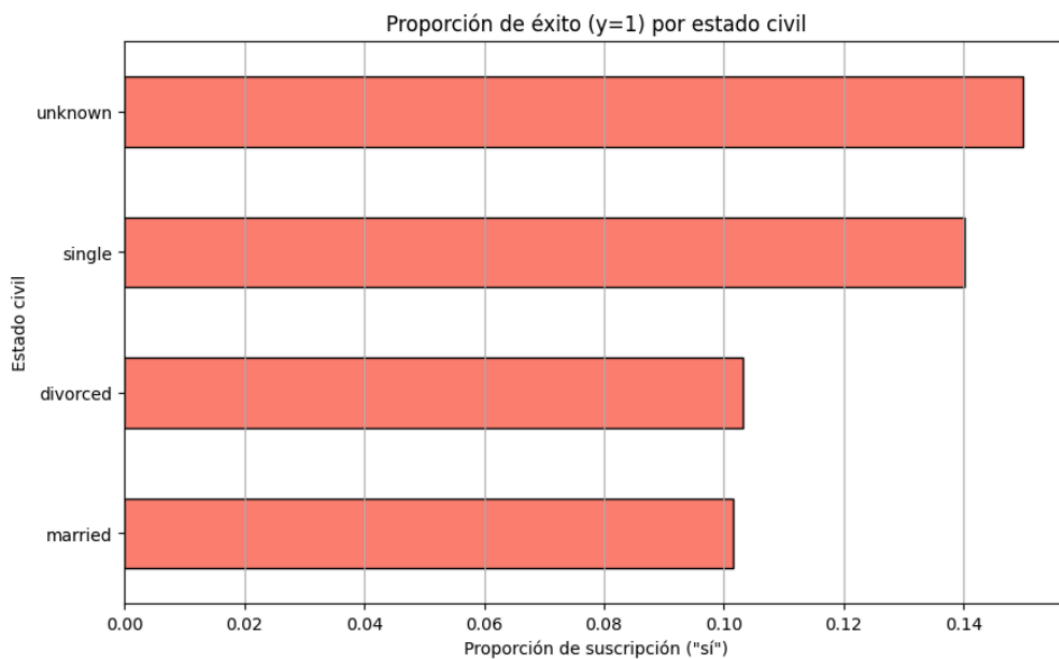
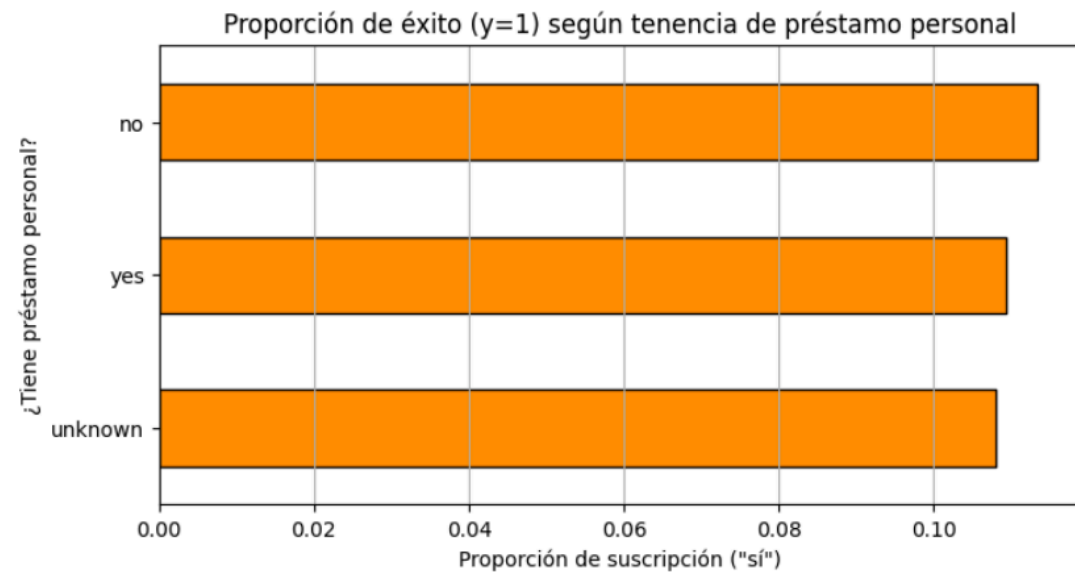
d. Gráficos complementarios del EDA

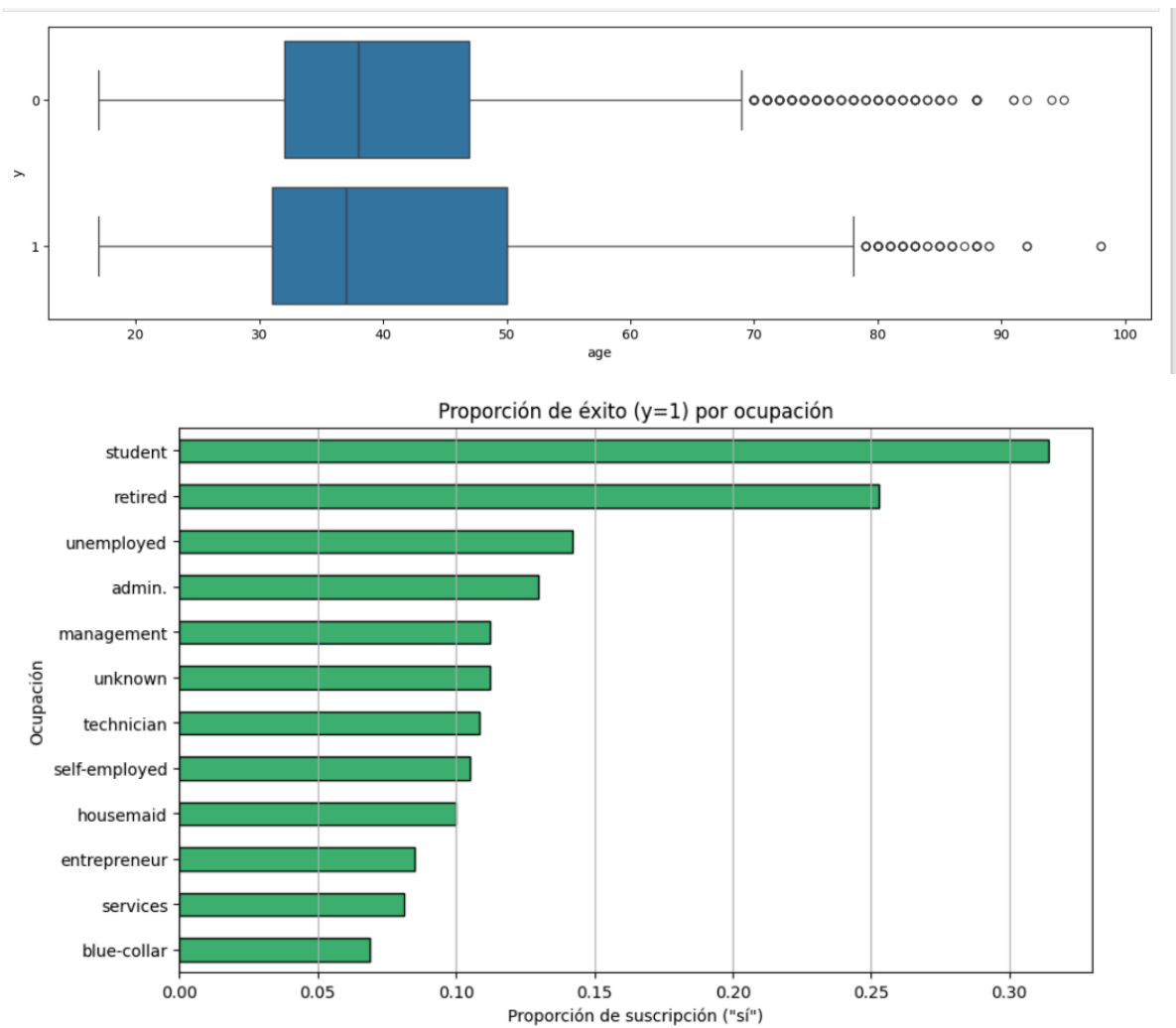




e. Segmentación por atributo clave







f. Métricas extendidas de modelos

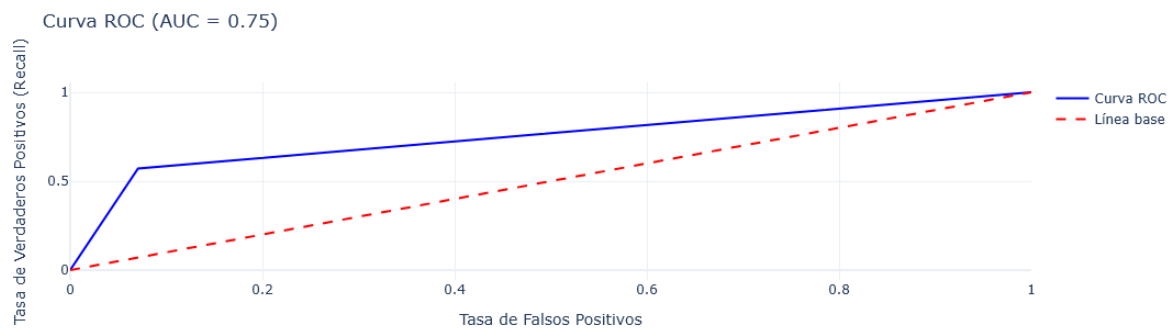

```
=====
Modelo: Decision Tree
=====
Matriz de Confusión:
[[6792 516]
 [ 398 530]]

Classification Report:
      precision    recall  f1-score   support

     No       0.94      0.93      0.94      7308
     Yes       0.51      0.57      0.54       928

 accuracy      0.89      8236
 macro avg     0.73      0.75      0.74      8236
weighted avg     0.90      0.89      0.89      8236

Accuracy: 0.889
Precision: 0.5067
Recall: 0.5711
F1 Score: 0.537
AUC: 0.7503
```



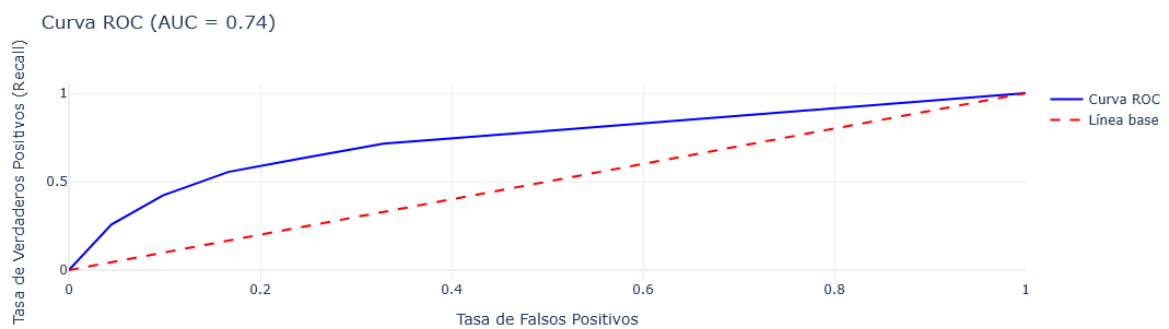
```
=====
Modelo: K-Nearest Neighbors
=====
Matriz de Confusión:
[[6090 1218]
 [ 413 515]]

Classification Report:
      precision    recall  f1-score   support

     No       0.94      0.83      0.88      7308
     Yes       0.30      0.55      0.39       928

 accuracy      0.80      8236
 macro avg     0.62      0.69      0.63      8236
weighted avg     0.86      0.80      0.83      8236

Accuracy: 0.802
Precision: 0.2972
Recall: 0.555
F1 Score: 0.3871
AUC: 0.7359
```



Modelo: Logistic Regression

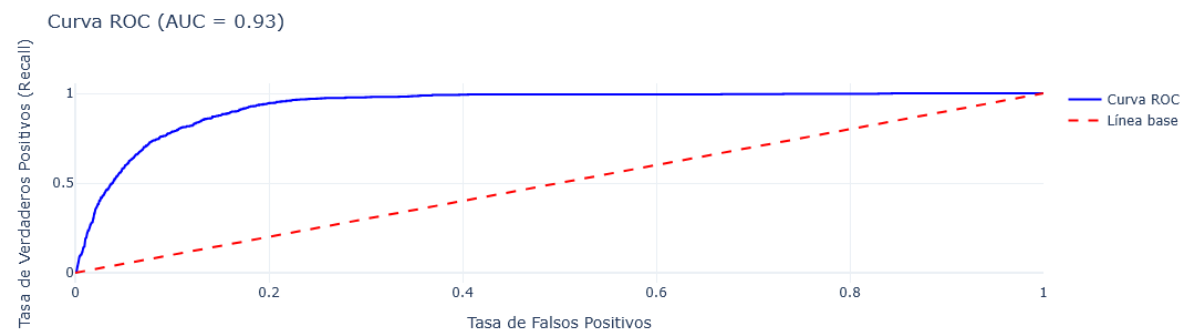
Matriz de Confusión:

```
[[6338  970]
 [ 135 793]]
```

Classification Report:

	precision	recall	f1-score	support
No	0.98	0.87	0.92	7308
Yes	0.45	0.85	0.59	928
accuracy			0.87	8236
macro avg	0.71	0.86	0.75	8236
weighted avg	0.92	0.87	0.88	8236

Accuracy: 0.8658
Precision: 0.4498
Recall: 0.8545
F1 Score: 0.5894
AUC: 0.933



Modelo: Naive Bayes

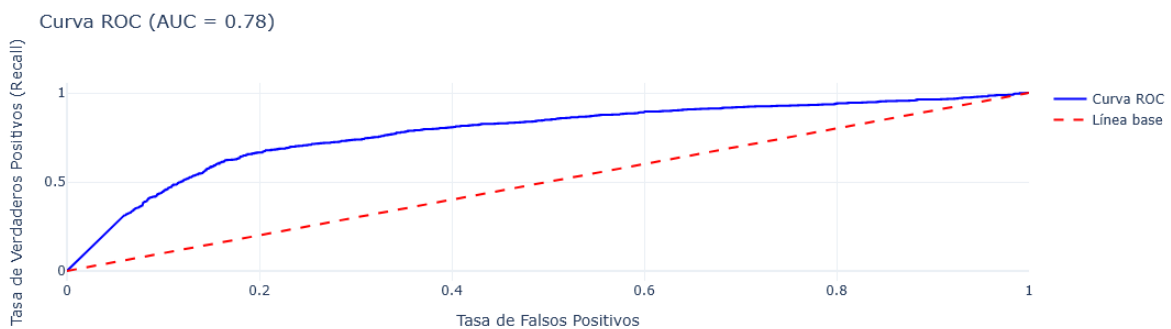
Matriz de Confusión:

```
[[4930 2378]
 [ 227 701]]
```

Classification Report:

	precision	recall	f1-score	support
No	0.96	0.67	0.79	7308
Yes	0.23	0.76	0.35	928
accuracy			0.68	8236
macro avg	0.59	0.71	0.57	8236
weighted avg	0.87	0.68	0.74	8236

Accuracy: 0.6837
Precision: 0.2277
Recall: 0.7554
F1 Score: 0.3499
AUC: 0.7767



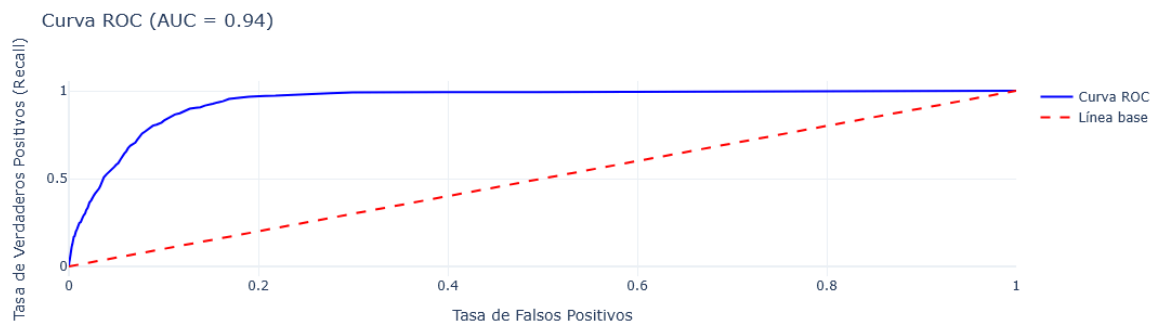
```
=====
Modelo: Random Forest
=====
Matriz de Confusión:
[[6971 337]
 [ 412 516]]

Classification Report:
      precision    recall  f1-score   support

     No       0.94      0.95      0.95      7308
     Yes       0.60      0.56      0.58       928

 accuracy      0.91      0.91      0.91      8236
 macro avg     0.77      0.75      0.76      8236
 weighted avg   0.91      0.91      0.91      8236

Accuracy: 0.9091
Precision: 0.6049
Recall: 0.556
F1 Score: 0.5794
AUC: 0.941
```



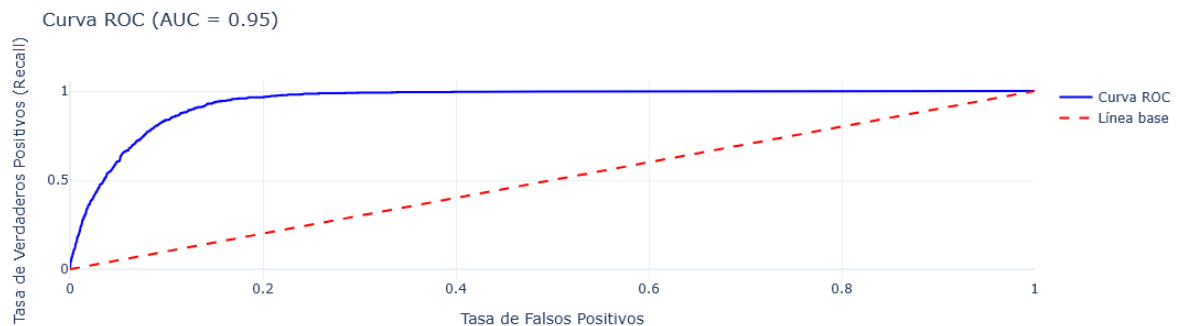
```
=====
Modelo: XGBoost
=====
Matriz de Confusión:
[[6435 873]
 [ 111 817]]

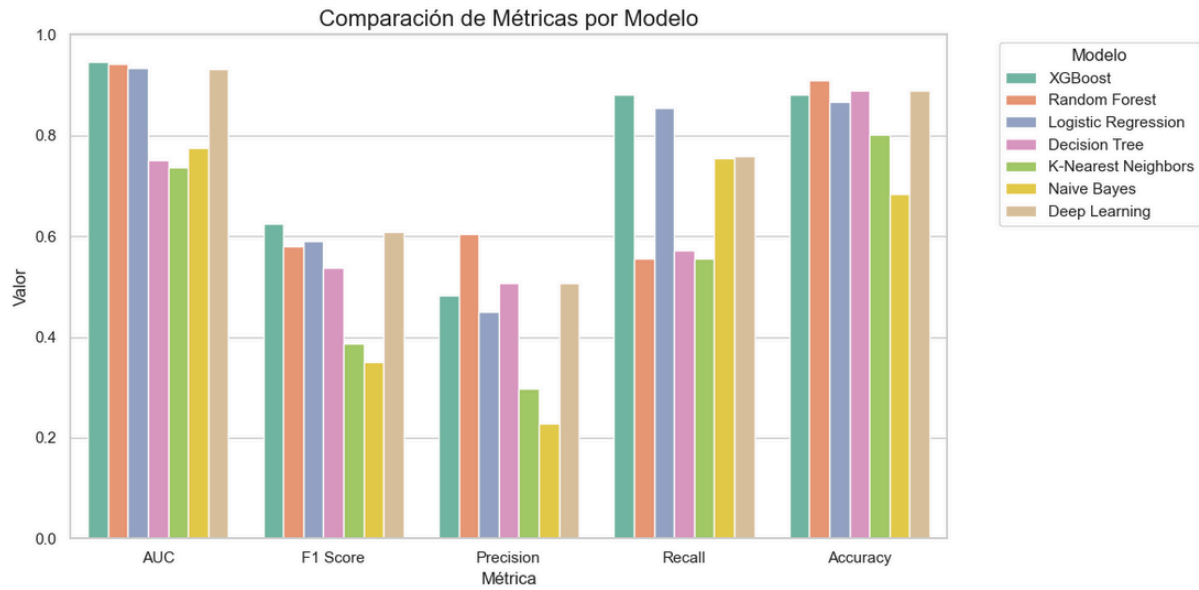
Classification Report:
      precision    recall  f1-score   support

     No       0.98      0.88      0.93      7308
     Yes       0.48      0.88      0.62       928

 accuracy      0.88      0.88      0.88      8236
 macro avg     0.73      0.88      0.78      8236
 weighted avg   0.93      0.88      0.89      8236

Accuracy: 0.8805
Precision: 0.4834
Recall: 0.8804
F1 Score: 0.6241
AUC: 0.945
```





g. Explicabilidad e interpretabilidad del modelo final tuneado

◆ Matriz de Confusión:

```
[[6664  644]
 [ 179  749]]
```

◆ Classification Report:

	precision	recall	f1-score	support
No	0.97	0.91	0.94	7308
Yes	0.54	0.81	0.65	928
accuracy			0.90	8236
macro avg	0.76	0.86	0.79	8236
weighted avg	0.92	0.90	0.91	8236

◆ Accuracy: 0.9001

◆ Precision: 0.5377

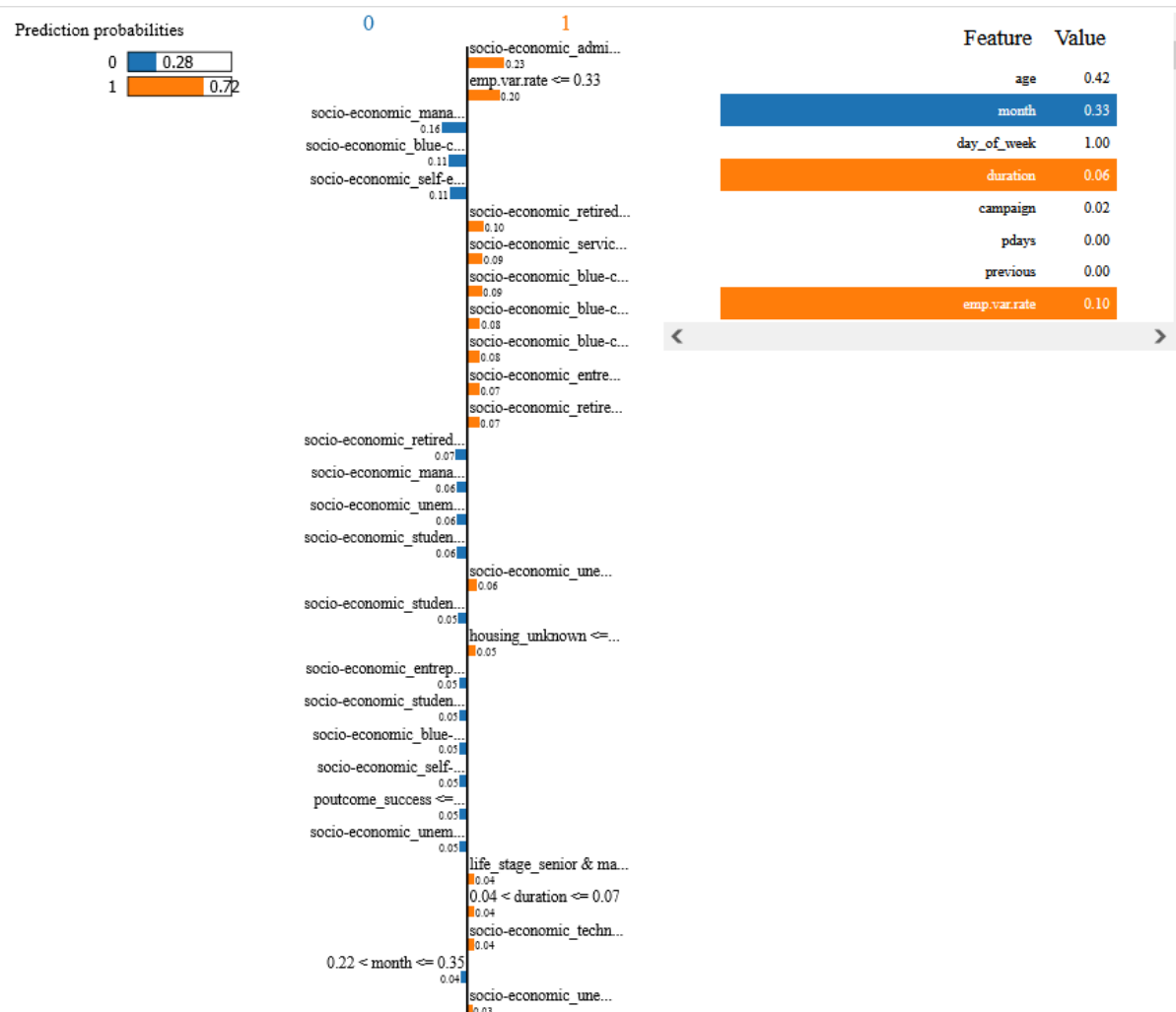
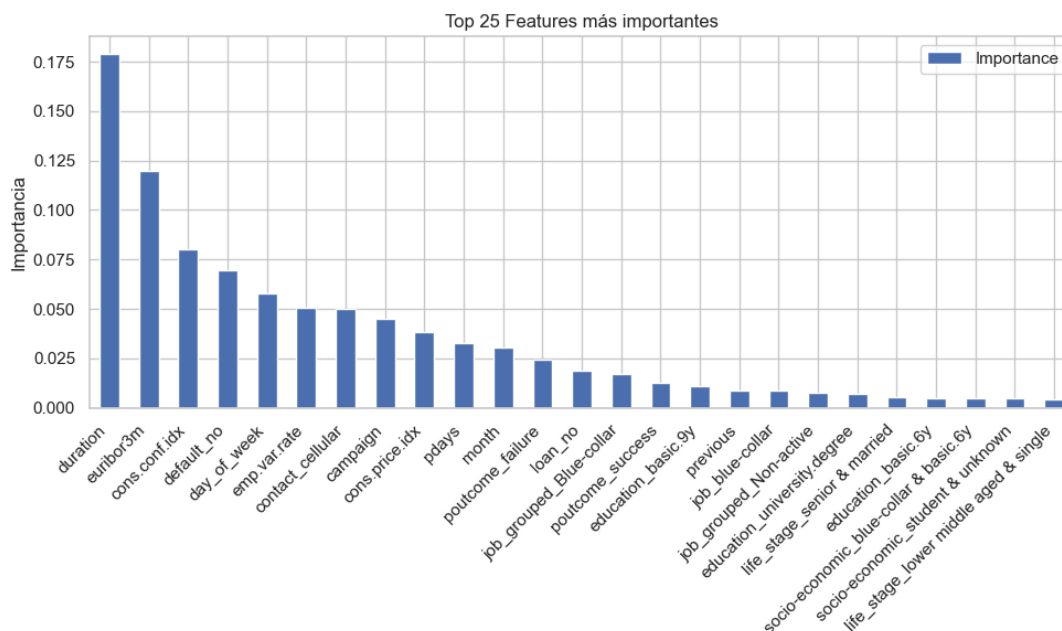
◆ Recall: 0.8071

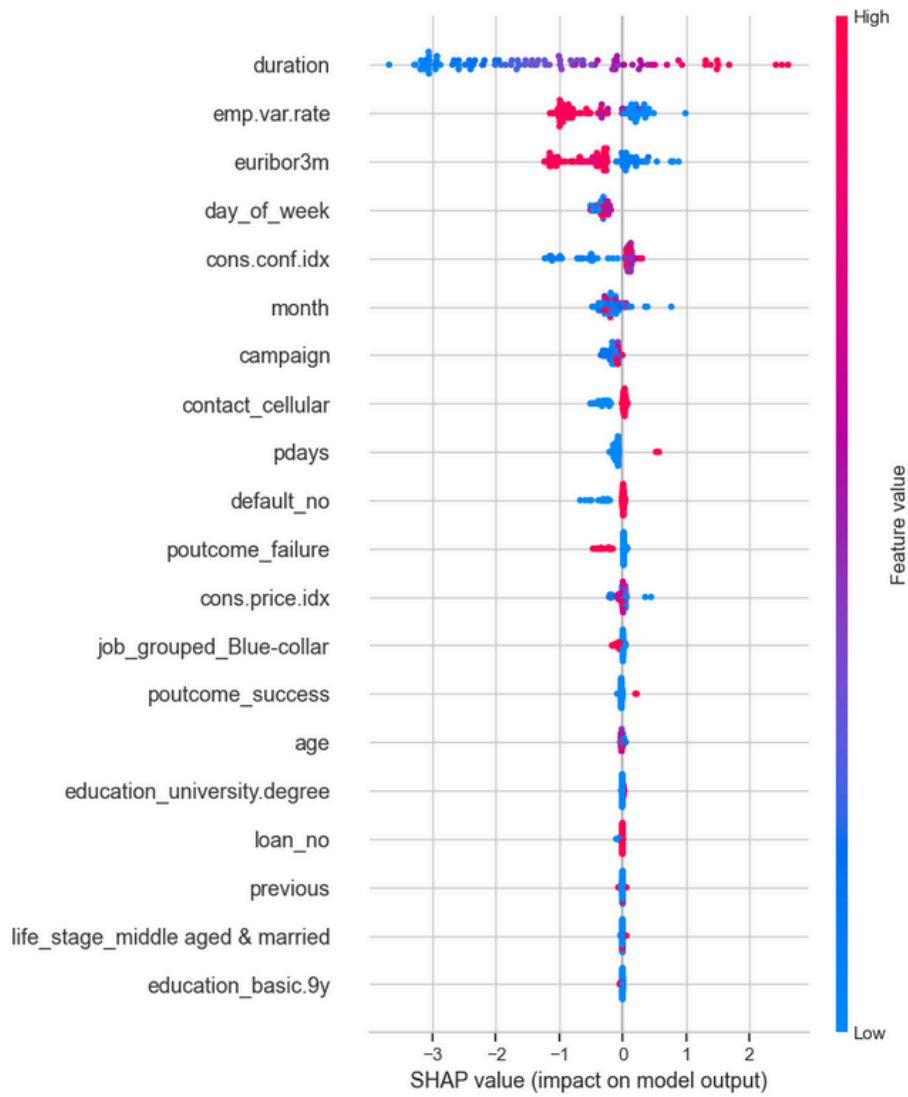
◆ F1 Score: 0.6454

◆ AUC: 0.9485

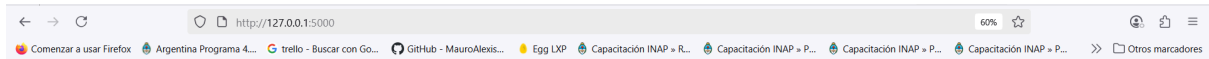
Curva ROC (AUC = 0.95)







h. Capturas de interfaz productiva



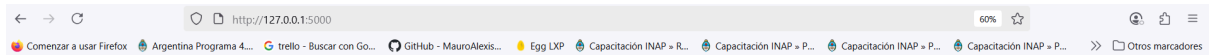
Chatbot para Predicción de la suscripción a depósitos a plazo

Por favor ingresa el dato para age.

Ingresa un número

Enviar

Proyecto realizado en la Universidad Complutense de Madrid



Chatbot para Predicción de la suscripción a depósitos a plazo

Por favor ingresa el dato para posicame.

Por favor ingresa el dato para emp.vac.rate.

Por favor ingresa el dato para com.unica.de.

Por favor ingresa el dato para com.confido.

Por favor ingresa el dato para exuberan.

Por favor ingresa el dato para re.employed.

Gracias! Estoy procesando tu predicción.

Predicción: Si contratas el depósito a plazo (Probabilidad: 56.21%)

Enviar

¿Tienes una pregunta sobre el modelo o el TFM?

Ej: ¿De dónde provienen los datos?

Preguntar

Proyecto realizado en la Universidad Complutense de Madrid



Chatbot para Predicción de la suscripción a depósitos a plazo

Por favor ingresa el dato para poutcome.

Por favor ingresa el dato para emp.var.rate.

Por favor ingresa el dato para cons.pnce.idx.

Por favor ingresa el dato para cons.conf.idx.

Por favor ingresa el dato para eurbor3m.

Por favor ingresa el dato para nr.employed.

Gracias! Estoy procesando tu predicción...

Predicción: No contratará el depósito a plazo (Probabilidad: 2.91%)

Enviar

¿Tienes una pregunta sobre el modelo o el TFM?

Ej: ¿De dónde provienen los datos?

Preguntar

Proyecto realizado en la Universidad Complutense de Madrid

¿Tienes una pregunta sobre el modelo o el TFM?

¿Qué es el EDA?

Preguntar

análisis

exploratorio

de

datos

¿Tienes una pregunta sobre el modelo o el TFM?

¿Cuáles modelo se probaron?

Preguntar

Modelos

de

Machine

Learning

clásicos

¿Tienes una pregunta sobre el modelo o el TFM?

¿Cuál es el objetivo general?

Preguntar

construir

un

modelo

de

clasificación

binaria



¿Tienes una pregunta sobre el modelo o el TFM?

¿Qué es el preprocesamiento de datos?

Preguntar

una

fase

crítica

para

garantizar

la

calidad

y

pertinencia

del

conjunto

de

datos

Enviar

¿Tienes una pregunta sobre el modelo o el TFM?

¿Cuál es el modelo más efectivo?

Preguntar

XGBoost

i. Instrucciones para ejecución del modelo productivo final

A continuación se presenta un gráfico con la estructura de archivos del proyecto, disponible en el repositorio de GitHub, en Google Drive o en el archivo [.zip](#) entregado en la plataforma del Máster. Se detallan los archivos necesarios para la ejecución del modelo productivo final, incluyendo los archivos [requirements.txt](#) con las especificaciones de las bibliotecas utilizadas, tanto para el entorno Anaconda como para Visual Studio Code.

```
TFM_UCM_DataScience/  
├── templates/   
│   └── chat.html   
├── TFM_Fernández_Mauro_Alexis_MODELO.ipynb   
├── TFM_Fernández_Mauro_Alexis_MODELO_HTML.html   
├── app.py   
├── bank-additional-full.csv   
├── build_rag_index.py   
├── chat_rag_local.py   
├── explanation.html   
├── feature_selector.py   
├── manual_scaler.py   
├── minmax_scaler.pkl   
├── modelo_final_xgb.pkl   
├── onehot_encoder.pkl   
├── onehot_transformer.py   
├── output_reporte.html   
├── pipeline_modelo_completo.pkl   
├── preprocessing.py   
├── rag_index.faiss   
├── rag_metadata.json   
├── selected_features_rfecv.pkl   
├── requirements_VisualStudioCode.txt   
├── requirements_anaconda.txt   
├── TFM_Mauro_Alexis_Fernandez.pdf   
└── shap_summary_plot.png 
```

Luego y en relación con las **instrucciones para la ejecución de la aplicación productiva**, se detallan a continuación los pasos sucesivos, a saber:

- 1) Abrir la terminal o línea de comandos de tu sistema operativo.
- 2) Ubicarse en la carpeta del proyecto donde se encuentran todos los archivos del TFM, por ejemplo:

```
cd C:\Users\PC\TFM
```

- 3) Ejecutar la aplicación Flask con el siguiente comando:

python [app.py](#)

4) Abrir un navegador web y acceder a la dirección:

<http://127.0.0.1:5000/>

5) Desde allí se podrá interactuar con la aplicación para obtener predicciones y utilizar el sistema RAG integrado.

6) Para cerrar la aplicación, volver a la terminal y presionar **Ctrl + C**.

j. Bibliografía

Alexander, R. (2021). *Telling Stories with Data: With Exercises in R*. CRC Press.

Dunning, T., & Friedman, E. (2017). *Machine Learning Logistics*. O'Reilly Media.

Fernández-Delgado, M., Cernadas, E., Barro, S., & Amorim, D. (2014). *Do we need hundreds of classifiers to solve real world classification problems?* **Journal of Machine Learning Research**, 15(1), 3133–3181. <https://jmlr.org/papers/volume15/delgado14a/delgado14a.pdf>

Lundberg, S. M., & Lee, S.-I. (2017). *A unified approach to interpreting model predictions*. In *Advances in Neural Information Processing Systems* (NeurIPS 2017). <https://proceedings.neurips.cc/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf>

McKinney, W. (2018). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython* (2nd ed.). O'Reilly Media.

Moro, S., Laureano, R., & Cortez, P. (2014). *Using data mining for bank direct marketing: An application of the CRISP-DM methodology*. **Expert Systems with Applications**, 41(11), 4642–4659. <https://doi.org/10.1016/j.eswa.2014.01.021>

Müller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly Media.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). *"Why should I trust you?": Explaining the predictions of any classifier*. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135–1144). <https://doi.org/10.1145/2939672.2939778>

Tunstall, L., von Werra, L., & Wolf, T. (2022). *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. O'Reilly Media.

Wilke, C. O. (2019). *Fundamentals of Data Visualization: A Primer on Making Informative and Compelling Figures*. O'Reilly Media.

k. Código completo del proyecto

A continuación se adjuntan capturas correspondientes a cada uno de los archivos de código usados para el proyecto (sin ejecutar en el caso del Notebook para limitar el espacio ocupado por gráficos en el presente informe). Asimismo, se comparten los enlaces en Github pertinentes para acceder directamente a cada uno de los mismos (en caso de requerirse acceso vía Google Drive, el vínculo es https://drive.google.com/drive/folders/11nXf3mFeh6A4Z6hL7YOt7je59O00-62H?usp=drive_link y los tutores poseen acceso a la carpeta):

k.1) Archivo de modelado en Jupyter Notebook: TFM_Fernández_Mauro_Alexis_MODELO

Enlace Github: versión HTML →

https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/TFM_Fern%C3%A1ndez_Mauro_Alexis_MODELO_HTML.html

Enlace Github: versión ipynb →

https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/TFM_Fern%C3%A1ndez_Mauro_Alexis_MODELO.ipynb

TFM MÁSTER DATA SCIENCE, BIG DATA & BUSINESS ANALYTICS

Predicción de la suscripción a depósitos a plazo mediante Machine Learning y Deep Learning en campañas de marketing bancario

MAURO ALEXIS FERNÁNDEZ

UNIVERSIDAD COMPLUTENSE DE MADRID

2024/2025

Link Repositorio Github del TFM https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience

En virtud de los requerimientos detallados para la realización del TFM

Se presentan a continuación las líneas de código y comentarios relativos a la exploración y visualización de datos, procesamiento, modelado, ajuste de hiperparámetros y productivización del modelo predictivo. El código relativo a la parte ya productivizada del TFM, se incorpora en los otros archivos disponibles en el repositorio arriba referido.

Fuente de datos disponible en: [Bank Marketing](#)

ÍNDICE

- 1. CARGA DE LAS LIBRERÍAS NECESARIAS Y EL DATASET
- 2. ANÁLISIS EXPLORATORIO DE DATOS (EDA)
 - 2.1 Distribución de la variable objetivo (desbalanceo)
 - 2.2 Análisis de variables categóricas y numéricas
 - 2.3 Correlaciones y relaciones significativas
 - 2.4 Segmentaciones por atributos clave
 - 2.5 Insights preliminares
- 3. PREPROCESAMIENTO DE DATOS
 - 3.1 Tratamiento de valores faltantes o inconsistentes
 - 3.2 Encoding de variables categóricas
 - 3.3 Transformaciones de variables numéricas
- 4. INGENIERÍA DE CARACTERÍSTICAS
 - 4.1 Agrupaciones o transformaciones lógicas
 - 4.2 Generación de nuevas variables derivadas
- 4. INGENIERÍA DE CARACTERÍSTICAS
 - 4.1 Agrupaciones o transformaciones lógicas
 - 4.2 Generación de nuevas variables derivadas
 - 4.3 Eliminación de variables redundantes
 - 4.4 Encoding de variables categóricas (OneHotEncoder)
- 5. MODELADO PREDICTIVO
 - 5.1 División del conjunto en entrenamiento y test
 - 5.2 Elección de métricas: accuracy, precision, recall, F1, AUC
 - 5.3 Definición del baseline
 - 5.4 Modelo de Regresión logística
 - 5.5 Modelo de Árboles de decisión
 - 5.6 Modelo de Random Forest
 - 5.7 Modelo de XGBoost
 - 5.8 Modelo de K-Nearest Neighbors
 - 5.9 Modelo de Naive Bayes
 - 5.10 Modelo de Deep Learning
 - 5.11 Comparación de modelos
- 6. INTERPRETABILIDAD Y EXPLICABILIDAD DE MODELOS
 - 6.1 Feature importance
 - 6.2 SHAP y LIME para interpretación local/global
 - 6.3 Discusión sobre sesgos y robustez del modelo
- 7. EVALUACIÓN FINAL Y SELECCIÓN DE MODELO FINAL
 - 7.1 Comparación global de métricas
 - 7.2 Selección del modelo con mejor balance interpretabilidad-rendimiento
- 8. PRODUCTIVIZACIÓN DEL MODELO

Descripción inicial del dataset y su propósito

"Una institución bancaria portuguesa llevó a cabo campañas de marketing directo con el objetivo de evaluar si los clientes contratarían un depósito a plazo luego de ser contactados por los operadores comerciales. Dichas campañas se basaron en llamadas telefónicas, y en algunos casos fue necesario contactar al mismo cliente en más de una ocasión."

Contribución esperada

"El presente trabajo tiene como objetivo construir y comparar modelos de clasificación que permitan predecir la suscripción a depósitos a plazo. Se espera identificar los atributos más influyentes en la decisión de los clientes y seleccionar un modelo robusto y explicable que pueda ser integrado en sistemas de apoyo a la decisión bancaria."

1- CARGA DE LAS LIBRERÍAS NECESARIAS Y EL DATASET

Se inicia el desarrollo del código mediante la siguiente celda donde se importarán la mayoría de las librerías necesarias para la realización de la actividad. Asimismo, se destaca que pueden existir repeticiones de importaciones en celdas posteriores (aunque se ha buscado minimizar las mismas).

```
[ ]: # =====
# Librerías básicas pandas y numpy de Python usadas para análisis de datos
# =====
import pandas as pd # Manipulación y análisis de datos tabulares
import numpy as np # Operaciones numéricas y matrices

# =====
# Librerías de visualización
# =====
import matplotlib.pyplot as plt # Visualizaciones básicas (gráficos de línea, barras, histograma, etc.)
import seaborn as sns # Visualizaciones estadísticas avanzadas, heatmaps, boxplots, etc.
import plotly.express as px # Gráficos interactivos de alto nivel
import plotly.graph_objects as go # Gráficos interactivos personalizados

# =====
# Librería para EDA (análisis exploratorio automatizado)
# =====
from ydata_profiling import ProfileReport # Generación automática de reportes de EDA

# =====
# Librería para transformar en numéricas aquellas variables categóricas
# (se destaca que la ejecución de muchos modelos de ML no aceptan la introducción de variables categóricas)
# =====
from sklearn.preprocessing import LabelEncoder # Codificación ordinal simple (útil para variables con orden implícito)
from sklearn.preprocessing import OneHotEncoder # Codificación one-hot (útil para modelos que no aceptan etiquetas)

# =====
# Librerías para partición del dataset, validación cruzada, ajuste de hiperparámetros y validación de modelo
# =====
from sklearn.model_selection import (
    train_test_split, # División en conjuntos de entrenamiento y prueba
    cross_val_score, # Evaluación cruzada de modelos
    GridSearchCV, # Búsqueda de hiperparámetros óptimos
    StratifiedKFold # Validación cruzada estratificada
)
from sklearn.dummy import DummyClassifier # Clasificador de referencia (baseline)

from sklearn.dummy import DummyClassifier # Clasificador de referencia (baseline)
from sklearn.feature_selection import RFECV # Eliminación recursiva de características con validación cruzada
from sklearn.base import BaseEstimator, TransformerMixin # Clases base para crear transformadores personalizados (útiles en pipelines)

# =====
# Modelos clásicos de clasificación
# =====
from sklearn.linear_model import LogisticRegression # Regresión Logística
from sklearn.tree import DecisionTreeClassifier # Árboles de decisión
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier # Ensamblados: Random Forest y AdaBoost
from sklearn.svm import LinearSVC, SVC # Máquinas de vectores de soporte (Lineales y no lineales)
from xgboost import XGBClassifier # Clasificador basado en gradient boosting (XGBoost)
from sklearn.naive_bayes import GaussianNB # Clasificador Naive Bayes
from sklearn.neighbors import KNeighborsClassifier # Clasificador K-Nearest Neighbors

# =====
# Librerías de Deep Learning
# =====
import keras # Framework de alto nivel para redes neuronales (sobre TensorFlow)
from keras.models import Sequential # Modelo secuencial de Keras
from keras.layers import Dense, Dropout # Capa densa totalmente conectada y de dropout
from tensorflow.keras import regularizers # Regularizadores L1, L2, etc.
from keras import layers # Acceso directo a capas como Dense, Dropout, etc.
from keras.metrics import Precision, Recall, AUC # Métricas específicas de Keras para evaluar modelos
from keras.callbacks import EarlyStopping # Detiene el entrenamiento si no hay mejora en la métrica monitorizada

# =====
# Escalado de variables
# =====
from sklearn.preprocessing import MinMaxScaler # Escalado entre 0 y 1 (normalización)

# =====
# Balanceo de clases con sobremuestreo
# =====
from imblearn.over_sampling import SMOTE # Técnica SMOTE para generar datos sintéticos de clases minoritarias

# =====
# Visualización de relaciones entre variables
```

```
# Visualización de relaciones entre variables
# =====
from pandas.plotting import scatter_matrix # Matriz de diagramas de dispersión entre variables numéricas

# =====
# Selección de características
# =====
from sklearn.feature_selection import SelectKBest # Selección de las mejores K variables
from sklearn.feature_selection import VarianceThreshold # Eliminación de variables con baja varianza

# =====
# Métricas de evaluación de modelos(curva ROC y área bajo la curva)
# =====
from sklearn.metrics import classification_report, accuracy_score, auc, confusion_matrix, f1_score, precision_score, recall_score, roc_curve

# =====
# Estadísticas adicionales
# =====
from scipy.stats import stats # Pruebas estadísticas diversas (t-test, normalidad, etc.)
from scipy.stats import chi2_contingency # Test de chi-cuadrado (asociación entre variables categóricas)

# =====
# Interpretación de modelos (Explainability)
# =====
import lime # Framework para interpretabilidad de modelos caja negra
import shap # Interpretación global y local de modelos (SHAP values)
from lime import lime_tabular # Visualización local basada en LIME para datos tabulares

# =====
# Pipelines y serialización
# =====
from sklearn.pipeline import Pipeline # Creación de pipelines de preprocesamiento y modelado
from sklearn.compose import ColumnTransformer # Aplicación de transformaciones por columnas
from imblearn.pipeline import Pipeline as ImbPipeline # Pipeline compatible con técnicas de sobremuestreo
import joblib # Serialización de modelos y objetos
import pickle # Serialización alternativa con Python puro

# =====
# Configuración de entorno
# =====
import warnings
warnings.filterwarnings("ignore") # Ignorar advertencias para evitar ruido en la salida
```

Se procede ahora a la lectura del archivo CSV con los datos originales y, a partir de los mismos, se genera un dataframe de pandas.

```
#Efectúo la lectura del csv y guardo todo en un dataframe de Pandas con nombre df
df = pd.read_csv('bank-additional-full.csv', sep=';')

#Reviso las primeras filas del dataframe generado
df.head()

#Obtengo el tamaño y cantidad de columnas de dataframe recién generado
df.shape
```

A partir de los resultados obtenidos en la celda anterior, se observa que el dataframe contiene **41,188 registros** y **21 columnas** (incluyendo la variable objetivo).

Este tamaño es representativo para el análisis que se llevará a cabo, permitiendo abordar un amplio rango de características para el modelado y evaluación de la variable objetivo.

2 - ANÁLISIS EXPLORATORIO DE DATOS (EDA)

```
#Reviso si la generación del dataframe df fue correcta y muestro todas las columnas presentes.
#Selecciono la opción 'display.max_columns' para que pueda ver todas las columnas, sin truncar las intermedias con puntos suspensivos.
pd.set_option('display.max_columns', None)
df
```

Identificación y descripción de las variables

Identificación y descripción de las variables

A continuación se incluye un listado con la descripción de las variables presentes en el dataset:

A. Datos sobre el cliente del banco

Variable	Descripción	Tipo	Nota
age	Edad en años del cliente	Numérica	
job	Tipo de empleo del cliente	Categórica	
marital	Estado civil del cliente	Categórica	"divorced" puede incluir viudo
education	Nivel educativo del cliente	Categórica	
default	¿El cliente posee algún crédito en default?	Categórica	
housing	¿El cliente posee algún crédito para vivienda?	Categórica	
loan	¿El cliente posee algún crédito personal?	Categórica	

B. Relativas al último contacto en la presente campaña

Variable	Descripción	Tipo
contact	Tipo de comunicación utilizada por el marketer	Categórica
month	Mes del último contacto con el cliente por el marketer	Categórica
day_of_week	Día de la semana del último contacto por el marketer	Categórica
duration	Duración del último contacto (en segundos) entre cliente-marketer	Numérica

C. Otros atributos relacionados con la campaña

Variable	Descripción	Tipo	Nota
campaign	Cantidad de contactos realizados en esta campaña con el cliente	Numérica	
pdays	Días desde el último contacto en campañas anteriores con el cliente	Numérica	999 indica que no fue contactado previamente
previous	Número de contactos previos antes de esta campaña con el cliente	Numérica	

C. Otros atributos relacionados con la campaña

Variable	Descripción	Tipo	Nota
campaign	Cantidad de contactos realizados en esta campaña con el cliente	Numérica	
pdays	Días desde el último contacto en campañas anteriores con el cliente	Numérica	999 indica que no fue contactado previamente
previous	Número de contactos previos antes de esta campaña con el cliente	Numérica	
poutcome	Resultado de campañas anteriores con este cliente	Categórica	

D. Atributos socioeconómicos

Variable	Descripción	Tipo
emp.var.rate	Tasa de variación del empleo registrado – indicador trimestral	Numérica
cons.price.idx	Índice de precios al consumidor – indicador mensual	Numérica
cons.conf.idx	Índice de confianza del consumidor – indicador mensual	Numérica
euribor3m	Tasa euribor a 3 meses – indicador diario	Numérica
nr.employed	Número de personas empleadas – indicador trimestral	Numérica

E. Variable objetivo 1

Variable	Descripción	Tipo
y	¿El cliente terminó por contratar un depósito a plazo fijo?	Binaria

Nota adicional 1:

Asimismo, se destaca el hecho de que el sitio, donde se aloja el dataset fuente, explicita que aquellos valores de **999** corresponden a **valores faltantes**.

Ahora procedo a analizar e identificar la presencia de filas **duplicadas** en el dataframe


```
[ ]: # Primero identifico las filas/registros duplicados
duplicates = df[df.duplicated()]
duplicates

[ ]: # Muestro la cantidad de duplicados en el dataframe df
print(f"Cantidad de registros duplicados: {duplicates.shape[0]}")
```

Se detectaron **12 filas duplicadas** en el dataframe.

```
[ ]: # Se procede a eliminar aquellas filas previamente identificadas como duplicadas
df.drop_duplicates(inplace = True)
```

```
[ ]: # Verifico que no queden filas duplicadas
duplicates = df[df.duplicated()]
duplicates
```

```
[ ]: # Vuelvo a mostrar la cantidad de duplicados para revisar
print(f"Cantidad de registros duplicados: {duplicates.shape[0]}")
```

```
[ ]: # Observo el tamaño del dataframe luego de estas tareas de limpieza de duplicados
df.shape
```

A continuación, procedo a analizar e identificar la presencia de **datos nulos** en el dataframe.

```
[ ]: df.isnull().sum()
```

No se identificaron valores nulos **explícitos** en ninguna de las columnas del dataframe. Sin embargo, se procederá a investigar la posible existencia de valores faltantes **implícitos o codificados** de otra forma.

```
[ ]: # Utilizo .info() sobre el dataframe para revisar los tipos de datos por variable en este momento.
df.info()
```

2.1 Distribución de la variable objetivo (desbalanceo)

```
[ ]: # Luego, busco obtener la distribución de valores entre las 2 categorías posibles para la variable objetivo (si se contratan o no créditos a plazo) .
df.y.value_counts()
```

Se observa un **marcado desbalance** en la distribución de la variable objetivo, con **36,537 registros correspondientes a la clase no** y **4,639 registros a la clase yes**. Este desequilibrio deberá ser tenido en cuenta en las posteriores etapas de modelado, especialmente en las técnicas de balanceo de datos.

```
[ ]: # Ahora lo presento de forma gráfica para evidenciarlo más claramente con una visualización
# Primero cuento los valores
y_counts = df['y'].value_counts()
sizes = y_counts.values
labels = y_counts.index

colors = ['#66b3ff', '#ff9999'] # Defino los colores para cada valor de la variable

def make_autopct(values):
    def my_autopct(pct):
        total = sum(values)
        count = int(round(pct * total / 100.0))
        return f'{count} ({pct:.1f}%)'
    return my_autopct

plt.figure(figsize=(6,6))
wedges, texts, autotexts = plt.pie(
    sizes,
    labels=None, # No muestro las etiquetas fuera para no repetir y sobre-cargar la vista
    autopct=make_autopct(sizes),
    startangle=90,
    colors=colors,
    textprops={'color': 'black'})

plt.title('Distribución de la variable objetivo (y)')

# Agrego leyenda que explique qué color corresponde a cada valor de y
plt.legend(wedges, labels,
           title="Valores de y",
           loc="center left",
           bbox_to_anchor=(1, 0, 0.5, 1)) # Lo coloco en una ubicación fuera del gráfico para no abigarrar demasiado
```

```
plt.title('Distribución de la variable objetivo (y)')

# Agrego Leyenda que explique qué color corresponde a cada valor de y
plt.legend(wedges, labels,
           title="Valores de y",
           loc="center left",
           bbox_to_anchor=(1, 0, 0.5, 1)) # Lo coloco en una ubicación fuera del gráfico para no abigarrar demasiado

plt.axis('equal')
plt.show()
```

2.2 Análisis de variables categóricas y numéricas

Análisis de variables numéricas

Se muestran a continuación las principales métricas descriptivas de las variables **numéricas**, incluyendo la media, los valores mínimo y máximo, así como los percentiles 25, 50 (mediana) y 75.

Esto permite obtener una primera aproximación a la distribución y posibles valores atípicos en los datos.

```
#Selecciono y separo las variables categóricas y numéricas del dataset xtrain
cat_cols= df.select_dtypes(include=['object', 'category']).columns
num_cols = df.select_dtypes(exclude=['object', 'category']).columns

# Con .describe() visualizo algunas métricas sobre las variables numéricas (media, valor mínimo,máximo, cuartiles, etc.)
df.describe()
```

```
# Función para visualizar histogramas y boxplots de variables numéricas
def visualizar_numericas(df, num_cols):
    for col in num_cols:
        plt.figure(figsize=(14, 5))

        # Histograma
        plt.subplot(1, 2, 1)
        sns.histplot(df[col], kde=True, bins=30, color='skyblue')
        plt.title(f'Histograma de {col}')
        plt.xlabel(col)
        plt.ylabel('Frecuencia')
```

```
# Función para visualizar histogramas y boxplots de variables numéricas
def visualizar_numericas(df, num_cols):
    for col in num_cols:
        plt.figure(figsize=(14, 5))

        # Histograma
        plt.subplot(1, 2, 1)
        sns.histplot(df[col], kde=True, bins=30, color='skyblue')
        plt.title(f'Histograma de {col}')
        plt.xlabel(col)
        plt.ylabel('Frecuencia')

        # Boxplot
        plt.subplot(1, 2, 2)
        sns.boxplot(x=df[col], color='salmon')
        plt.title(f'Boxplot de {col}')
        plt.xlabel(col)

    plt.tight_layout()
    plt.show()

# Llamo a la función
visualizar_numericas(df, num_cols)
```

Análisis descriptivo de variables numéricas

A partir del análisis preliminar, se observan los siguientes aspectos relevantes que serán fundamentales para la etapa de preprocesamiento:

- **Edad (age)**: La distribución varía entre 17 y 98 años, con una media cercana a los 40 años y una desviación estándar de aproximadamente 10. Esto refleja una población mayoritariamente adulta y heterogénea. Podría considerarse agrupar por rangos etarios o detectar valores extremos.
- **campaign**: Presenta una fuerte asimetría. El valor máximo es 56, mientras que el percentil 75 es apenas 3, lo que sugiere la presencia de *outliers* (clientes contactados muchas veces). Este comportamiento podría impactar negativamente en algunos algoritmos sensibles a valores extremos.
- **duration**: La variable presenta una distribución fuertemente asimétrica hacia la derecha, con una gran concentración de observaciones en valores bajos (cerca de cero) y una larga cola que se extiende hasta los 4918 segundos. El boxplot revela una abundante presencia de valores atípicos.
- **pdays**: Se observa que los percentiles 25, 50 y 75 coinciden en 999, lo cual indica que este valor está siendo utilizado como código para "no contactado previamente". Por su naturaleza, debería ser recodificado o transformado en una variable categórica o binaria.
- **previous**: Tanto la mediana como el tercer cuartil son cero, lo que indica que la mayoría de los clientes no tuvo contactos previos exitosos. Esta variable presenta una alta concentración de ceros, por lo que podría analizarse su distribución o convertirla en una variable binaria.
- **Variables macroeconómicas (emp.var.rate, cons.price.idx, euribor3m, nr.employed)**: Muestran una baja dispersión relativa y valores bastante acotados, lo que refleja su estabilidad como indicadores de contexto económico. Si bien su variabilidad es limitada, podrían aportar valor al modelo al capturar el entorno macro durante cada campaña.

En conjunto, este análisis permite identificar necesidades de transformación como recodificación de valores especiales, detección de outliers, y posibles decisiones de ingeniería de variables. Estos hallazgos guiarán el tratamiento adecuado de los datos en etapas posteriores.

Análisis de variables categóricas

A continuación, se procederá a analizar las variables categóricas del conjunto de datos.

```
]# Visualizo la distribución de todas las variables categóricas del dataframe
for col in cat_cols:
    plt.figure(figsize=(10, 5))

    num_categories = df[col].nunique()
    palette = sns.color_palette("husl", num_categories)

    sns.countplot(
        x=col,
        data=df,
        order=df[col].value_counts().index,
        palette=palette
    )

    plt.title(f'Conteo de valores para la variable categórica: {col}')
    plt.xticks(rotation=45, ha='right') # Con esto mejoro la legibilidad de aquellas etiquetas demasiado largas
    plt.tight_layout()
    plt.show()
```

Análisis descriptivo de variables categóricas

En esta sección se analiza la distribución de las variables categóricas presentes en el dataset. Se utilizaron gráficos de barras (`countplot` de seaborn) para visualizar la frecuencia de cada categoría en las distintas variables, ordenando los valores de mayor a menor para facilitar su interpretación.

A continuación se presentan las principales observaciones:

`poutcome` (Resultado de campañas anteriores)

La mayoría de los clientes no ha participado en campañas previas (`nonexistent`). Las categorías `failure` y `success` tienen una representación significativamente menor. Podría considerarse recodificar esta variable en binaria (participó / no participó).

`job` (Empleos/ocupaciones)

La mayoría de los clientes tienen ocupaciones como `admin.`, `blue-collar` y `technician`. También se observan categorías como `student`, `unemployed` y `unknown`, que podrían requerir un tratamiento especial.

`day_of_week` (Día del contacto)

Distribución relativamente equilibrada entre los días de la semana, aunque martes y miércoles presentan una ligera mayor frecuencia. Puede mantenerse sin necesidad de transformación.

`month` (Mes del contacto)

Se observa una fuerte concentración en los meses de mayo, junio, julio y agosto. Esto sugiere una estacionalidad en la ejecución de las campañas, lo cual podría ser relevante para el modelado.

`contact` (Tipo de contacto)

La mayoría de los contactos se realizó vía celular. La categoría `telephone` tiene baja frecuencia. A pesar del desbalance, la variable puede aportar información útil.

`loan`, `housing`, `default`

En los tres casos, predomina la categoría `no`. La variable `default` tiene una frecuencia muy baja en la categoría `yes`, lo que podría reducir su poder predictivo. Se recomienda evaluar su utilidad más adelante.

`education`

Existen múltiples niveles educativos, con predominancia de `university.degree`, `high.school` y `basic.9y`. Algunas categorías como `illiterate` y `unknown` tienen muy poca representación. Sería conveniente agrupar niveles en categorías más amplias (ej.: básico, medio, superior) para reducir cardinalidad y mejorar interpretabilidad.

`marital`

Distribución clara entre `married`, `single` y `divorced`, siendo `married` la más frecuente. Es una variable informativa que puede mantenerse sin modificaciones.

Conclusión:

Este análisis permitió identificar desbalances, patrones de estacionalidad y posibles oportunidades de transformación o agrupamiento en variables categóricas. Estas observaciones serán tenidas en cuenta en las etapas de preprocesamiento y modelado.

Generación de reporte/informe con el perfilado de datos

En la siguiente celda, genero un informe con visualizaciones e información útil para profundizar esta etapa de EDA

```
#Esta celda genera un informe con el perfilado de los datos. El archivo resultante puede ser consultado y explorado.
#A partir de este informe, obtengo una serie de gráficos, recomendaciones y voy tomando nota de las características de los datos.
prof = ProfileReport(df)
prof.to_file(output_file='output_reporte.html')
```

2.3 Correlaciones y relaciones significativas

```
#Genero un heatmap (mapa de calor) de las correlaciones entre las variables numéricas del DataFrame
#Esto puede permitirme detectar variables altamente correlacionadas (multicolinealidad),
#entender relaciones lineales entre features o tomar decisiones sobre selección de variables o ingeniería de features.
%matplotlib inline
df_correlacion = df.select_dtypes(include=['number'])
correlation_mat = df_correlacion.corr()
sns.heatmap(correlation_mat)
plt.show()
```

Análisis de correlación entre variables numéricas

Análisis de correlación entre variables numéricas

Variables incluidas:

- age, duration, campaign, pdays, previous
- emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed

Observaciones del heatmap:

- **Alta correlación positiva** entre:
 - emp.var.rate, euribor3m, nr.employed y cons.price.idx:
 - Estas variables económicas están **fuertemente correlacionadas** entre sí (coef. > 0.8), lo que indica **riesgo de multicolinealidad** si se incluyen juntas en modelos lineales.
- **Correlaciones negativas destacadas:**
 - pdays y previous tienen una fuerte correlación **negativa** (coef. < -0.5).
 - emp.var.rate y cons.conf.idx también tienen una correlación negativa notable.
- **Variables independientes:**
 - age, duration y campaign tienen **baja correlación** con las demás variables numéricas, lo cual es bueno si buscamos independencia entre features.

Recomendaciones que deberemos considerar:

- **Reducción de multicolinealidad:** Puede ser necesario **dejar solo una o pocas** entre: euribor3m, emp.var.rate, nr.employed o cons.price.idx si se utiliza un modelo lineal o de regresión logística. Estas variables macro podrían ser muy útiles para explicar el contexto de la campaña, pero deben analizarse con cuidado por su interdependencia.
- **Transformaciones o selección:** Para modelos basados en árboles (como XGBoost o Random Forest), la **multicolinealidad no es un problema grave**, aunque puede afectar la interpretabilidad.

```
# Luego y para observar las correlaciones entre las variables catagóricas, utilizo esta función para calcular la V de Cramér
def cramers_v(x, y):
    confusion_matrix = pd.crosstab(x, y)
    chi2 = chi2_contingency(confusion_matrix)[0]
    n = confusion_matrix.sum().sum()
    phi2 = chi2 / n
    r, k = confusion_matrix.shape
    return np.sqrt(phi2 / min(k - 1, r - 1))

# Creo matriz de Cramér's V entre variables categóricas
cat_cols = df.select_dtypes(include=['object', 'category']).columns
cramer_matrix = pd.DataFrame(index=cat_cols, columns=cat_cols)

for col1 in cat_cols:
    for col2 in cat_cols:
        if col1 == col2:
            cramer_matrix.loc[col1, col2] = 1.0
        else:
            try:
                cramer_matrix.loc[col1, col2] = cramers_v(df[col1], df[col2])
            except:
                cramer_matrix.loc[col1, col2] = np.nan # En caso de error (por ejemplo, columnas con un solo valor)

cramer_matrix = cramer_matrix.astype(float)

# Visualizo la matriz como heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(cramer_matrix, annot=True, cmap='YlGnBu', fmt='.2f', linewidths=0.5)
plt.title("Matriz de asociación entre variables categóricas (Cramér's V)")
plt.tight_layout()
plt.show()
```

Asociación entre variables categóricas (V de Cramer)

Esta matriz muestra los niveles de asociación entre pares de variables categóricas mediante la **V de Cramer**, que toma valores entre 0 (sin relación) y 1 (asociación perfecta).

Observaciones clave:

Alta asociación entre variables ($V > 0.5$):

- housing y loan: **0.71**

Asociación entre variables categóricas (V de Cramer)

Esta matriz muestra los niveles de asociación entre pares de variables categóricas mediante la **V de Cramer**, que toma valores entre 0 (sin relación) y 1 (asociación perfecta).

Observaciones clave:

Alta asociación entre variables ($V > 0.5$):

- `housing` y `loan`: **0.71**
- `contact` y `month`: **0.61**

Estas relaciones indican que los valores de una de las variables podrían anticiparse parcialmente con la otra. Puede evaluarse si alguna es redundante o si conviene tratarlas juntas.

Asociación moderada con la variable objetivo (`y`):

- `poutcome` → `y`: **0.32** ► La más asociada con el target.
- `month` → `y`: **0.27**
- `poutcome` → `month`: **0.24**
- `contact` → `y`: **0.14**
- `job` → `y`: **0.15**

Estas variables categóricas pueden ser relevantes para el modelo. Especial atención merece `poutcome`, que parece tener la mayor capacidad explicativa.

Asociación baja o nula ($V < 0.1$):

- `day_of_week`, `default`, `loan`, `housing`, `marital` y `education` tienen poca asociación con `y` ($V < 0.1$).
 - Aunque no se descartan, podrían tener menor prioridad en el modelado o en el análisis exploratorio.

Recomendaciones que debemos considerar:

- **Reducir redundancia:** Revisar si conviene eliminar una entre `loan` y `housing`, o bien combinarlas. Lo mismo aplica para `month` y `contact`.
- **Seleccionar variables con mayor poder predictivo:** Priorizar `poutcome`, `month`, `job`, `contact`.

Si se usa un modelo como Random Forest o XGBoost, puede conservarse el conjunto completo, pero se recomienda revisar la **importancia de características** luego del entrenamiento.

2.4 Segmentaciones por atributos clave

```
# En este punto asigno valores numéricos para cada una de los valores posibles que puede adoptar la variable objetivo.
df.y.replace(('no','yes'),(0,1),inplace=True)
df['y'].value_counts()
```

```
# Genero un gráfico de torta de la variable objetivo

target_labels = ['No', 'Yes']
target_counts = [df.y[df.y == 0].count(), df.y[df.y == 1].count()]

explode = (0, 0.25)

plt.figure(figsize=(6,6))
cmap = plt.get_cmap('tab20')
colors = [cmap(i) for i in np.linspace(0, 1, 8)]

# Creo el gráfico
target_pie = plt.pie(
    target_counts,
    labels=target_labels,
    explode=explode,
    autopct='%1.2f%%',
    textprops={'fontsize': 14},
    shadow=True,
    colors=colors
)

# Agrego el título
plt.title('Distribución de la variable objetivo "y"', fontsize=18)

# Agrego una leyenda explicativa
plt.legend(
    labels=["0 = No", "1 = Yes"],
    loc='lower left',
    fontsize=12,
    title="Codificación de y"
)

plt.show()
```

Segmentaciones por atributos clave de la variable job

Segmentaciones por atributos clave de la variable job

```
[ ]: #Obtengo los valores posibles que adopta la variable job
df.job.unique()

[ ]: #Calculo la proporción de plazos suscritos para cada valor posible de la columna trabajo
job_proporcion = ((df.y[df.y == 1].groupby(df.job).count())/
                  (df.y.groupby(df.job).count()))*100

[ ]: # Proporción de plazos suscritos para cada tipo de trabajo
job_proporcion

[ ]: # Genero gráfico de la proporción de plazos suscritos (y=1) para cada tipo de trabajo
job_rate = df.groupby('job')['y'].mean().sort_values()

job_rate.plot(
    kind='barh', figsize=(10,6), color='mediumseagreen', edgecolor='black'
)
plt.title('Proporción de éxito (y=1) por ocupación')
plt.xlabel('Proporción de suscripción ("si")')
plt.ylabel('Ocupación')
plt.grid(True, axis='x')
plt.show()
```

Interpretación:

- Las ocupaciones con mayor proporción de éxito ($y=1$) son:
 - "student": $\approx 31\%$
 - "retired": $\approx 25\%$
 - "unemployed": $\approx 14\%$
- Las ocupaciones con menor proporción de éxito:
 - "blue-collar" y "services": $< 10\%$
 - "entrepreneur", "housemaid", "self-employed", "technician": también en torno al 10%

Conclusiones preliminares:

- Hay fuertes diferencias en la tasa de suscripción según la ocupación.
- Estudiantes y jubilados son los grupos más propensos a suscribirse, lo que puede deberse a mayor disponibilidad de tiempo, menor exposición a productos financieros previos o perfil de riesgo conservador.
- Hay fuertes diferencias en la tasa de suscripción según la ocupación.
- Estudiantes y jubilados son los grupos más propensos a suscribirse, lo que puede deberse a mayor disponibilidad de tiempo, menor exposición a productos financieros previos o perfil de riesgo conservador.
- Los trabajadores manuales y de servicios presentan baja tasa de éxito, posiblemente asociado a menores ingresos, educación financiera o desconfianza en instituciones bancarias.

Recomendaciones/Ideas:

- Esta variable es **altamente informativa** y debe incluirse en el modelo.
- Puede ser útil agrupar ocupaciones en **niveles socioeconómicos** o en **segmentos de comportamiento financiero**.
- Analizar si el patrón observado se mantiene en combinación con otras variables como edad o nivel educativo.

Segmentaciones por atributos clave de la variable age

```
[ ]: # Genero gráfico boxplot de la proporción de plazos suscritos (y=1) según edad
plt.figure(figsize=(15,4))
sns.boxplot(x='age', y='y', data=df, orient='h');

[ ]: # Analizo la correlación entre edad y la variable objetivo
df.age.corr(df.y)

[ ]: # Reviso si hay correlación entre la variable objetivo frente a la edad y el hecho de que el cliente que sea jubilado
df['age'][df.job == 'retired'].corr(df.y)

[ ]: # Uso el mismo criterio que la celda anterior pero con clientes que sean de ocupación estudiante.
df['age'][df.job == 'student'].corr(df.y)
```

Interpretación (según el boxplot):

- Los clientes que se suscribieron ($y = 1$) tienden a ser ligeramente mayores en promedio que los que no lo hicieron ($y = 0$).
- El rango intercuartílico (Q1–Q3) para $y = 1$ está aproximadamente entre los **32 y 50 años**, con una mediana cercana a los **38–39 años**.
- Para $y = 0$, la mediana está más cerca de los **35 años**, y el rango intercuartílico va aproximadamente de **30 a 45 años**.
- Hay **outliers en ambos grupos** que superan los **70 años**, incluso hasta los **90+**, aunque representan una proporción baja del total.

Conclusiones preliminares:

- La variable `age` presenta una **distribución ligeramente desplazada hacia edades mayores** en los casos positivos ($y = 1$).

Conclusiones preliminares:

- La variable `age` presenta una **distribución ligeramente desplazada hacia edades mayores** en los casos positivos (`y = 1`).
- Esto sugiere que **las personas de mayor edad tienen más probabilidad de suscribirse**, aunque la diferencia no es drástica.
- La dispersión es amplia, por lo que podría haber un efecto no lineal (por ejemplo, aumento de probabilidad hasta cierta edad y luego caída).

Recomendaciones:

- Incluir la variable `age` en el modelo, posiblemente como una **variable continua**.
- Probar si **transformaciones no lineales** (como splines, categorización en tramos, o `age^2`) mejoran la capacidad predictiva.
- También se puede combinar con otras variables como ocupación o estado civil para identificar perfiles específicos por grupo etario.

Segmentaciones por atributos clave de la variable education

```
] #Calculo la proporción de plazos suscritos para cada valor posible de la columna educación
edu_proporcion = ((df.y[df.y == 1].groupby(df.education).count())/
                  (df.y.groupby(df.education).count()))*100
```

```
] # Proporción de plazos suscritos para cada nivel educativo
edu_proporcion
```

```
] # Genero gráfico de la proporción de plazos suscritos (y=1) para cada nivel educativo
education_rate = df.groupby('education')['y'].mean().sort_values()

education_rate.plot(
    kind='barh', figsize=(10,6), color='skyblue', edgecolor='black'
)
plt.title('Proporción de éxito (y=1) por nivel educativo')
plt.xlabel('Proporción de suscripción ("sí")')
plt.ylabel('Nivel educativo')
plt.grid(True, axis='x')
plt.show()
```

Interpretación:

- Las **proporciones de suscripción (`y=1`)** varían significativamente según el nivel educativo:
 - `"illiterate"` : **≈ 22%** (*mayor tasa de suscripción*)
 - `"unknown"` : **≈ 14%**
 - `"university.degree"` : **≈ 13.5%**
 - `"professional.course"`, `"high.school"` : **≈ 11-12%**
 - `"basic"` (4y, 6y, 9y): **≈ 8-10%**

Conclusiones preliminares:

- **Contrariamente a lo esperado**, los clientes analfabetos muestran la **mayor tasa de éxito**. Esto podría deberse a un tamaño muestral pequeño o a campañas especialmente dirigidas a ese grupo.
- En general, a mayor nivel educativo, **mayor probabilidad de suscripción**, aunque no de forma lineal.
- El grupo `"unknown"` tiene una proporción relativamente alta, lo cual sugiere que puede contener información útil y no debería eliminarse sin análisis adicional.

Recomendaciones/Ideas:

- Esta variable tiene **valor predictivo moderado a alto** y debe incluirse en el modelo.
- Puede ser útil agrupar los niveles en:
 - Bajo (`basic`)
 - Medio (`high.school`, `professional.course`)
 - Alto (`university.degree`)
 - Otros (`illiterate`, `unknown`)

Segmentaciones por atributos clave de la variable poutcome

```
#Calculo la proporción de plazos suscritos para cada valor posible de resultado de la campaña anterior
poutcome_proporcion = ((df.y[df.y == 1].groupby(df.poutcome).count())/
                       (df.y.groupby(df.poutcome).count()))*100
```

```
# Proporción de plazos suscritos según resultados de campañas previas
poutcome_proporcion
```



```
# Genero gráfico de la proporción de plazos suscritos (y=1) según resultados de campañas previas
poutcome_rate = df.groupby('poutcome')['y'].mean().sort_values()

poutcome_rate.plot(
    kind='barh', figsize=(9,4), color='mediumvioletred', edgecolor='black'
)
plt.title('Proporción de éxito (y=1) según resultado de campaña anterior')
plt.xlabel('Proporción de suscripción ("sí")')
plt.ylabel('Resultado campaña anterior')
plt.grid(True, axis='x')
plt.show()
```

Interpretación:

- La **proporción de suscripción (y=1)** varía significativamente según el resultado de la campaña anterior:
 - "success": $\approx 65\%$
 - "failure": $\approx 13\%$
 - "nonexistent": $\approx 9\%$

Conclusiones preliminares:

- Si la campaña anterior fue **exitosa**, hay **alta probabilidad de que el cliente vuelva a suscribirse** ($\sim 65\%$).
- Si la campaña **no existió** o fue un **fracaso**, la probabilidad de éxito actual baja mucho.
- Esta variable **es altamente informativa** y podría ser una de las más relevantes para predecir la variable objetivo.

Recomendaciones/Ideas:

- Incluir esta variable en el modelo predictivo como un factor clave.
- Puede ser útil generar una variable binaria adicional que indique si el cliente tuvo una campaña previa o no.
- También puede ser útil explorar interacciones con el número de contactos o canal de comunicación para profundizar el análisis.

Segmentaciones por atributos clave de la variable marital

```
#Calculo la proporción de plazos suscritos para cada valor posible de estado civil
marital_proporcion = ((df.y[df.y == 1].groupby(df.marital).count())/
    (df.y.groupby(df.marital).count()))*100
```

```
# Proporción de plazos suscritos para cada valor posible de estado civil
marital_proporcion
```

```
# Genero gráfico de la proporción de plazos suscritos (y=1) para cada valor posible de estado civil
marital_rate = df.groupby('marital')['y'].mean().sort_values()

marital_rate.plot(
    kind='barh', figsize=(10,6), color='salmon', edgecolor='black'
)
plt.title('Proporción de éxito (y=1) por estado civil')
plt.xlabel('Proporción de suscripción ("sí")')
plt.ylabel('Estado civil')
plt.grid(True, axis='x')
plt.show()
```

Interpretación:

- La **proporción de suscripción (y=1)** varía levemente según el estado civil:
 - "unknown": $\approx 15\%$
 - "single": $\approx 14\%$
 - "divorced": $\approx 10.3\%$
 - "married": $\approx 10.2\%$

Conclusiones preliminares:

- Las personas **solteras o con estado civil desconocido** presentan **mayor tasa de suscripción** al depósito a plazo fijo.
- Los **clientes casados o divorciados** muestran proporciones más bajas de éxito.
- Aunque las diferencias no son enormes, esta variable podría aportar **valor moderado** al modelo.

Recomendaciones/Ideas:

Recomendaciones/Ideas:

- Considerar esta variable como parte del modelo, especialmente si se combina con edad o ingresos.
- Evaluar si la categoría "unknown" está sesgada hacia algún segmento particular (por ejemplo, omisiones sistemáticas en ciertas edades o profesiones).
- Posible transformación: agrupar "married" y "divorced" como "no solteros", si mejora la capacidad predictiva del modelo.

Segmentaciones por atributos clave de la variable default

```
#Calculo La proporción de plazos suscriptos para cada valor posible de default
default_proporcion = ((df.y[df.y == 1].groupby(df.default).count())/
                      (df.y.groupby(df.default).count()))*100

# Proporción de plazos suscriptos para cada valor posible de default
default_proporcion

# Genero gráfico de la proporción de plazos suscritos (y=1) para cada valor posible de default
default_rate = df.groupby('default')['y'].mean().sort_values()

default_rate.plot(
    kind='barh', figsize=(8,4), color='tomato', edgecolor='black'
)
plt.title('Proporción de éxito (y=1) según historial de default')
plt.xlabel('Proporción de suscripción ("sí")')
plt.ylabel('¿Tiene créditos en default?')
plt.grid(True, axis='x')
plt.show()
```

Interpretación:

- **Proporción de suscripción (y=1)** por categoría:
 - "no" : ≈ **12.9%**
 - "unknown" : ≈ **5.1%**
 - "yes" : ≈ **0%** (sin suscripciones en esta categoría)

Conclusiones preliminares:

Conclusiones preliminares:

- **Clientes sin historial de default (no)** tienen la **mayor tasa de éxito**, lo cual es esperable.
- **Ningún cliente con historial de default (yes) se ha suscrito**, lo que indica una **relación fuertemente negativa** con la variable objetivo.
- "unknown" tiene una proporción intermedia pero significativamente más baja que "no" , lo que sugiere cierta incertidumbre o desconfianza.

Recomendaciones/Ideas:

- Esta variable tiene **alto poder discriminante** y debe ser considerada clave en el modelo.
- El valor "yes" puede ser un indicador crítico para **segmentar negativamente** la base de clientes.
- Verificar el tamaño del grupo "yes" : si es muy pequeño, su impacto podría ser limitado pero igualmente útil para reglas de negocio.
- Considerar imputar o categorizar "unknown" si se puede cruzar con otras variables de comportamiento financiero.

Segmentaciones por atributos clave de la variable housing

```
#Calculo La proporción de plazos suscriptos para cada valor posible de housing
housing_proporcion = ((df.y[df.y == 1].groupby(df.housing).count())/
                      (df.y.groupby(df.housing).count()))*100

# Proporción de plazos suscriptos para cada valor posible de housing
housing_proporcion

# Genero gráfico de la proporción de plazos suscritos (y=1) para cada valor posible de housing
housing_rate = df.groupby('housing')['y'].mean().sort_values()

housing_rate.plot(
    kind='barh', figsize=(8,4), color='steelblue', edgecolor='black'
)
plt.title('Proporción de éxito (y=1) según tenencia de préstamo hipotecario')
plt.xlabel('Proporción de suscripción ("sí")')
plt.ylabel('¿Tiene préstamo hipotecario?')
plt.grid(True, axis='x')
plt.show()
```

Interpretación:

- Las **proporciones de suscripción (y=1)** son similares entre las categorías:
 - "yes" : ≈ **11.6%**

Interpretación:

- Las **proporciones de suscripción (y=1)** son similares entre las categorías:
 - "yes": $\approx 11.6\%$
 - "no": $\approx 11.0\%$
 - "unknown": $\approx 10.9\%$

Conclusiones preliminares:

- La tenencia de un **préstamo hipotecario no parece ser un factor decisivo** para predecir si un cliente se suscribirá.
- Las diferencias entre las categorías son **muy pequeñas**, por lo tanto esta variable tiene **bajo poder discriminante** por sí sola.

Recomendaciones/Ideas:

- Esta variable podría **no tener un impacto fuerte de manera individual**, pero aún así incluirla en el modelo puede aportar valor en combinación con otras variables (por ejemplo: ingresos, edad, ocupación).
- Es recomendable revisar si la categoría "unknown" representa un grupo particular de clientes, o simplemente datos faltantes que pueden ser imputados o tratados como una categoría aparte.
- Considerar la creación de una variable binaria simplificada (**tiene_prestamo_hipotecario**) si se busca reducir dimensionalidad sin pérdida de información.

Segmentaciones por atributos clave de la variable loan

```
] : #Calculo la proporción de plazos suscritos para cada valor posible de loan
loan_proporcion = ((df.y[df.y == 1].groupby(df.loan).count())/
                  (df.y.groupby(df.loan).count()))*100

] : # Proporción de plazos suscritos para cada valor posible de loan
loan_proporcion

# Genero gráfico de la proporción de plazos suscritos (y=1) para cada valor posible de loan
loan_rate = df.groupby('loan')['y'].mean().sort_values()

loan_rate.plot(
    kind='barh', figsize=(8,4), color='darkorange', edgecolor='black'
)
plt.title('Proporción de éxito (y=1) según tenencia de préstamo personal')
plt.xlabel('Proporción de suscripción ("sí")')
plt.ylabel('¿Tiene préstamo personal?')
plt.grid(True, axis='x')
plt.show()
```

Interpretación:

- La **proporción de suscripción (y=1)** es muy similar entre las tres categorías:
 - "no": $\sim 10.9\%$
 - "yes": $\sim 10.7\%$
 - "unknown": $\sim 10.6\%$

Conclusiones preliminares:

- La **tenencia de un préstamo personal no parece tener una influencia clara** sobre la probabilidad de suscripción. Las diferencias son mínimas.
- Esta variable **no discrimina bien** entre quienes se suscriben y quienes no.
- La categoría "unknown" tiene una proporción levemente más baja, aunque esto podría deberse a ruido o falta de datos.

Recomendaciones/Ideas:

- Esta variable **podría tener baja importancia predictiva** en un modelo de clasificación.
- Se recomienda explorar **interacciones con otras variables** (por ejemplo: edad, ingresos, estado civil) para detectar posibles patrones no lineales.
- Evaluar si el valor "unknown" debe imputarse o conservarse como categoría separada, según el análisis de los datos faltantes.

2.5 Insights preliminares

A) Información general

- El conjunto de datos contiene **21 variables** y **41.176 registros**.
- No se detectan **valores nulos explícitos** ni **duplicados**, lo cual indica una buena calidad inicial del dataset.
- Se identifican **10 variables numéricas**, **10 categóricas** y **1 booleana**.

B) Variable objetivo (y)

- La variable objetivo está **desbalanceada**, con una distribución marcada hacia la clase negativa (**no**), lo que implica la necesidad de utilizar **técnicas de balanceo** (como sobremuestreo, submuestreo o SMOTE) en la etapa de modelado.

C) Valores atípicos y dominantes

- La variable **pdays** muestra un valor dominante de **999**, que denota ausencia de contacto previo. Se deberá derivar una **variable binaria** (**was_contacted**) a partir de esta.
- campaign** y **previous** presentan distribuciones con valores extremos o altamente sesgadas, lo que sugiere la existencia de **outliers** que deben ser tratados o analizados.
- age** tiene un rango amplio (hasta 98 años), por lo que podría resultar útil **agrupar en tramos etarios** para facilitar su análisis e interpretación.

D) Variables categóricas con valores unknown

- Varias variables categóricas contienen el valor **'unknown'**, lo que representa **valores faltantes implícitos**. En particular:
 - default**, **education**, **job**, **contact**, **outcome** y **housing**.
- Evaluar si estos valores deben tratarse adecuadamente mediante **imputación**, **recategorización** o **exclusión**, dependiendo del análisis.

E) Correlaciones relevantes

- Se observan **altas correlaciones** entre variables macroeconómicas, como:
 - euribor3m** y **nr.employed**.
 - emp.var.rate** y **euribor3m**.
- Esto sugiere **redundancia de información** y puede evaluarse el uso de **reducción de dimensionalidad** (PCA) o selección de atributos.

F) Recomendaciones/Ideas generales para secciones posteriores

- Debemos pensar si se deberán tratar explícitamente los valores **'unknown'** como datos faltantes.
- Crear variables derivadas útiles (por ejemplo, binarizar **pdays** y agrupar **age**).

F) Recomendaciones/Ideas generales para secciones posteriores

- Debemos pensar si se deberán tratar explícitamente los valores **'unknown'** como datos faltantes.
- Crear variables derivadas útiles (por ejemplo, binarizar **pdays** y agrupar **age**).
- Abordar el desequilibrio de la variable objetivo antes del modelado.
- Analizar y mitigar el impacto de valores extremos en algunas variables clave.
- Evaluar la pertinencia de mantener variables altamente correlacionadas o aplicar técnicas de reducción.

3 - PREPROCESAMIENTO DE DATOS

A partir de aquí, comienzo a realizar una serie de preprocesamientos sobre el dataframe **df** para adecuar los datos.

Algunas modificaciones involucran:

- Imputación o modificación de valores faltantes o inconsistentes (tanto explícitos como implícitos).**
- Encoding de variables categóricas.**
- Transformación de variables numéricas**

Posteriormente, se realizarán tareas de **feature engineering**, donde se buscará crear nuevas variables derivadas o transformar las existentes para mejorar el rendimiento del modelo.

3.1 Tratamiento de valores faltantes o inconsistentes

Imputación de valores faltantes en job

Ahora, para inferir los valores faltantes en "job" y "education", utilizo la tabulación cruzada entre ambos. La hipótesis a considerar es que el "job" se ve influenciado por la "education" de una persona. Por lo tanto, es factible inferir "job" basándose en el nivel educativo de la persona.

```
# Genero una función para una tabla cruzada entre dos variables categóricas del DataFrame.  
# Para cada categoría de la segunda variable (f2), se calcula cuántas veces aparece cada categoría de la primera variable (f1).  
# Devuelve un DataFrame donde las filas son las categorías de f1, las columnas son las de f2, y los valores representan los conteos.  
def cross_tab(df, f1, f2):  
    jobs=list(df[f1].unique())  
    edu=list(df[f2].unique())
```

```
# Genero una función para una tabla cruzada entre dos variables categóricas del DataFrame.
# Para cada categoría de la segunda variable (f2), se calcula cuántas veces aparece cada categoría de la primera variable (f1).
# Devuelve un DataFrame donde las filas son las categorías de f1, las columnas son las de f2, y los valores representan los conteos.
def cross_tab(df,f1,f2):
    jobs=list(df[f1].unique())
    edu=list(df[f2].unique())
    dataframes=[]
    for e in edu:
        dfe=df[df[f2]==e]
        dfejob=dfe.groupby(f1).count()[f2]
        dataframes.append(dfejob)
    xx=pd.concat(dataframes,axis=1)
    xx.columns=edu
    xx=xx.fillna(0)
    return xx
```

```
# Ahora, muestro la cantidad de personas por combinación de job y education.
cross_tab(df,'job','education')
```

También propongo que personas cuya edad sea mayor a 60 probablemente sea jubilado(retired) en caso de ser desconocida su situación.

```
# Distribución de categorías de job entre personas mayores de 60 años.
df['job'][df['age']>60].value_counts()
```

Inferir la educación a partir de los empleos: De la tabulación cruzada, se observa que las personas con puestos directivos suelen tener un título universitario. Por lo tanto, donde 'job' = management y 'education' = unknown, podemos sustituir 'education' por 'university.degree'. De forma similar, 'job' = 'services' --> 'education' = 'high.school' y 'job' = 'housemaid' --> 'education' = 'basic.4y'.

Inferir los empleos a partir de la educación: Si 'education' = 'basic.4y', 'basic.6y' o 'basic.9y', el 'job' suele ser 'blue-collar'. Si 'education' = 'professional.course', el 'job' es 'technician'.

Inferir los empleos a partir de la edad: Como se ve, si 'age' > 60, el 'job' es 'retired', lo cual tiene sentido.

Al imputar los valores de empleo y educación, tuve en cuenta que las correlaciones debían ser coherentes con la realidad. Si no lo eran, no reemplazaba los valores faltantes.

```
#Genero una serie de transformaciones que imputen valores en 'job' cuando se den ciertas condiciones en otras variables
df.loc[(df['age']>60) & (df['job']=='unknown'), 'job'] = 'retired'
df.loc[(df['education']=='unknown') & (df['job']=='management'), 'education'] = 'university.degree'
df.loc[(df['education']=='unknown') & (df['job']=='services'), 'education'] = 'high.school'
df.loc[(df['education']=='unknown') & (df['job']=='housemaid'), 'education'] = 'basic.4y'
df.loc[(df['job'] == 'unknown') & (df['education']=='basic.4y'), 'job'] = 'blue-collar'
```

```
#Genero una serie de transformaciones que imputen valores en 'job' cuando se den ciertas condiciones en otras variables
df.loc[(df['age']>60) & (df['job']=='unknown'), 'job'] = 'retired'
df.loc[(df['education']=='unknown') & (df['job']=='management'), 'education'] = 'university.degree'
df.loc[(df['education']=='unknown') & (df['job']=='services'), 'education'] = 'high.school'
df.loc[(df['education']=='unknown') & (df['job']=='housemaid'), 'education'] = 'basic.4y'
df.loc[(df['job'] == 'unknown') & (df['education']=='basic.4y'), 'job'] = 'blue-collar'
df.loc[(df['job'] == 'unknown') & (df['education']=='basic.6y'), 'job'] = 'blue-collar'
df.loc[(df['job'] == 'unknown') & (df['education']=='basic.9y'), 'job'] = 'blue-collar'
df.loc[(df['job']=='unknown') & (df['education']=='professional.course'), 'job'] = 'technician'
```

```
# Vuelvo a mostrar la cantidad de personas por combinación de job y education.
cross_tab(df,'job','education')
```

De esta manera, se reduce el número de 'unknowns' y mejoro el dataset.

Imputación valores faltantes pdays

Según la fuente de los datos (Repositorio de ML de U.C. Irvine), los valores faltantes, o NaN, se codifican como '999'. De la tabla anterior, se desprende claramente que solo 'pdays' presenta valores faltantes. Además y en virtud a lo arriba explicitado, la mayoría de los valores de 'pdays' están faltantes.

```
# Uso esta función para crear un histograma con matplotlib para visualizar la distribución de una variable numérica.
def drawhist(df, feature):
    plt.hist(df[feature], bins=20, edgecolor='black')
    plt.title(f'Distribución de {feature}')
    plt.xlabel(feature)
    plt.ylabel('Frecuencia')
    plt.show()
```

```
# Muestro el histograma de la variable 'pdays', incluyendo todos los valores (incluido 999).
drawhist(df,'pdays')
plt.show()
# Muestro el histograma de 'pdays' excluyendo los valores 999 (que indican que no hubo contacto previo).
plt.hist(df.loc[df.pdays != 999, 'pdays'])
plt.show()
```

```
# Creo una tabla cruzada entre 'pdays' y 'poutcome', mostrando la proporción (normalizada) de registros por combinación.
# Se usa 'age' como columna de referencia solo para contar, sin afectar el resultado.
pd.crosstab(df['pdays'],df['poutcome'], values=df['age'], aggfunc='count', normalize=True)
```

Como se puede observar en la tabla anterior, la mayoría de los valores de "pdays" faltan. La mayoría de estos valores faltantes se producen cuando el "outcome" es un valor faltante. Esto significa que la mayoría de los valores de "pdays" faltan porque nunca se contactó al cliente. Para abordar esta variable, se elimina la variable numérica "pdays" y se la reemplaza por el valor 0.

```
# Transformación de los valores 999 de la variable 'pdays'
df['pdays'] = df.pdays.map(lambda x: 0 if x == 999 else x)

# Muestro nuevamente el histograma de la variable 'pdays', incluyendo todos los valores (incluido 999).
drawhist(df, 'pdays')
plt.show()
# Muestro nuevamente el histograma de 'pdays' excluyendo los valores 999 (que indican que no hubo contacto previo).
plt.hist(df.loc[df.pdays != 999, 'pdays'])
plt.show()
```

Nota sobre Valores atípicos

Los valores atípicos se definen como $1.5 \times \text{valor Q3}$ (percentil 75). De las visualizaciones y análisis en la etapa de EDA, se observa que solo 'age' y 'campaign' presentan valores atípicos como $\text{max}(\text{age})$ y $\text{max}(\text{campaign}) > 1.5Q3(\text{age})$ y $> 1.5Q3(\text{campaign})$, respectivamente.

Pero también observamos que el valor de estos valores atípicos es bastante realista ($\text{max}(\text{age}) = 98$ y $\text{max}(\text{campaign}) = 56$). Por lo tanto, no es necesario eliminarlos, ya que el modelo de predicción debe representar el mundo real. Esto mejora la generalización del modelo y lo hace robusto para situaciones reales. Por lo tanto y en este caso, los valores atípicos no se eliminarán.

3.2 Encoding de variables categóricas

Encoding de las variables relativas a días de la semana y meses del año

Dado que tanto los días de la semana como los meses poseen un ordenamiento, considero codificarlos según su orden natural. Pero en primer lugar procedo a visualizar la distribución de valores para cada variable.

```
#Reviso la distribución de valores en la variable month
df['month'].value_counts()

#Reviso la distribución de valores en la variable day_of_week
df['day_of_week'].value_counts()
```

```
#Reviso la distribución de valores en la variable day_of_week
df['day_of_week'].value_counts()

# En la variable month y day_of_week, cambio sus valores categóricos (y en string) por valores numéricos.
#Esta transformación con los números respectivos para los meses y días suele usarse para el paso de este tipo de variables categóricas a numéricas.

df.month.replace(['jan', 'feb', 'mar', 'apr', 'may', 'jun', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec'],
                 (1,2,3,4,5,6,7,8,9,10,11,12), inplace=True)
df.day_of_week.replace(['mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun'],
                      (1,2,3,4,5,6,7), inplace=True)
```

```
#Reviso que la transformación de la variable month haya sido exitosa
df['month'].value_counts()
```

```
#Reviso que la transformación de la variable day_of_week haya sido exitosa
df['day_of_week'].value_counts()
```

3.3 Transformaciones de variables numéricas

En sección me propongo transformar aquella variable numérica que pueda beneficiarse de dicho procesamiento.

```
#Extraigo las variables numéricas del dataset y obtengo información relevante para analizar
numerical_variables = ['age', 'campaign', 'pdays', 'previous', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m',
                      'nr.employed']
df[numerical_variables].describe()
```

Observo que la variable 'age' posee una gran amplitud, la cual podría segmentarse en una nueva variable asociada.

```
# Defino los límites de edad para cada grupo y sus etiquetas
bins = [0, 25, 40, 60, 140]
labels = ['young', 'lower middle aged', 'middle aged', 'senior']

# Genero la columna 'age_binned' con los grupos etiquetados sobre la columna 'age'
df['age_binned'] = pd.cut(df['age'], bins=bins, labels=labels, right=True, include_lowest=True)

# Verifico la distribución por grupos resultantes
df['age_binned'].value_counts()
```

4 - INGENIERÍA DE CARACTERÍSTICAS

Ingeniería de Características

En esta sección se generan las variables que serán utilizadas por los modelos de *machine learning*. La ingeniería de características consiste en transformar, crear o seleccionar atributos a partir del conjunto de datos original, con el objetivo de mejorar el rendimiento del modelo y capturar relaciones relevantes entre las variables. Estas transformaciones no solo buscan mejorar la precisión, sino también la interpretabilidad y eficiencia del modelo.

Análisis de correlación entre variables numéricas relevantes

```
]# Selección de variables
M = df[['emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m',
        'nr.employed', 'campaign', 'pdays', 'previous', 'y']]

# Figura y máscara
fig, ax = plt.subplots(figsize=(12, 10))
mask = np.triu(np.ones_like(M.corr(), dtype=bool))

# Heatmap
sns.heatmap(M.corr(), mask=mask, annot=True, cmap='coolwarm',
            fmt=".2f", linewidths=0.5, cbar_kws={"shrink": 0.8}, ax=ax)

# Ajustes para evitar recortes
plt.title("Matriz de correlación entre variables numéricas seleccionadas", fontsize=14, pad=12)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
ax.set_ylim(len(M.columns), 0) # Fuerza que aparezca el eje Y completo
plt.tight_layout()             # Ajuste general de espaciado

plt.show()
```

Análisis de correlación entre variables numéricas

Se construyó una matriz de correlación para examinar la relación lineal entre las variables numéricas seleccionadas, incluyendo la variable objetivo `y`.

A partir del heatmap generado, se pueden destacar los siguientes hallazgos relevantes:

- **Alta correlación positiva** entre:

Análisis de correlación entre variables numéricas

Se construyó una matriz de correlación para examinar la relación lineal entre las variables numéricas seleccionadas, incluyendo la variable objetivo `y`.

A partir del heatmap generado, se pueden destacar los siguientes hallazgos relevantes:

- **Alta correlación positiva** entre:

- `euribor3m` y `emp.var.rate` ($r = 0.97$)
- `euribor3m` y `nr.employed` ($r = 0.94$)
- `emp.var.rate` y `nr.employed` ($r = 0.91$)

Estas asociaciones indican una fuerte redundancia entre variables macroeconómicas, lo que podría justificar la eliminación de alguna de ellas o la aplicación de técnicas de reducción de dimensionalidad.

- **Correlaciones con la variable objetivo `y`:**

- `euribor3m` ($r = -0.31$)
- `nr.employed` ($r = -0.35$)
- `emp.var.rate` ($r = -0.30$)
- `pdays` ($r = 0.27$)
- `previous` ($r = 0.23$)

Estas variables muestran correlaciones más relevantes con el resultado de la campaña (`y`), por lo que podrían tener valor predictivo en etapas posteriores del modelado.

- **Variables como `campaign` y `cons.conf.idx` presentan baja correlación lineal** con el resto de las variables, lo que sugiere independencia lineal, aunque podrían aportar desde otras perspectivas no lineales.

En conjunto, este análisis de correlación permite identificar tanto redundancias como posibles variables relevantes, lo que guiará decisiones de selección y transformación de atributos en las próximas etapas del proyecto.

4.1 Agrupaciones o transformaciones lógicas

Agrupación de valores en variable job

```
# Agrupación de valores en variable job
df["job"].value_counts(normalize=True)

# Creo un diccionario para agrupar las categorías de la variable 'job' en grupos más generales e interpretables.
job_map = {
    'admin.': 'White-collar',
    'management': 'White-collar',
    'technician': 'White-collar',

    'blue-collar': 'Blue-collar',
    'services': 'Blue-collar',
    'housemaid': 'Blue-collar',

    'entrepreneur': 'Self-employed',
    'self-employed': 'Self-employed',

    'retired': 'Non-active',
    'student': 'Non-active',
    'unemployed': 'Non-active',

    'unknown': 'Other'
}

# Aplico el mapeo a la columna 'job' para crear una nueva variable agrupada: 'job_grouped'.
df['job_grouped'] = df['job'].map(job_map)

# Verifico las nuevas proporciones
df['job_grouped'].value_counts()
```

Agrupación de valores en variable education

```
# Agrupación de valores en variable education
df['education'].value_counts()

# Al igual que con job, defino mapeo a categorías agrupadas
education_map = {

# Agrupación de valores en variable education
df['education'].value_counts()

# Al igual que con job, defino mapeo a categorías agrupadas
education_map = {
    'basic.9y': 'Basic',
    'basic.4y': 'Basic',
    'basic.6y': 'Basic',
    'high.school': 'Middle',
    'professional.course': 'Middle',
    'university.degree': 'Superior',
    'unknown': 'Other',
    'illiterate': 'Other'
}

# Aplico la creación de la nueva variable agrupada
df['education_grouped'] = df['education'].map(education_map)

# Verifico el conteo
df['education_grouped'].value_counts()
```

4.2 Generación de nuevas variables derivadas

A partir de variables ya establecidas, procedo a generar algunas variables derivadas que puedan enriquecer y potenciar mi dataset.

Generación de variable para identificar existencia o no de contactos previos con el cliente.

Habida cuenta que existe una alta presencia (mayoritaria incluso) de valores 0 en la variable 'previous' decido generar una variable derivada. Esta nueva variable permitirá identificar si el cliente ha sido contactado previamente, al menos 1 vez.

```
# Creación de variable contacted_previously (contactado previamente). Si el cliente tuvo al menos 1 contacto previo su valor será 1, sino será 0
df['contacted_previously'] = (df['previous'] >= 1).astype(int)

df['contacted_previously'].value_counts()
```

Generación de variable para combinar simultáneamente la edad y estado civil del cliente.

Generación de variable para combinar simultáneamente la edad y estado civil del cliente.

En este caso genero una variable adicional que concatena los valores de 'age_binned' y 'marital' para así mostrar una mayor dimensionalidad del cliente.

```

: #Creo variable 'life_stage' que se obtiene al concatenar el segmento etario y el estado civil.
df['life_stage'] = df['age_binned'].astype(str) + ' & ' + df['marital']
#Cuento los valores resultantes para la nueva variable
df['life_stage'].value_counts()

```

Generación de variable para combinar simultáneamente el empleo y nivel educativo del cliente.

En este caso genero una variable adicional que concatena los valores de 'job' y 'education' para así mostrar una mayor dimensionalidad del cliente.

```

: #Creo variable 'socio-economic' que se obtiene al concatenar el oficio/trabajo y el grado educativo obtenido.
df['socio-economic'] = df['job'].astype(str) + ' & ' + df['education']
#Cuento los valores resultantes para la nueva variable
df['socio-economic'].value_counts()

```

4.3 Eliminación de variables redundantes

En esta sección procedo a eliminar una de las variables macroeconómicas que observé que podrías redundante durante el EDA.

```

: # Eliminación de variable macroeconómica redundante
df.drop(columns=['nr.employed'], inplace=True)

# Verificación de que la columna fue efectivamente eliminada
print("Columnas actuales en el DataFrame:")
print(df.columns.tolist())

```

Eliminación de variable macroeconómica con baja relevancia o redundancia

Como parte del proceso de selección de variables, se identificó y eliminó la siguiente variable macroeconómica del conjunto de datos:

- nr.employed

La decisión se basó en los siguientes criterios:

La decisión se basó en los siguientes criterios:

- Redundancia por alta correlación entre variables
 - nr.employed presentó una **alta correlación** con emp.var.rate (r = 0.91) y euribor3m (r = 0.94).
 - Estas tres variables capturan información económica muy similar, lo cual puede generar problemas de **multicolinealidad**.
 - Se optó por conservar euribor3m, que mostró la **mayor correlación con la variable objetivo y** y ofrece una interpretación macroeconómica relevante.
- Interpretabilidad limitada
 - Esta variable representa índices económicos agregados cuya relación directa con la decisión del cliente bancario es difusa.
 - En un enfoque orientado a la explicación y claridad de resultados, se priorizaron variables más fácilmente interpretables.

En consecuencia, se decidió eliminar esta variable para **reducir la dimensionalidad, minimizar redundancias y conservar únicamente aquellas variables con mayor relevancia estadística y práctica** para el análisis y modelado predictivo.

4.4 Encoding de variables categóricas (OneHotEncoder)

En esta sección se realiza la codificación **One-Hot Encoding** sobre las variables categóricas seleccionadas (cat_cols1). Esta técnica transforma cada categoría en una nueva columna binaria (0 o 1), permitiendo que los modelos de machine learning puedan procesar variables categóricas de manera adecuada.

Finalmente, se eliminan las columnas categóricas originales y se agregan las columnas codificadas al DataFrame principal, manteniendo la correspondencia de índices para preservar la integridad de los datos.

```

: # Primero obtengo las variables categóricas del dataframe
cat_cols1= df.select_dtypes(include=['object', 'category']).columns

: #Reviso cuáles variables se encuentran almacenadas en cat_cols1
cat_cols1

: #Observo los tipos de datos de cada variable, antes de efectuar el encoding con OneHotEncoder.
df.info()

```

```
# Inicializo OneHotEncoder con algunos parámetros clave:
# - sparse_output=False: Devuelve un array denso (para mejor compatibilidad con pandas)
# - handle_unknown='ignore': Permitirá manejar categorías no vistas durante el entrenamiento
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')

# Realizo una conversión preventiva a string para evitar errores con tipos mixtos
df[cat_cols1] = df[cat_cols1].astype(str)

# Aplico el ajuste y transformación
ohe_array = ohe.fit_transform(df[cat_cols1])
ohe_columns = ohe.get_feature_names_out(cat_cols1)

# Efectúo la reconstrucción del DataFrame:
df_ohe = pd.DataFrame(ohe_array, columns=ohe_columns, index=df.index) # 1. Creo DF con las nuevas columnas one-hot (manteniendo el índice original)
df.drop(columns=cat_cols1, inplace=True) # 2. Elimino columnas categóricas originales
df = pd.concat([df, df_ohe], axis=1) # 3. Concateno con el DataFrame original

# Hago el guardado del encoder para producción debido a que:
# - Es útil para aplicar la misma transformación a nuevos datos
# - El formato .pkl mantiene todos los parámetros aprendidos
onehot_encoder = ohe
joblib.dump(onehot_encoder, 'onehot_encoder.pkl')

#Reviso por última vez los tipos de datos de cada variable, luego de efectuar el encoding con OneHotEncoder.
df.info()
```

5 - MODELADO PREDICTIVO

Ahora que poseo los elementos necesarios para alimentar esta primera etapa de modelización de ML, genero una división de los datos estratificada según la variable objetivo, para luego pasarle al modelo y validar su eficiencia.

5.1 - División del conjunto en entrenamiento y test

```
] : # Divido el dataset en entrenamiento y prueba (80% train, 20% test) Y estratificado según variable objetivo
X_train, X_test, y_train, y_test = train_test_split(df.drop('y',axis=1),
                                                    df.y,
                                                    test_size=0.2,
                                                    random_state=42,
                                                    stratify = df.y)

]: #Reviso cómo ha quedado la división X_train
X_train

]: #Reviso cómo ha quedado la división X_test
X_test

]: #Reviso cómo ha quedado la división y_train
y_train

]: #Reviso cómo ha quedado la división y_test
y_test
```

Selección automática de features

La selección de variables es una etapa clave en todo proceso de modelado predictivo, ya que permite reducir la dimensionalidad del conjunto de datos, eliminar redundancias y mejorar el rendimiento de los modelos. Para este trabajo se testearon distintos métodos de selección automática, tales como SelectKBest, importancia de variables basada en modelos (como Random Forest y XGBoost), y enfoques recursivos como RFECV (el cual se terminó seleccionando como el método con mejor rendimiento). Estas técnicas ayudan a identificar las características más relevantes que explican la variable objetivo, optimizando así tanto la capacidad predictiva como la interpretabilidad del modelo final.

```
] : #Vuelvo a generar un heatmap (mapa de calor) de las correlaciones entre las variables numéricas del DataFrame previo
#Esto(ahora que poseo todas las variables como numéricas) puede permitirme analizar las posibilidades sobre selección de variables.
%matplotlib inline
```

```
#Vuelvo a generar un heatmap (mapa de calor) de las correlaciones entre las variables numéricas del DataFrame previo
#Esto(ahora que poseo todas las variables como numéricas) puede permitirme analizar las posibilidades sobre selección de variables.
%matplotlib inline
df_correlacion = df.select_dtypes(include=['number'])
correlation_mat = df_correlacion.corr()
sns.heatmap(correlation_mat)
plt.show()
```

Del análisis del mapa de calor de correlaciones se puede aventurar que no se identifican pares de variables con alta correlación (por ejemplo, $|r| > 0.85$) que justifiquen su eliminación. En general, las correlaciones entre variables numéricas son bajas, lo que indica que cada atributo aporta información relativamente independiente. Esta observación respaldaría la inclusión de una cantidad mayor de variables a las esperadas en etapas posteriores del modelado.

A continuación y para optimizar el conjunto de variables utilizadas en el modelo, se aplicó el método Recursive Feature Elimination with Cross-Validation (RFECV) utilizando un clasificador XGBoost como estimador base. Esta técnica evalúa recursivamente la importancia de las variables, eliminando en cada iteración la menos relevante, y valida el desempeño a través de validación cruzada estratificada con 5 particiones. Como métrica de evaluación se empleó el F1-score, debido al desbalance de clases presente en el dataset. El proceso permitió identificar el subconjunto de variables más relevantes para la tarea de clasificación, lo que contribuye a reducir la complejidad del modelo, mejorar su interpretabilidad y potencialmente mitigar el riesgo de sobreajuste.

```
# Empleo modelo base con XGBoost
modelo_xgb = XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42,
    scale_pos_weight=7.85
)

# Realizo la validación cruzada estratificada
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Ejecuto RFECV
selector_rfecv = RFECV(
    estimator=modelo_xgb,
    step=1,
    cv=cv,
    scoring='f1',
    n_jobs=-1,
    verbose=2
)
selector_rfecv.fit(X_train, y_train)
```

```
# Empleo modelo base con XGBoost
modelo_xgb = XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42,
    scale_pos_weight=7.85
)

# Realizo la validación cruzada estratificada
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Ejecuto RFECV
selector_rfecv = RFECV(
    estimator=modelo_xgb,
    step=1,
    cv=cv,
    scoring='f1',
    n_jobs=-1,
    verbose=2
)
selector_rfecv.fit(X_train, y_train)

# Obtengo las features seleccionadas
selected_features_rfecv = X_train.columns[selector_rfecv.support_].tolist()
print(f"Se seleccionaron {len(selected_features_rfecv)} variables:")
print(selected_features_rfecv)
```

Como resultado, el algoritmo seleccionó un total de **108 variables** de un total inicial de 162.

Entre las variables seleccionadas se incluyen:

- Variables originales: `age`, `month`, `duration`, `emp.var.rate`, etc.
- Variables transformadas y codificadas: `job_admin.`, `housing_yes`, `poutcome_success`, etc.
- Variables creadas durante el preprocesamiento: `age_binned_middle`, `aged`, `job_grouped_White-collar`, `socio-economic_management` & `university.degree`, entre otras.

Estas variables seleccionadas se utilizaron en el pipeline final para entrenar el modelo definitivo.

```
# Guardo la lista de features seleccionadas para un potencial uso posterior
joblib.dump(selected_features_rfecv, 'selected_features_rfecv.pkl')
```

```
# Aplico las mismas características seleccionadas al conjunto de train y test para mantener la consistencia en el espacio de características
X_train = X_train[selected_features_rfecv] # Filtro solo las features seleccionadas en el train
X_test = X_test[selected_features_rfecv] # Aplico el mismo filtro al test set
```

Escalado de los datos

El escalado es una etapa clave del preprocesamiento, especialmente cuando se utilizan modelos sensibles a la magnitud de las variables. En este TFM, las variables numéricas como la duración de llamada, la edad o el número de contactos presentan distintas escalas, lo que podría sesgar el modelo.

Para garantizar que cada variable contribuya equitativamente al aprendizaje, se aplica `MinMaxScaler`, que normaliza los valores en un rango de 0 a 1. Esto mejora la estabilidad del modelo, acelera la convergencia y evita que variables con mayor rango dominen el proceso de entrenamiento.

```
# Inicializo el escalador MinMax para normalizar los datos al rango [0, 1]
scaler = MinMaxScaler()

# Guardo los índices y columnas originales
columns = X_train.columns # Nombres originales de las features
index_train = X_train.index # Índices del conjunto de entrenamiento
index_test = X_test.index # Índices del conjunto de prueba

# Escalo y reconstruyo los DataFrames con nombres
X_train = pd.DataFrame(scaler.fit_transform(X_train), columns=columns, index=index_train)
X_test = pd.DataFrame(scaler.transform(X_test), columns=columns, index=index_test)

# Guardo el scaler en formato pickle
joblib.dump(scaler, 'minmax_scaler.pkl')
```

Balanceo de los datos

El dataset presenta un fuerte desbalance entre las clases: la mayoría de los clientes no contrata el depósito, mientras que solo un pequeño porcentaje sí lo hace. Esta desproporción puede sesgar al modelo hacia la clase mayoritaria, afectando negativamente su capacidad para identificar correctamente los casos positivos.

Para abordar este problema, se aplica la técnica de sobremuestreo **SMOTE (Synthetic Minority Over-sampling Technique)**, que genera ejemplos sintéticos de la clase minoritaria a partir de sus vecinos más cercanos. De este modo, se obtiene un conjunto de entrenamiento más equilibrado, mejorando la capacidad del modelo para aprender patrones representativos y detectar clientes potenciales de manera más efectiva.

```
# Análisis de distribución de clases antes del balanceo
print("Before OverSampling, counts of label '1': {}".format(sum(y_train == 1))) # Clase minoritaria
print("Before OverSampling, counts of label '0': {}".format(sum(y_train == 0))) # Clase mayoritaria

# Aplico SMOTE solo al conjunto de entrenamiento
smote = SMOTE(random_state=42) # random_state para reproducibilidad
X_train, y_train = smote.fit_resample(X_train, y_train) # Solo aplicamos al conjunto de entrenamiento

# Verificación de resultados post-aplicación
print('After OverSampling, the shape of train_X: {}'.format(X_train.shape))
print('After OverSampling, the shape of train_y: {}'.format(y_train.shape))

# Confirmación de balanceo perfecto (50-50)
print("After OverSampling, counts of label '1': {}".format(sum(y_train == 1)))
print("After OverSampling, counts of label '0': {}".format(sum(y_train == 0)))
```

Conclusión del balanceo con SMOTE

Antes del balanceo, el conjunto de entrenamiento presentaba un fuerte desbalance: solo el 11% de las instancias correspondían a la clase positiva ($y = 1$). Esta desproporción podía sesgar al modelo a favor de la clase mayoritaria, reduciendo su capacidad de detectar correctamente clientes que aceptan la oferta bancaria.

Tras aplicar **SMOTE**, se obtuvo un conjunto de entrenamiento perfectamente balanceado, con igual cantidad de instancias para ambas clases (29,229 cada una). Esto permite entrenar modelos más justos y sensibles a la clase minoritaria, mejorando la capacidad predictiva en escenarios desbalanceados como el de este TFM.

5.2 - Elección de métricas: accuracy, precision, recall, F1, AUC

El objetivo del modelo es ayudar a identificar qué clientes tienen mayor probabilidad de contratar un depósito a plazo. En este tipo de problemas, donde la mayoría de los clientes no contrata el producto, medir solo el porcentaje de aciertos (*accuracy*) puede dar una visión incompleta.

Por eso se utilizan métricas adicionales que permiten evaluar mejor el impacto real del modelo:

- **Precision:** ayuda a entender cuántas de las predicciones positivas fueron realmente acertadas (importante para no malgastar recursos en clientes poco interesados).
- **Recall:** indica cuántos clientes realmente interesados fueron correctamente identificados (clave para no perder oportunidades comerciales).
- **F1-Score:** equilibra ambos aspectos.
- **AUC:** permite comparar modelos considerando su capacidad general de diferenciación entre quienes contratarían o no.

Estas métricas ofrecen una visión más útil para la toma de decisiones comerciales y el diseño de campañas efectivas.

```
# Función que imprime métricas de evaluación de un modelo de clasificación
# Acepta:
# y_true -> etiquetas reales
# y_pred -> predicciones del modelo (clase 0 o 1)
# y_proba -> probabilidades predichas (necesarias para AUC y curva ROC)
def saca_metricas(y_true, y_pred, y_proba=None):
    print('♦ Matriz de Confusión:')
    print(confusion_matrix(y_true, y_pred))

    print('\n♦ Classification Report:')
    print(classification_report(y_true, y_pred, target_names=['No', 'Yes']))

    print('♦ Accuracy:', round(accuracy_score(y_true, y_pred), 4))
    print('♦ Precision:', round(precision_score(y_true, y_pred, pos_label=1), 4))
    print('♦ Recall:', round(recall_score(y_true, y_pred, pos_label=1), 4))
    print('♦ F1 Score:', round(f1_score(y_true, y_pred, pos_label=1), 4))

    # Curva ROC
    if y_proba is not None:
        fpr, tpr, _ = roc_curve(y_true, y_proba, pos_label=1)
        roc_auc = auc(fpr, tpr)
        print('♦ AUC:', round(roc_auc, 4))

        fig = go.Figure()
        fig.add_trace(go.Scatter(x=fpr, y=tpr, mode='lines', name='Curva ROC', line=dict(color='blue')))
        fig.add_trace(go.Scatter(x=[0, 1], y=[0, 1], mode='lines', name='Línea base', line=dict(color='red', dash='dash')))
        fig.update_layout(
            title=f'Curva ROC (AUC = {roc_auc:.2f})',
            xaxis_title='Tasa de Falsos Positivos',
            yaxis_title='Tasa de Verdaderos Positivos (Recall)',
            template='plotly_white'
        )
        fig.show()
    else:
        print('No se calculó la curva ROC porque no se pasó `y_proba` (probabilidades).')
```

5.3 - Definición del baseline

Para evaluar el desempeño del modelo y garantizar su capacidad de generalización, se aplica validación cruzada *k-fold* durante el entrenamiento.

Además, se define un **modelo baseline** utilizando un clasificador dummy con la estrategia `most_frequent`, que siempre predice la clase mayoritaria. Esto sirve como referencia mínima para comparar y validar que los modelos entrenados aportan un valor real.

```
] # Definición del modelo baseline que siempre predice la clase mayoritaria
dummy = DummyClassifier(strategy='most_frequent')
# Entrenamiento del baseline con los datos de entrenamiento
dummy.fit(X_train, y_train)
# Predicción de etiquetas para el conjunto de prueba
y_pred_dummy = dummy.predict(X_test)
# Predicción de probabilidades para la clase positiva (usada para métricas como AUC)
y_proba_dummy = dummy.predict_proba(X_test)[1, 1]
# Cálculo y visualización de métricas de evaluación
saca_metricas(y_test, y_pred_dummy, y_proba_dummy)
```

Resultados del modelo baseline

Después de entrenar el clasificador dummy con la estrategia `most_frequent`, que siempre predice la clase mayoritaria ("No"), obtenemos los siguientes resultados:

- La matriz de confusión muestra que el modelo clasifica correctamente todos los casos negativos (7308), pero no detecta ningún caso positivo (928), lo que refleja un sesgo hacia la clase mayoritaria.
- El reporte de clasificación indica una precisión, recall y F1-score nulos para la clase minoritaria ("Yes"), confirmando que el modelo no es capaz de identificar clientes que sí suscriben al depósito.
- La exactitud general (accuracy) es alta (aprox. 88.7%) debido al desbalance del dataset, pero métricas como el AUC (0.5) y el F1 Score evidencian la ineficacia del baseline para predecir correctamente la clase positiva.

Estos resultados resaltan la importancia de desarrollar modelos más sofisticados que superen este baseline y sean capaces de identificar patrones significativos para predecir la suscripción con mayor precisión.

5.4 - Modelo de Regresión logística

En esta sección se entrena y evalúa un **modelo de regresión logística** como una primera aproximación real al problema de clasificación.

```
|: # Definición del modelo con random_state para asegurar reproducibilidad
modelo_lr = LogisticRegression(max_iter=1000, random_state=42)

# Configuración de la validación cruzada estratificada con 5 particiones
# Se mantiene la proporción de clases en cada fold y se añade aleatoriedad con shuffle
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Evaluación del modelo con validación cruzada utilizando la métrica F1 (adecuada para clases desbalanceadas)
scores = cross_val_score(modelo_lr, X_train, y_train, cv=cv, scoring='f1')

print("F1 promedio con validación cruzada:", scores.mean())

# Entreno en todo el train para evaluar en test (final)
modelo_lr.fit(X_train, y_train)
y_pred_lr = modelo_lr.predict(X_test)
y_proba = modelo_lr.predict_proba(X_test)[:, 1]

# Evaluación del modelo sobre el conjunto de test y grafico las métricas correspondientes a este modelo
saca_metricas(y_test, y_pred_lr, y_proba)
```

Evaluación del Modelo de Regresión Logística

A continuación se presentan las observaciones obtenidas a partir del desempeño del modelo de regresión logística.

Observaciones clave

- Excelente capacidad de discriminación global**
 - El valor de **AUC = 0.93** indica que el modelo tiene una muy buena capacidad para distinguir entre clientes que aceptan y los que no aceptan la oferta.
 - La curva ROC se sitúa claramente por encima de la línea base, lo que valida su buen comportamiento como clasificador binario.
- F1 Score elevado en validación cruzada**
 - Se obtuvo un **F1 promedio de 0.88** con validación cruzada estratificada, lo que sugiere un rendimiento consistente del modelo durante el entrenamiento.
- Desempeño por clase: fuerte en recall, débil en precisión para la clase positiva**
 - Para la clase **"Yes" (positiva)**:
- Desempeño por clase: fuerte en recall, débil en precisión para la clase positiva**
 - Para la clase **"Yes" (positiva)**:
 - Precision:** 0.45 → de todas las predicciones positivas, solo el 45% fueron correctas.
 - Recall:** 0.85 → el modelo logra capturar correctamente el 85% de los casos verdaderamente positivos.
 - F1 Score:** 0.59 → balancea ambos extremos, pero se ve penalizado por la baja precisión.
- Alta cantidad de verdaderos positivos, pero también muchos falsos positivos**
 - La **matriz de confusión** indica:
 - Verdaderos positivos: 793
 - Falsos positivos: 970
 - Esto indica una alta sensibilidad, pero con muchos clientes clasificados erróneamente como potenciales positivos.

Conclusiones

- El modelo muestra un **muy buen rendimiento general**, especialmente en la capacidad de identificar clientes que aceptan la oferta (**recall** alto).
- Sin embargo, genera una cantidad considerable de **falsos positivos**, lo que reduce la **precisión** y puede traducirse en campañas ineficientes o molestas para los clientes.

El análisis sugiere que la regresión logística es un modelo competitivo y útil como punto de partida, aunque probablemente mejorable con enfoques no lineales.

5.5 - Modelo de Árboles de decisión

Luego y en esta sección se entrena y evalúa un modelo de **Árbol de decisión**.

```
|: # Definición del modelo con random_state para asegurar reproducibilidad
modelo_dt = DecisionTreeClassifier(random_state=42)

# Configuración de la validación cruzada estratificada con 5 particiones
# Se mantiene la proporción de clases en cada fold y se añade aleatoriedad con shuffle
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Evaluación del modelo con validación cruzada utilizando la métrica F1 (adecuada para clases desbalanceadas)
scores_dt = cross_val_score(modelo_dt, X_train, y_train, cv=cv, scoring='f1')

print("F1 promedio con validación cruzada (Árbol de Decisión):", round(scores_dt.mean(), 4))

# Entrenamiento en todo el conjunto de entrenamiento
```



```
# Definición del modelo con random_state para asegurar reproducibilidad
modelo_dt = DecisionTreeClassifier(random_state=42)

# Configuración de la validación cruzada estratificada con 5 particiones
# Se mantiene la proporción de clases en cada fold y se añade aleatoriedad con shuffle
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Evaluación del modelo con validación cruzada utilizando la métrica F1 (adecuada para clases desbalanceadas)
scores_dt = cross_val_score(modelo_dt, X_train, y_train, cv=cv, scoring='f1')

print("F1 promedio con validación cruzada (Árbol de Decisión):", round(scores_dt.mean(), 4))

# Entrenamiento en todo el conjunto de entrenamiento
modelo_dt.fit(X_train, y_train)

# Predicción sobre conjunto de prueba
y_pred_dt = modelo_dt.predict(X_test)
y_proba_dt = modelo_dt.predict_proba(X_test)[:, 1]

# Evaluación del modelo sobre el conjunto de test y grafico las métricas correspondientes a este modelo
saca_metricas(y_test, y_pred_dt, y_proba_dt)
```

5.4.2 Evaluación del Modelo de Árbol de Decisión

A continuación se presentan las observaciones obtenidas a partir del desempeño del modelo de árbol de decisión.

Observaciones clave

1. Rendimiento aceptable en discriminación global

- El valor de **AUC = 0.75** indica que el modelo tiene una capacidad **moderada** para distinguir entre clientes que aceptan y no aceptan la oferta.
- La curva ROC se sitúa por encima de la línea base, aunque no tan alejada como en otros modelos más robustos (como la regresión logística o modelos de ensamblado).

2. F1 promedio alto en validación cruzada

- Se obtuvo un **F1 promedio de 0.9283**, lo que sugiere un **buen ajuste durante el entrenamiento** en los distintos folds.
- Sin embargo, al evaluar en el conjunto de prueba, el desempeño baja, lo que puede estar indicando **sobreajuste** (overfitting), fenómeno común en árboles sin poda o sin regularización.

3. Desempeño por clase: balance discreto, con mejores resultados en la clase mayoritaria

3. Desempeño por clase: balance discreto, con mejores resultados en la clase mayoritaria

- Para la clase **"Yes" (positiva)**:
 - Precision:** 0.51 → algo mejor que la regresión logística.
 - Recall:** 0.58 → captura solo el 58% de los positivos reales.
 - F1 Score:** 0.54 → indica un rendimiento moderado, penalizado por el bajo recall.
- Para la clase **"No" (negativa)**:
 - Alto desempeño con un F1 de 0.94, reflejo del desbalance de clases y de la facilidad para identificar los negativos.

4. Mejor precisión que la regresión logística, pero menor capacidad de cobertura de positivos

- El árbol de decisión logra **menos falsos positivos** (mejor precisión), pero a costa de **no identificar correctamente tantos casos positivos reales**.
- Esto puede observarse en la **matriz de confusión**:
 - Verdaderos positivos: 539
 - Falsos positivos: 520

Conclusiones

- El modelo de árbol de decisión ofrece un **buen rendimiento general**, con una **precisión ligeramente superior** en la clase positiva respecto al modelo de regresión logística.
- No obstante, su menor **recall (0.58)** implica que **deja pasar una parte considerable de clientes que sí aceptarían la oferta**, lo que podría representar oportunidades de negocio perdidas.
- El valor de **AUC = 0.75** lo posiciona por debajo de otros modelos más sofisticados, lo que indica un margen importante de mejora.
- Además, el posible **sobreajuste** observado entre entrenamiento y test sugiere que sería recomendable aplicar técnicas de **poda, ajuste de hiperparámetros o incluso optar por modelos de ensamblado** como Random Forest o XGBoost para mejorar su capacidad generalizadora.

En resumen, el árbol de decisión resulta útil como modelo interpretable y de bajo costo computacional, pero presenta limitaciones que podrían abordarse con enfoques más complejos o regularizados.

5.6 - Modelo de Random Forest

Continúa ahora por ajustar y entrenar un **modelo de Random Forest**

```
# Definición del modelo con random_state para asegurar reproducibilidad
modelo_rf = RandomForestClassifier(random_state=42)
```

5.6 - Modelo de Random Forest

Continúo ahora por ajustar y entrenar un **modelo de Random Forest**

```
# Definición del modelo con random_state para asegurar reproducibilidad
modelo_rf = RandomForestClassifier(random_state=42)

# Configuración de la validación cruzada estratificada con 5 particiones
# Se mantiene la proporción de clases en cada fold y se añade aleatoriedad con shuffle
cv = StratifiedFold(n_splits=5, shuffle=True, random_state=42)

# Evaluación del modelo con validación cruzada utilizando la métrica F1 (adecuada para clases desbalanceadas)
scores_rf = cross_val_score(modelo_rf, X_train, y_train, cv=cv, scoring='f1')

print("F1 promedio con validación cruzada (Random Forest):", round(scores_rf.mean(), 4))

# Entreno el modelo en todo el conjunto de entrenamiento
modelo_rf.fit(X_train, y_train)

# Predicción sobre el conjunto de test
y_pred_rf = modelo_rf.predict(X_test)
y_proba_rf = modelo_rf.predict_proba(X_test)[: , 1]

# Evaluación del modelo sobre el conjunto de test y grafico las métricas correspondientes a este modelo
saca_metricas(y_test, y_pred_rf, y_proba_rf)
```

Evaluación del Modelo Random Forest

A continuación se presentan las observaciones obtenidas a partir del desempeño del modelo Random Forest aplicado al conjunto de test, complementado con validación cruzada.

Observaciones clave

Excelente capacidad de discriminación global

- **AUC = 0.94**, lo cual indica una capacidad sobresaliente para distinguir entre clientes que aceptan y los que no aceptan la oferta.
- La **curva ROC** se sitúa muy por encima de la línea base, reforzando su eficacia como clasificador binario.

Observaciones clave

Excelente capacidad de discriminación global

- **AUC = 0.94**, lo cual indica una capacidad sobresaliente para distinguir entre clientes que aceptan y los que no aceptan la oferta.
- La **curva ROC** se sitúa muy por encima de la línea base, reforzando su eficacia como clasificador binario.

F1 Score alto en validación cruzada

- Se obtuvo un **F1 promedio de 0.9566** con validación cruzada estratificada, lo que sugiere un rendimiento muy sólido y consistente del modelo en los diferentes folds del entrenamiento.

Desempeño por clase: fuerte en clase negativa, limitado en clase positiva

- Para la clase **"Yes" (positiva)**:
 - **Precision: 0.61** → De todas las predicciones positivas, solo el 61% fueron correctas.
 - **Recall: 0.56** → El modelo logra capturar el 56% de los verdaderos positivos.
 - **F1 Score: 0.58** → Moderado, penalizado por el balance entre precisión y recall.
- Para la clase **"No" (negativa)**:
 - El desempeño es notable, con una precisión y recall de aproximadamente **0.94-0.95**.

Menor proporción de falsos positivos respecto a Regresión Logística

- Según la matriz de confusión:
 - **Verdaderos positivos: 519**
 - **Falsos positivos: 331**
- Esto representa una mejora considerable en la precisión del modelo respecto al baseline logístico, con menor cantidad de clientes mal clasificados como positivos.

Conclusiones

- El modelo **Random Forest mejora claramente la precisión y el AUC** respecto a la regresión logística, lo cual es esperable dada su naturaleza no lineal y capacidad para modelar relaciones complejas.
- Sin embargo, **el recall de la clase positiva se reduce**, lo que implica que el modelo podría estar dejando pasar algunos clientes que aceptarían la oferta.
- Puede ser un buen modelo si se desea **minimizar falsos positivos** y tener un clasificador más conservador.
- Como estrategia complementaria, podría explorarse un ajuste del **umbral de decisión**, o un **modelo de ensamblado** que combine Random Forest con otro algoritmo con mayor sensibilidad.

5.7 - Modelo de XGBoost

En esta parte probaré el uso de un **modelo XGBoost**.

```
# Definición del modelo con random_state para asegurar reproducibilidad
modelo_xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)

# Configuración de la validación cruzada estratificada con 5 particiones
# Se mantiene la proporción de clases en cada fold y se añade aleatoriedad con shuffle
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Evaluación del modelo con validación cruzada utilizando la métrica F1 (adecuada para clases desbalanceadas)
scores_xgb = cross_val_score(modelo_xgb, X_train, y_train, cv=cv, scoring='f1')

print("F1 promedio con validación cruzada (XGBoost):", round(scores_xgb.mean(), 4))

# Entreno el modelo con todos los datos de entrenamiento
modelo_xgb.fit(X_train, y_train)

# Predicciones sobre conjunto de test
y_pred_xgb = modelo_xgb.predict(X_test)
y_proba_xgb = modelo_xgb.predict_proba(X_test)[:, 1]

# Evaluación del modelo sobre el conjunto de test y grafico las métricas correspondientes a este modelo
saca_metricas(y_test, y_pred_xgb, y_proba_xgb)
```

5.X - Evaluación del Modelo XGBoost

A continuación se detallan las observaciones clave obtenidas a partir del desempeño del modelo XGBoost aplicado al conjunto de test.

Observaciones clave

Excelente capacidad de discriminación global

- El valor **AUC = 0.95** indica que el modelo posee una **capacidad sobresaliente para distinguir** entre clientes que aceptan y los que no aceptan la oferta.
- La curva ROC se mantiene muy por encima de la línea base, lo que **valida la eficacia del modelo como clasificador binario**.

F1 Score elevado en validación cruzada

- Se obtuvo un **F1 promedio de 0.951** con validación cruzada estratificada, lo que sugiere un **rendimiento consistente y robusto** del modelo durante el entrenamiento.

F1 Score elevado en validación cruzada

- Se obtuvo un **F1 promedio de 0.951** con validación cruzada estratificada, lo que sugiere un **rendimiento consistente y robusto** del modelo durante el entrenamiento.

Desempeño mixto por clase: fuerte en la clase negativa, débil en la clase positiva

- Para la clase **"Yes"** (positiva):
 - **Precision: 0.63** → de todas las predicciones positivas, solo el 63% fueron correctas.
 - **Recall: 0.59** → el modelo identifica correctamente el 59% de los clientes que efectivamente aceptaron la oferta.
 - **F1 Score: 0.61** → se logra un equilibrio razonable entre precisión y recall, aunque aún lejos de ser óptimo.
- Para la clase **"No"**:
 - Alto rendimiento, con una F1-score de **0.95**, reflejando la facilidad del modelo para detectar correctamente a quienes no aceptan la oferta.

Reducción en falsos positivos respecto a la regresión logística

- Matriz de confusión:
 - **Verdaderos positivos (VP): 546**
 - **Falsos positivos (FP): 325**
- En comparación con la regresión logística (970 FP), XGBoost **reduce considerablemente los falsos positivos**, lo que implica **menos interferencias comerciales innecesarias**.

Buen rendimiento global en métricas generales

- **Accuracy: 0.9142** → más del 91% de las predicciones fueron correctas.
- **Precision global: 0.6269**
- **Recall global: 0.5884**
- **F1 global: 0.607**

Conclusiones

- El modelo **XGBoost supera a la regresión logística** en métricas clave como **AUC, F1 en validación cruzada** y especialmente en la **reducción de falsos positivos**.
- Aunque aún queda margen de mejora en la **precisión y recall para la clase positiva**, se observa un **mayor equilibrio y eficiencia general**.
- Su buen rendimiento general, junto con su bajo nivel de sobreajuste, lo convierten en una **excelente alternativa para la predicción de aceptación de campañas**.

5.8 - Modelo de K-Nearest Neighbors

En esta sub-sección testeamos una solución basada en **K-Nearest Neighbors**.

```
# Definir modelo
modelo_knn = KNeighborsClassifier()

# Configuración de la validación cruzada estratificada con 5 particiones
# Se mantiene la proporción de clases en cada fold y se añade aleatoriedad con shuffle
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Evaluación del modelo con validación cruzada utilizando la métrica F1 (adecuada para clases desbalanceadas)
scores_knn = cross_val_score(modelo_knn, X_train, y_train, cv=cv, scoring='f1')

print("F1 promedio con validación cruzada (KNN):", round(scores_knn.mean(), 4))

# Entrenamiento final
modelo_knn.fit(X_train, y_train)

# Predicciones
y_pred_knn = modelo_knn.predict(X_test)
y_proba_knn = modelo_knn.predict_proba(X_test)[: , 1]

# Evaluación del modelo sobre el conjunto de test y grafico las métricas correspondientes a este modelo
saca_metricas(y_test, y_pred_knn, y_proba_knn)
```

Evaluación del Modelo K-Nearest Neighbors (KNN)

A continuación se presentan las observaciones obtenidas a partir del desempeño del modelo de K-Nearest Neighbors.

Observaciones clave

- **Capacidad de discriminación razonable**
 - El valor de **AUC = 0.74** sugiere que el modelo tiene una capacidad moderada para distinguir entre clientes que aceptan y los que no aceptan la oferta.
 - La **curva ROC** se sitúa por encima de la línea base, aunque no de forma sobresaliente, lo que indica un desempeño razonable como clasificador binario, pero lejos de óptimo.
- **F1 Score alto en validación cruzada**
- **F1 Score alto en validación cruzada**
 - Se obtuvo un **F1 promedio de 0.8872** usando validación cruzada estratificada, lo cual sugiere un rendimiento robusto durante el entrenamiento, especialmente considerando el desbalance de clases.
- **Desempeño por clase: buen recall, baja precisión para la clase positiva**
 - Para la clase **"Yes"** (cliente que acepta la oferta):
 - **Precisión: 0.30** → Solo el 30% de las predicciones positivas fueron correctas.
 - **Recall: 0.55** → El modelo logra capturar correctamente el 55% de los casos verdaderamente positivos.
 - **F1 Score: 0.39** → Se ve penalizado por la baja precisión, aunque se beneficia del recall.
- **Matriz de confusión: muchos falsos positivos**
 - Verdaderos negativos: 6090
 - Falsos positivos: 1218
 - Falsos negativos: 413
 - Verdaderos positivos: 515
 - Esto muestra un número significativo de **falsos positivos**, lo cual podría generar costos o esfuerzos innecesarios en campañas dirigidas.

Conclusiones

- El modelo KNN muestra un desempeño aceptable en cuanto a recall, logrando identificar más de la mitad de los clientes interesados.
- No obstante, su **precisión es baja**, lo que indica que muchas predicciones positivas resultan ser incorrectas.
- El valor de AUC cercano a 0.74 confirma que se trata de un modelo **mejor que el azar**, aunque **menos eficaz que otros modelos más sofisticados**.
- En contextos donde es importante reducir falsos positivos (por ejemplo, para minimizar costos de campañas o evitar molestar clientes), **KNN podría no ser la mejor opción sin algún tipo de ajuste adicional** (por ejemplo, optimización de hiperparámetros o técnicas de balanceo).

5.9 - Modelo de Naive Bayes

Ahora, procedo a probar el desempeño con un **modelo Naive Bayes**.

```
# Modelo Naive Bayes
modelo_nb = GaussianNB()

# Configuración de la validación cruzada estratificada con 5 particiones
# Se mantiene la proporción de clases en cada fold y se añade aleatoriedad con shuffle
```

5.9 - Modelo de Naive Bayes

Ahora, procedo a probar el desempeño con un **modelo Naive Bayes**.

```
# Modelo Naive Bayes
modelo_nb = GaussianNB()

# Configuración de la validación cruzada estratificada con 5 particiones
# Se mantiene la proporción de clases en cada fold y se añade aleatoriedad con shuffle
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Evaluación del modelo con validación cruzada utilizando la métrica F1 (adecuada para clases desbalanceadas)
scores_nb = cross_val_score(modelo_nb, X_train, y_train, cv=cv, scoring='f1')

print("F1 promedio con validación cruzada (Naive Bayes):", round(scores_nb.mean(), 4))

# Entreno el modelo final
modelo_nb.fit(X_train, y_train)

# Predicción y evaluación
y_pred_nb = modelo_nb.predict(X_test)
y_proba_nb = modelo_nb.predict_proba(X_test)[:, 1]

# Evaluación del modelo sobre el conjunto de test y grafico las métricas correspondientes a este modelo
saca_metricas(y_test, y_pred_nb, y_proba_nb)
```

Evaluación del Modelo Naive Bayes

A continuación se presentan las observaciones obtenidas a partir del desempeño del modelo Naive Bayes aplicado al conjunto de datos de marketing bancario.

Observaciones clave

Buena capacidad general de discriminación

- El valor de AUC = **0.78** indica que el modelo posee una **buena capacidad para distinguir** entre clientes que aceptan y los que no aceptan la oferta.
- La **curva ROC** se encuentra bien por encima de la línea base aleatoria, mostrando un rendimiento clasificatorio aceptable, aunque inferior al de otros modelos más complejos como regresión logística.

F1 Score razonable con validación cruzada

Se obtuvo un **F1 promedio de 0.72** utilizando validación cruzada estratificada, lo cual sugiere una **consistencia adecuada durante el entrenamiento** a pesar de

F1 Score razonable con validación cruzada

- Se obtuvo un **F1 promedio de 0.72** utilizando validación cruzada estratificada, lo cual sugiere una **consistencia adecuada durante el entrenamiento** a pesar de tratarse de un modelo simple y lineal.

Desempeño por clase: excelente recall, baja precisión para la clase positiva

- Para la clase **"Yes"** (respuesta positiva del cliente):
 - Precision: 0.26** → el modelo clasifica como positivos muchos casos incorrectos, es decir, **alta tasa de falsos positivos**.
 - Recall: 0.71** → logra capturar correctamente una gran proporción de los clientes que realmente aceptaron la oferta.
 - F1 Score: 0.38** → penalizado por la baja precisión, aunque mejora por el buen recall.

Matriz de confusión: fuerte sensibilidad, pero con costo de precisión

- Verdaderos positivos: **656**
- Falsos positivos: **1831**
- Esto refuerza que el modelo **prioriza la detección de positivos** (alta sensibilidad), aunque incurre en muchos errores al clasificar clientes no interesados como potenciales positivos.

Conclusiones

- El modelo **Naive Bayes** presenta un rendimiento **decente y balanceado**, destacándose por su **simplicidad computacional** y su **alta capacidad de recuperación de positivos (recall)**.
- No obstante, la **muy baja precisión lo hace menos recomendable** en contextos donde los costos de contactar a un cliente no interesado sean altos.
- Este modelo puede ser útil como **baseline** o como parte de un **ensamble inicial**, pero es probable que su rendimiento se vea superado por enfoques más sofisticados o no lineales.

5.10 - Modelo de Deep Learning

En esta sección evalúo el rendimiento de un modelo de **Deep Learning** como alternativa al enfoque clásico basado en modelos estadísticos y de aprendizaje automático / machine learning supervisado. Si bien las redes neuronales suelen requerir mayor cantidad de datos y capacidad computacional, creo que pueden resultar útiles para capturar relaciones no lineales complejas entre las variables del dataset.

Dado que el problema abordado consiste en una tarea de clasificación binaria, implemento una red neuronal multicapa (**Multilayer Perceptron**, MLP) con una arquitectura simple y entrenada sobre el mismo conjunto de datos procesado previamente. El objetivo es analizar si este enfoque logra mejorar el rendimiento predictivo observado en

5.10 - Modelo de Deep Learning

En esta sección evalué el rendimiento de un modelo de **Deep Learning** como alternativa al enfoque clásico basado en modelos estadísticos y de aprendizaje automático / machine learning supervisado. Si bien las redes neuronales suelen requerir mayor cantidad de datos y capacidad computacional, creo que pueden resultar útiles para capturar relaciones no lineales complejas entre las variables del dataset.

Dado que el problema abordado consiste en una tarea de clasificación binaria, implemento una red neuronal multicapa (**Multilayer Perceptron**, MLP) con una arquitectura simple y entrenada sobre el mismo conjunto de datos procesado previamente. El objetivo es analizar si este enfoque logra mejorar el rendimiento predictivo observado en los modelos previos, como la regresión logística o los árboles de decisión, y si justifica su complejidad adicional en este contexto.

Se emplean técnicas de regularización y validación cruzada para evitar el sobreajuste, y se comparan métricas clave como el *F1-score*, la *AUC-ROC*, la *matriz de confusión* y los valores de *precision* y *recall*.

Primero comienzo por observar la cantidad de variables del dataset.

```
# Imprimo la forma/shape del dataset de entrenamiento (La cantidad de filas y especialmente la cantidad de columnas).
print(X_train.shape)
```

Confirmando que la cantidad de variables presentes en el dataset de entrenamiento es igual a 108. Esto es relevante dado que el modelo de deep learning que emplearé necesita especificar las dimensiones del input (es decir, la cantidad de variables a procesar).

```
# Se define un modelo secuencial (modelo de red neuronal de tipo "capa por capa")
model = Sequential()
# Uso regularización L2 para evitar reducir el riesgo de sobreajuste / overfitting (penaliza los pesos grandes)
kernel_regularizer_l2 = regularizers.L2(5e-4)
# Ahora vendrá la primera capa oculta con 64 neuronas, activación tanh y regularización L2
# Se especifica el input_dim igual al número de features de entrada del dataset
model.add(Dense(64, input_dim=X_train.shape[1], activation='tanh', kernel_regularizer=kernel_regularizer_l2))
# Luego, aquí viene un dropout para reducir sobreajuste: apaga aleatoriamente el 30% de las neuronas
model.add(layers.Dropout(0.3))
# Segunda capa oculta, también con 64 neuronas y tanh. Se repite la regularización
model.add(Dense(64, input_dim=64, activation='tanh', kernel_regularizer=kernel_regularizer_l2))
model.add(layers.Dropout(0.3))
# Finalizo aquí con una capa de salida con 1 neurona y activación sigmoide para clasificación binaria
model.add(Dense(1, activation='sigmoid'))
```

A continuación procedo a compilar el modelo de red neuronal y especifico parámetros específicos.

```
model.compile(
    # Función de pérdida adecuada para clasificación binaria
    loss='binary_crossentropy'
```

```
model.compile(
    # Función de pérdida adecuada para clasificación binaria
    loss='binary_crossentropy',
    # Optimizador Adam: combina momentum y adaptatividad en el Learning rate
    optimizer='adam',
    # Métricas a monitorizar durante el entrenamiento y la evaluación
    metrics=['accuracy', Precision(name='precision'), Recall(name='recall'), AUC(name='auc')]
)
```

```
# Definición de EarlyStopping como técnica preventiva contra el sobreajuste:
# detiene el entrenamiento si la métrica monitoreada no mejora tras un número determinado de épocas
es_callback = keras.callbacks.EarlyStopping(
    monitor='val_loss', # Se monitorea la pérdida sobre el conjunto de validación
    patience=5, # El entrenamiento se detiene si no mejora después de 5 épocas consecutivas
    verbose=1) # Muestra un mensaje cuando se detiene el entrenamiento
# Convierto los datos de entrenamiento a arrays NumPy, requeridos por Keras
x_train_keras = np.array(X_train)
y_train_keras = np.array(y_train)
# Ajusto las dimensiones para el vector objetivo (y) a formato (n_samples, 1)
y_train_keras = y_train_keras.reshape(y_train_keras.shape[0], 1)
```

Ahora paso a entrenar el modelo de Deep Learning luego de su compilación y definición de ajustes.

```
# Hago el entrenamiento del modelo con los datos de entrenamiento
model.fit(X_train, # Conjunto de entrenamiento con las features
        y_train, # Variable objetivo binaria
        epochs=200, # Número máximo de épocas de entrenamiento
        batch_size=32, # Tamaño del batch: se actualizarán los pesos cada 32 muestras
        validation_split=0.2, # Proporción del conjunto de entrenamiento reservado para validación (20%)
        verbose=1, # Muestra el progreso del entrenamiento
        callbacks=[es_callback]) # Lista de callbacks utilizado (EarlyStopping para evitar sobreajuste)
```

```
# Conversión del conjunto de test a arrays de NumPy para compatibilidad con Keras
X_test_keras = np.array(X_test)
```

```
# Me aseguro que las etiquetas de test tengan la forma adecuada (matriz columna)
y_test_keras = np.array(y_test).reshape(-1, 1)
```

```
# Evaluación del modelo entrenado sobre el conjunto de test
# Devuelve una lista con los valores de las métricas definidas en model.compile()
scores = model.evaluate(X_test_keras, y_test_keras)
```

```
# Imprimo la métrica de accuracy obtenida en la evaluación del modelo sobre el conjunto de test
# 'model.metrics_names[1]' corresponde al nombre de la segunda métrica definida en model.compile(), que es 'accuracy'
print("\n%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))
```

```
scores = model.evaluate(X_test_keras, y_test_keras, verbose=0)
print(f'◆ Loss: {scores[0]:.4f}')
print(f'◆ Accuracy: {scores[1]:.4f}')
print(f'◆ Precision: {scores[2]:.4f}')
print(f'◆ Recall: {scores[3]:.4f}')
print(f'◆ AUC: {scores[4]:.4f}')
```

```
# Realizo las predicciones del modelo sobre el conjunto de test
y_pred_dl = model.predict(X_test_keras)
# Binarizo las probabilidades obtenidas: si la probabilidad es mayor o igual a 0.5 se clasifica como 1, si no como 0
y_pred_dl = (y_pred_dl >= 0.5).astype(int)

# Calculo el F1 Score comparando las etiquetas reales con las predichas
# El F1 Score es útil para evaluar el balance entre precisión y recall, especialmente en datasets desbalanceados
f1 = f1_score(y_test_keras, y_pred_dl)
# Muestra el valor del F1 Score con 4 decimales
print(f'◆ F1 Score: {f1:.4f}')
```

Evaluación del Modelo de Deep Learning

A continuación se presentan las observaciones clave sobre el desempeño del modelo de Deep Learning (Red Neuronal) aplicado al conjunto de prueba.

Observaciones clave

Alta capacidad de discriminación global

- El valor **AUC = 0.9315** revela una **excelente capacidad del modelo para distinguir entre clientes que aceptan y no aceptan la oferta**.
- La alta AUC, combinada con un buen recall, indica que la red logra **capturar correctamente patrones relevantes**, incluso en presencia de desequilibrio de clases.

Buen rendimiento en la clase positiva (acepta la oferta)

- Para la clase **"Yes"** (positiva):
 - Precision: 0.5068** → de cada 100 predicciones positivas, solo ~51 fueron correctas.
 - Recall: 0.7597** → el modelo detecta correctamente un 76% de los clientes que realmente aceptaron la campaña.

Buen rendimiento en la clase positiva (acepta la oferta)

- Para la clase **"Yes"** (positiva):
 - Precision: 0.5068** → de cada 100 predicciones positivas, solo ~51 fueron correctas.
 - Recall: 0.7597** → el modelo detecta correctamente un 76% de los clientes que realmente aceptaron la campaña.
 - F1 Score: 0.6080** → representa un **equilibrio razonable entre precisión y recall**, aunque la precisión aún es baja para uso operativo directo.

Reducción del error global, pero con falsos positivos a considerar

- El valor de **Loss: 0.2869** sugiere una **buena minimización del error** en términos de función de pérdida binaria.
- Sin embargo, la **precisión moderada** indica una tendencia a generar falsos positivos, lo cual podría traducirse en **contactos comerciales innecesarios** si no se ajusta el umbral de decisión o no se combina con reglas adicionales.

Buen rendimiento global en métricas generales ¶

- Accuracy: 0.8896** → cerca del 89% de las predicciones fueron correctas.
- Precision global: 0.5068**
- Recall global: 0.7597**
- F1 global: 0.6080**

Conclusiones

- El modelo de Deep Learning muestra un **rendimiento competitivo en términos de recall y AUC**, siendo capaz de **detectar correctamente una alta proporción de clientes interesados**.
- Aunque la **precisión es menor que en otros modelos como XGBoost**, el **modelo es útil para escenarios donde el recall sea prioritario**, por ejemplo, en **etapas tempranas de una campaña** donde se busca maximizar el alcance.
- Podría beneficiarse de **ajustes en el umbral de clasificación**, **técnicas de balanceo** adicionales o incluso de una **estrategia de ensamblado** con modelos más precisos para mejorar la eficiencia final.

5.11 - Comparación de modelos

Se realizaron comparaciones previas entre modelos, pero en esta sección profundizo la comparación para obtener una conclusión definitiva entre los rendimientos de cada modelo frente a la tarea de clasificación que necesitamos para este proyecto.

5.11 - Comparación de modelos

Se realizaron comparaciones previas entre modelos, pero en esta sección profundizo la comparación para obtener una conclusión definitiva entre los rendimientos de cada modelo frente a la tarea de clasificación que necesitamos para este proyecto.

```
[ ]: # Diccionario de clasificadores con parámetros para tratar el desbalance de clases donde sea posible
clf_dict = {
    'Logistic Regression': LogisticRegression(class_weight='balanced', max_iter=3000, random_state=8),
    'Decision Tree': DecisionTreeClassifier(class_weight='balanced', random_state=8),
    'Random Forest': RandomForestClassifier(criterion='entropy', class_weight='balanced', random_state=8),
    'XGBoost': XGBClassifier(scale_pos_weight=7.85, use_label_encoder=False, eval_metric='logloss', seed=8),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5), # KNN no soporta class_weight
    'Naive Bayes': GaussianNB(priors=[0.113, 0.887]) # Ajuste manual de probabilidades según proporción real
}

[ ]: for nombre_modelo, modelo in clf_dict.items():
    print(f"\n{' '*40}\n Modelo: {nombre_modelo}\n{' '*40}")

    # Entrenamiento
    modelo.fit(X_train, y_train)

    # Predicción de clases
    y_pred = modelo.predict(X_test)

    # Predicción de probabilidades
    if hasattr(modelo, "predict_proba"):
        y_proba = modelo.predict_proba(X_test)[: , 1]
    else:
        y_proba = None

    # Evaluación
    saca_metricas(y_test, y_pred, y_proba)
```

5.3 - Tunes de hiperparámetros para el modelo seleccionado

5.3 - Tunes de hiperparámetros para el modelo seleccionado

Una vez identificado XGBoost como el modelo con mejor desempeño general en la etapa comparativa, se procedió a optimizar sus hiperparámetros con el objetivo de mejorar su capacidad predictiva y adaptar el modelo de manera más precisa a las características del conjunto de datos.

La optimización de hiperparámetros se llevó a cabo mediante **GridSearchCV**, una técnica de búsqueda exhaustiva que permite evaluar todas las combinaciones posibles dentro de un espacio definido de hiperparámetros. Este enfoque permite encontrar la configuración óptima que maximice el rendimiento del modelo según una métrica de evaluación determinada (en este caso, `roc_auc`).

Los hiperparámetros evaluados incluyen:

- `n_estimators`: número de árboles a construir.
- `max_depth`: profundidad máxima de cada árbol.
- `learning_rate`: tasa de aprendizaje utilizada para la actualización de pesos.
- `subsample`: proporción de muestras utilizadas en cada iteración.
- `colsample_bytree`: proporción de columnas utilizadas por cada árbol.

El proceso se llevó a cabo utilizando validación cruzada de 5 pliegues, lo que garantiza una evaluación robusta del modelo y ayuda a mitigar problemas de sobreajuste. A continuación, se presentan los resultados obtenidos y la mejor combinación de hiperparámetros seleccionada.

```
# Se instancia el modelo base de XGBoost con una métrica de evaluación adecuada
# y una semilla fija para asegurar reproducibilidad.
xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)

# Se define la grilla de hiperparámetros a evaluar con GridSearchCV.
param_grid = {
    'n_estimators': [100, 200], # Número total de árboles a entrenar
    'max_depth': [3, 5], # Profundidad máxima de cada árbol, controla la complejidad del modelo.
    'learning_rate': [0.05, 0.1], # Tasa de aprendizaje que reduce la contribución de cada árbol.
    'subsample': [0.85, 1.0], # Proporción de muestras utilizadas en cada iteración (para evitar sobreajuste).
    'colsample_bytree': [0.85, 1.0], # Proporción de columnas (features) muestreadas por árbol.
    'scale_pos_weight': [1, 5, 7.85] # Peso relativo de la clase positiva, útil para desbalance de clases.
}

# Se configura GridSearchCV para realizar búsqueda exhaustiva de combinaciones de hiperparámetros.
grid_search = GridSearchCV(
    estimator=xgb, # Modelo base (XGBoost) a tunear.
    param_grid=param_grid, # Grilla de hiperparámetros definida previamente.
    scoring='f1', # Métrica objetivo para seleccionar el mejor modelo (F1 Score).
```



```
# Se configura GridSearchCV para realizar búsqueda exhaustiva de combinaciones de hiperparámetros.
grid_search = GridSearchCV(
    estimator=xgb, # Modelo base (XGBoost) a tunear.
    param_grid=param_grid, # Grilla de hiperparámetros definida previamente.
    scoring='f1', # Métrica objetivo para seleccionar el mejor modelo (F1 Score).
    cv=3, # Número de particiones para validación cruzada (3-fold CV).
    n_jobs=-1, # Usa todos los núcleos disponibles para acelerar la búsqueda.
    verbose=2 # Nivel de detalle en la salida durante el entrenamiento.
)

# Se ajusta el modelo a los datos de entrenamiento utilizando validación cruzada.
grid_search.fit(X_train, y_train)

# Imprimo los mejores hiperparámetros encontrados para el modelo XGBoost.
print("Mejores hiperparámetros:")
print(grid_search.best_params_)
```

Análisis de los Hiperparámetros Seleccionados

Los hiperparámetros seleccionados durante la búsqueda con `GridSearchCV` han contribuido de manera significativa al rendimiento del modelo:

- **max_depth moderado:** Un valor adecuado de profundidad controla el sobreajuste permitiendo al modelo capturar relaciones complejas sin memorizar ruido. Esto es clave dado que los datos poseen múltiples variables categóricas y relaciones no lineales.
- **learning_rate bajo:** Una tasa de aprendizaje más baja (por ejemplo, 0.05 o 0.1) hace que el modelo aprenda de forma más gradual y estable, lo cual tiende a mejorar el rendimiento general en validación cruzada.
- **n_estimators elevado:** Usar un número mayor de árboles junto a un `learning_rate` bajo permite que el modelo construya una solución más robusta y generalizable, incrementando el AUC y la capacidad de generalización.
- **subsample y colsample_bytree < 1:** Estas tasas de muestreo controlan el sobreajuste al limitar el número de observaciones y características usadas por cada árbol. Son especialmente útiles en datasets como este, donde hay riesgo de que el modelo se adapte demasiado a los patrones particulares del conjunto de entrenamiento.
- **scale_pos_weight ajustado:** En contextos de clases desbalanceadas, este parámetro permite al modelo penalizar más los errores de clasificación en la clase minoritaria ("Yes"), lo cual explica el fuerte aumento en el recall sin sacrificar demasiado la precisión.

En conjunto, estos valores permiten que `XGBoost` actúe como un **modelo regulado, eficiente y con buena capacidad de generalización**, maximizando su utilidad práctica en campañas reales.

Estos valores obtenidos fueron luego utilizados para reentrenar el modelo XGBoost final con el objetivo de maximizar su desempeño predictivo sobre datos no vistos.

```
# Se extrae el mejor modelo ajustado (con los mejores hiperparámetros) del objeto GridSearchCV
mejor_modelo_xgb = grid_search.best_estimator_

# Se generan las predicciones de clase sobre el conjunto de test
y_pred = mejor_modelo_xgb.predict(X_test)

# Se generan las probabilidades de predicción para la clase positiva (Label = 1)
y_proba = mejor_modelo_xgb.predict_proba(X_test)[:, 1]

# Se evalúa el desempeño del modelo utilizando las métricas definidas en la función personalizada 'saca_metricas'
saca_metricas(y_test, y_pred, y_proba)
```

Evaluación del Modelo XGBoost tras el Tuneo de Hiperparámetros

Los resultados del modelo optimizado mediante `GridSearchCV` muestran una mejora significativa en la capacidad del clasificador para identificar correctamente los casos positivos (clientes que aceptan la oferta de marketing):

- ◆ **AUC = 0.95:** El área bajo la curva ROC indica un excelente desempeño general del modelo, muy por encima del azar (0.5). El modelo discrimina bien entre las clases.
- ◆ **Recall = 0.81** para la clase "Yes": Este valor refleja una gran capacidad para captar los casos positivos, lo cual es crucial en este contexto si se busca maximizar la tasa de respuesta de la campaña.
- ◆ **Precision = 0.54:** Si bien el recall es alto, la precisión es más baja, lo que indica una cantidad significativa de falsos positivos. Esto es aceptable en campañas donde el costo de contactar a un cliente que no convertirá es bajo comparado con el beneficio de captar uno que sí lo hará.
- ◆ **F1 Score = 0.65:** El balance entre precisión y recall es razonable, confirmando que el modelo logra un buen compromiso entre ambas métricas.
- ◆ **Accuracy = 0.90:** Aunque elevado, debe interpretarse con precaución dada la desbalanceada distribución de clases (mayoría de "No").
- ◆ **Matriz de Confusión:**

Verdaderos negativos: 6664 Falsos positivos: 644 Verdaderos positivos: 749 Falsos negativos: 179

Esto indica que el modelo logra identificar correctamente el 81% de los clientes que aceptan la oferta, aunque todavía existen errores de clasificación (clientes que no aceptarán pero son identificados como positivos).

En conjunto, estas métricas y el gráfico ROC sugieren que el modelo XGBoost optimizado representa una **solución robusta para predecir la aceptación de campañas**, especialmente si se prioriza la captación de clientes potenciales por encima del costo de algunos falsos positivos.

Ahora, muestro las 10 mejores combinaciones de hiperparámetros junto con su score promedio y desviación estándar.

```
: # Convertimos los resultados del GridSearchCV a un DataFrame para facilitar el análisis
resultados = pd.DataFrame(grid_search.cv_results_)
# Ordenamos las filas del DataFrame según el F1 Score promedio obtenido en validación cruzada (de mayor a menor)
resultados = resultados.sort_values(by="mean_test_score", ascending=False)
# Mostramos las 10 mejores combinaciones de hiperparámetros junto con su score promedio y desviación estándar
resultados[["params", "mean_test_score", "std_test_score"]].head(10)
```

Una vez obtenidos los mejores hiperparámetros entonces procedo a guardarlo como el **modelo final**.

```
: # Asignamos el mejor modelo encontrado por GridSearchCV (ya entrenado) a una variable para usarlo como modelo final
model_final = grid_search.best_estimator_

: # Imprimo el tipo de objeto del modelo final; en este caso, un clasificador XGBoost ya entrenado
print(type(model_final))
# Muestro todos los hiperparámetros (tanto los definidos manualmente como los optimizados por GridSearchCV)
print(model_final.get_params())
# Realizo predicciones sobre el conjunto de test con el modelo final optimizado
model_final.predict(X_test)

: # Guardar a disco el modelo final ajustado
joblib.dump(model_final, 'modelo_final_xgb.pkl')
```

Este modelo puede ahora ser evaluado, exportado o **integrado dentro de un pipeline de despliegue para su uso en aplicaciones reales**.

6- INTERPRETABILIDAD Y EXPLICABILIDAD DE MODELOS

En esta sección se aborda la importancia de comprender el funcionamiento interno del modelo predictivo desarrollado, con el fin de generar confianza, validar su comportamiento y facilitar la toma de decisiones basadas en sus resultados. La **interpretabilidad y explicabilidad** son aspectos clave para garantizar que las predicciones no solo sean precisas, sino también comprensibles para usuarios técnicos y no técnicos.

Se explorarán técnicas y herramientas que permiten analizar la contribución de las variables de entrada, identificar patrones relevantes y explicar las decisiones del modelo, especialmente en contextos donde la transparencia es fundamental para la adopción de la solución (como es el sector bancario que nos compete aquí en este proyecto).

6.1 Feature importance

6.1 Feature importance

El **análisis de importancia de variables** permite identificar cuáles características del conjunto de datos tienen mayor influencia en las predicciones del modelo. Esta información es fundamental para comprender el comportamiento del modelo, detectar posibles sesgos y validar si las variables más relevantes tienen sentido desde el punto de vista del dominio del problema.

```
: # Crear un DataFrame con las importancias de las variables del modelo final
feat_importances = pd.DataFrame(model_final.feature_importances_, index=X_train.columns, columns=["Importance"])
# Ordenar las variables de mayor a menor importancia
feat_importances.sort_values(by='Importance', ascending=False, inplace=True)

# Seleccionar las 25 variables más importantes
top_features = feat_importances.head(25)

# Grafico las 25 variables más relevantes para el modelo final
top_features.plot(kind='bar', figsize=(10, 6))
plt.title("Top 25 Features más importantes")
plt.ylabel("Importancia")
plt.xticks(rotation=45, ha='right') # Roto las etiquetas del eje X para mejorar la legibilidad del gráfico
plt.tight_layout() # Ajusto el layout para evitar recortes en el gráfico
plt.show()
```

Insights preliminares de las features

1. La variable `duration` domina ampliamente la predicción del modelo

- Es, por mucho, la característica más influyente.
- Esto es esperable, ya que la duración de la llamada está altamente correlacionada con la respuesta positiva del cliente: a mayor duración, mayor probabilidad de aceptación.

2. Variables económicas externas como `euribor3m`, `cons.conf.idx` y `cons.price.idx` tienen alta importancia

- Estas variables macroeconómicas afectan significativamente la decisión del cliente, reflejando que el contexto económico influye en la predisposición a contratar productos financieros.
- Es recomendable tener en cuenta el entorno macroeconómico al lanzar campañas.

3. Variables como `default_no`, `emp.var.rate` y `day_of_week` también presentan relevancia destacada

- `default_no` sugiere que clientes con historial crediticio limpio tienen más probabilidad de aceptar.
- `day_of_week` indica que el día del contacto podría tener un efecto relevante en la tasa de conversión.

4. El tipo de contacto (`contact_cellular`) y la frecuencia de contactos previos (`campaign`, `pdays`, `previous`) son relevantes

4. El tipo de contacto (`contact_cellular`) y la frecuencia de contactos previos (`campaign` , `pdays` , `previous`) son relevantes

- Reflejan el impacto del canal y la intensidad de la campaña sobre la respuesta del cliente.

5. Variables categóricas derivadas como `job_grouped` , `education_grouped` , `life_stage` y `socio-economic` aparecen con menor importancia

- Aunque su impacto individual es bajo, aportan valor contextual y pueden ser relevantes en combinación con otras variables.

Conclusiones parciales de las features

- El modelo se apoya fuertemente en variables que describen la **interacción durante la campaña** (especialmente `duration`) y en **indicadores macroeconómicos**.
- El perfil sociodemográfico del cliente tiene un impacto moderado, lo cual sugiere que la disposición a contratar el producto depende más del contexto que del perfil estático del cliente.
- Estos hallazgos permiten identificar oportunidades de optimización para campañas futuras:
 - Elegir los días más efectivos para contactar (`day_of_week`).
 - Evitar campañas demasiado insistentes (controlar `campaign` y `pdays`).
 - Lanzar campañas en momentos macroeconómicamente favorables (`euribor3m` , `cons.conf.idx`).

Este análisis refuerza la importancia de combinar información contextual, del cliente y del entorno económico para mejorar la eficacia de las campañas de marketing predictivo.

6.2 SHAP y LIME para interpretación local/global

Explicabilidad Local con LIME

Con el objetivo de comprender el comportamiento del modelo a nivel individual, se utilizó la técnica **LIME** (*Local Interpretable Model-agnostic Explanations*). Esta herramienta permite generar explicaciones locales e interpretables para modelos complejos, proporcionando información sobre cómo cada característica influyó en la predicción de una instancia específica.

LIME actúa generando ligeras perturbaciones alrededor del punto de interés y entrenando un modelo interpretable (por lo general, lineal) sobre estos datos simulados. De este modo, aproxima el comportamiento del modelo original en un entorno local, permitiendo identificar las variables que más contribuyeron a la predicción, así como el sentido de dicha contribución (positivo o negativo).

En este caso, se analizó una observación particular del conjunto de prueba, y se generó una explicación visual que destaca las 30 características más influyentes en la decisión del modelo. Esta aproximación permite validar las decisiones del modelo y facilita su interpretación para usuarios finales o stakeholders no técnicos.

```
# Se crea una instancia de LimeTabularExplainer para generar explicaciones locales del modelo.
# Esta clase entrena un modelo interpretable (e.g., lineal) sobre datos perturbados de una instancia puntual.
explainer = lime.lime_tabular.LimeTabularExplainer(X_train.values, # Datos de entrenamiento en formato numpy array
                                                    mode='classification', # Tipo de tarea: clasificación
                                                    training_labels=y_train, # Etiquetas reales del conjunto de entrenamiento
                                                    feature_names=X_train.columns, # Nombres de las variables para que la explicación sea legible
                                                    random_state=42) # Fijación de semilla para reproducibilidad

# Selecciono un registro del dataset de prueba al azar junto con sus valores.
X_test.iloc[100]

# Se obtienen las probabilidades predichas por el modelo.
model_final.predict_proba(X_test)[: , 1]

# Se obtienen las predicciones finales del modelo para el conjunto de test. Cada valor corresponde a la clase predicha (0 o 1) para cada observación.
model_final.predict(X_test)

# Genero aquí una explicación local con LIME para una instancia específica del conjunto de test.
# En este caso, analizo la observación en la posición 100.
# LIME utiliza el método predict_proba del modelo para estimar la contribución de cada variable.
# El parámetro num_features=30 indica que se mostrarán las 30 variables más influyentes en la predicción en esta celda.
exp=explainer.explain_instance(X_test.iloc[100], model_final.predict_proba,num_features=30)

# Luego muestro la explicación generada de forma interactiva dentro del notebook.
# El gráfico resultante indica el impacto (positivo o negativo, con color naranja o azul respectivamente) por variable sobre la predicción del modelo.
exp.show_in_notebook()
```

En la celda anteriores limité la cantidad de variables a considerar por LIME, en cambio en la siguiente celda incluyo todas las variables para dar una explicabilidad más exhaustiva.

```
# Genero aquí nuevamente la explicación local con LIME para una instancia específica del conjunto de test (registro 100).
# Ahora habilito que aparezcan todas las variables y su explicabilidad.
exp.show_in_notebook(show_all=True)

# Se guarda la explicación generada en un archivo HTML interactivo para su consulta o presentación posterior.
exp.save_to_file('explanation.html')
```

Explicabilidad Local con LIME

Explicabilidad Local con LIME

Se utilizó la herramienta **LIME (Local Interpretable Model-Agnostic Explanations)** para analizar el comportamiento del modelo XGBoost sobre un caso específico del conjunto de testeo. LIME permite comprender **por qué el modelo tomó una decisión concreta**, revelando qué características influyeron más en la predicción de esa instancia.

Resultado de la Predicción

- **Probabilidad predicha de aceptación (clase "Yes"):** 0.72
 - **Probabilidad de no aceptación:** 0.28
- El modelo predice con alta confianza que el cliente aceptará la oferta.

Principales características que contribuyeron positivamente (hacia clase "Yes"):

- `age = 0.42`: la edad del cliente (escalada entre 0 y 1) se vincula positivamente con la predicción.
- `month = 0.33`: el mes en que se realizó el contacto parece tener influencia positiva.
- `day_of_week = 1.00`: día de la semana puede haber coincidido con una tendencia de mayor conversión.
- `education_university.degree = 1.00`: el cliente posee título universitario, lo que se relaciona positivamente con la aceptación.
- `marital_single = 1.00`: clientes solteros parecen más propensos a aceptar la campaña.
- `housing_yes = 1.00` y `loan_yes = 1.00`: aunque se tiene préstamo y vivienda, en este caso no actuaron como frenos para la predicción positiva.
- `contact_cellular = 1.00`: contacto por celular tiende a estar asociado a respuestas más afirmativas.
- `life_stage_middle aged & single = 1.00` y `job_grouped_white-collar = 1.00`: estos perfiles socio-demográficos están asociados a una mayor propensión a aceptar.

Características con impacto neutro o bajo:

- Variables como `pdays`, `previous`, `campaign`, `job_blue-collar`, `outcome_failure`, y muchas combinaciones socioeconómicas no presentaron impacto relevante (valor = 0.00), lo que indica que no influyeron en esta predicción particular.

Conclusiones de la explicación local

- La predicción **no solo se apoya en variables cuantitativas**, como `age`, `duration`, o `emp.var.rate`, sino también en **variables categóricas codificadas**, relacionadas con nivel educativo, tipo de empleo y estado civil.
- LIME revela que el modelo **asocia ciertos perfiles sociodemográficos con mayor probabilidad de aceptación**, en línea con el análisis general realizado con SHAP.
- Esta explicación local valida que el modelo no es una "caja negra", y que su decisión se basa en **factores interpretables y coherentes con el dominio del negocio**.

Explicabilidad Global con SHAP

Para entender el comportamiento general del modelo y la importancia relativa de cada variable, se utilizó la técnica **SHAP (SHapley Additive exPlanations)**. Esta metodología calcula, para cada predicción individual, la contribución de cada característica utilizando conceptos de la teoría de juegos, y luego permite agregar esas explicaciones para obtener una visión global.

En este trabajo, se calcularon los valores SHAP para una muestra representativa del conjunto de test (100 observaciones). El gráfico resumen (`summary_plot`) generado a partir de estos valores visualiza la influencia promedio y el efecto que cada variable tiene sobre las predicciones del modelo.

Esta aproximación ofrece una explicación global del modelo, resaltando qué variables son más determinantes y cómo sus diferentes valores afectan la salida del clasificador, facilitando la interpretación y validación del modelo completo.

```
# Se crea un objeto explainer SHAP para el modelo entrenado.
explainer = shap.Explainer(model_final)
# Se calculan los valores SHAP para las primeras 100 instancias del conjunto de test, obteniendo la contribución de cada variable para cada predicción.
shap_values = explainer.shap_values(X_test.iloc[0:100])
# Se genera un gráfico resumen que muestra la importancia global de las variables y cómo sus valores afectan las predicciones a través de la distribución.
shap.summary_plot(shap_values, X_test.iloc[0:100])
```

Interpretabilidad Global del Modelo – SHAP Summary Plot

En el gráfico SHAP summary se observa lo siguiente:

- `duration` es, con diferencia, la variable que más impacto tiene en la predicción del modelo. A mayor duración de la llamada (color rosa), mayor es la probabilidad de que el cliente acepte el producto. Esto se alinea con la lógica comercial: llamadas más largas suelen indicar mayor interés.
- Variables económicas como `emp.var.rate` (tasa de variación de empleo) y `euribor3m` (tipo de interés a 3 meses) también tienen fuerte influencia. Sus valores más bajos (color azul) están asociados a un aumento en la probabilidad de aceptación, lo que puede reflejar épocas de menor confianza económica donde las campañas bancarias son más efectivas.
- Variables de comportamiento como `campaign` (número de contactos durante la campaña), `pdays` y `previous` muestran que más contactos previos tienden a tener un efecto negativo si no fueron exitosos, reflejando potencial saturación del cliente.
- Variables categóricas transformadas, como `contact_cellular`, `default_no`, `education_university.degree`, entre otras, también muestran influencia aunque menor, evidenciando el impacto de factores demográficos y de canal de contacto.

Este gráfico proporciona una visión clara y transparente de cómo las diferentes variables interactúan con el modelo, reforzando la confianza en su funcionamiento y permitiendo una mejor toma de decisiones en un entorno bancario.

6.3 Discusión sobre sesgos y robustez del modelo

Discusión sobre Sesgos y Robustez del Modelo

El análisis predictivo realizado se basó en un dataset con características demográficas, laborales y de interacción previa con la entidad bancaria durante campañas de marketing. Si bien el modelo XGBoost optimizado alcanzó métricas destacadas (AUC = 0.95, F1 Score = 0.65, Recall = 0.81), es fundamental evaluar críticamente su **robustez ante variaciones de los datos** y la posible existencia de **sesgos inherentes**.

Posibles sesgos del modelo

- **Desbalance de clases:** La variable objetivo presenta un marcado desbalance, con una mayoría de clientes que no aceptaron la oferta. Esto puede inducir al modelo a favorecer la clase negativa ("No"), afectando la precisión sobre la clase minoritaria. Se aplicaron técnicas y métricas adecuadas como F1 y AUC para mitigar este efecto, aunque aún persisten falsos positivos y negativos.
- **Variables socioeconómicas sensibles:** Algunas variables utilizadas, como `education`, `job`, `marital status` o combinaciones socioeconómicas, pueden estar correlacionadas con factores sensibles o discriminatorios. Si bien se incluyeron por su valor predictivo, se recomienda monitorear su influencia para evitar posibles sesgos indirectos en la toma de decisiones.
- **Información histórica y sesgada por campañas previas:** Variables como `pdays` o `previous` reflejan interacciones anteriores con el cliente, lo que podría introducir un sesgo por selección si las campañas anteriores no fueron aleatorias.

Robustez del modelo

- **Validación cruzada estratificada:** Se implementó validación cruzada estratificada para asegurar estabilidad en los resultados, logrando un F1 promedio consistente durante el entrenamiento.
- **Rendimiento estable ante nuevos datos:** El desempeño sobre el conjunto de test refleja una generalización sólida del modelo, sin indicios de sobreajuste. Las métricas mantienen un nivel elevado y balanceado, especialmente el AUC, que demuestra una buena capacidad de discriminación global.
- **Explicabilidad con SHAP y LIME:** Se incorporaron herramientas de interpretabilidad (SHAP y LIME) que permiten identificar las variables más influyentes en las predicciones. Estas explicaciones refuerzan la confianza en el modelo al mostrar un razonamiento coherente con el dominio del problema.

Conclusión

El modelo XGBoost muestra una buena robustez general y capacidad de generalización. Sin embargo, se identifican potenciales fuentes de sesgo que deben considerarse si se busca aplicar el modelo en producción. Es recomendable complementar el modelo con evaluaciones éticas y análisis de equidad, especialmente si se utiliza en contextos sensibles o se automatizan decisiones comerciales.

7- EVALUACION FINAL Y SELECCION DE MODELO FINAL

En esta sección se presenta la **evaluación definitiva de los modelos desarrollados** utilizando métricas adecuadas para el problema planteado. Se comparan los resultados obtenidos para determinar cuál fue el modelo que ofrece el mejor desempeño y equilibrio entre precisión, robustez y capacidad de generalización.

Además, se describen los criterios considerados para la **selección del modelo final**, teniendo en cuenta tanto aspectos cuantitativos como cualitativos, con el objetivo de elegir la solución más adecuada para su aplicación práctica.

7.1 Comparación global de métricas

A continuación, se presenta el gráfico comparativo final entre los modelos utilizados y sus valores obtenidos para las diversas métricas de evaluación empleadas:

```
# Datos de las métricas obtenidas para cada modelo.
data = {
  'Modelo': [
    'XGBoost', 'Random Forest', 'Logistic Regression',
    'Decision Tree', 'K-Nearest Neighbors', 'Naive Bayes', 'Deep Learning'
  ],
  'AUC': [0.945, 0.941, 0.933, 0.750, 0.736, 0.776, 0.9315],
  'F1 Score': [0.6241, 0.5794, 0.5894, 0.5370, 0.3871, 0.3499, 0.6080],
  'Precision': [0.4834, 0.6049, 0.4498, 0.5067, 0.2972, 0.2277, 0.5068],
  'Recall': [0.8804, 0.5560, 0.8545, 0.5711, 0.5550, 0.7554, 0.7597],
  'Accuracy': [0.8805, 0.9091, 0.8658, 0.8890, 0.8020, 0.6837, 0.8896]
}

# Creo el DataFrame y lo transformo a formato largo (long-form)
df = pd.DataFrame(data)
df_melted = df.melt(id_vars='Modelo', var_name='Métrica', value_name='Valor')

# Ajusto el estilo y visualización
plt.figure(figsize=(12, 6))
sns.set(style="whitegrid")

# Gráfico de barras
ax = sns.barplot(data=df_melted, x='Métrica', y='Valor', hue='Modelo', palette='Set2')

# Ajustes estéticos
plt.title('Comparación de Métricas por Modelo', fontsize=16)
plt.ylim(0, 1)
plt.ylabel('Valor', fontsize=12)
```

Comparación de Modelos y Conclusión General

Se evaluaron seis modelos de clasificación sobre el conjunto de test, ajustando los parámetros para abordar el desbalance de clases donde fue posible. A continuación, se resumen las observaciones más relevantes y se comparan los modelos en función de métricas clave como **AUC**, **F1 Score**, **Precisión**, **Recall** y **Accuracy**.

Comparación General de Desempeño

Modelo	AUC	F1 Score	Precision	Recall	Accuracy
XGBoost	0.945	0.6241	0.4834	0.8804	0.8805
Random Forest	0.941	0.5794	0.6049	0.5560	0.9091
Logistic Regression	0.933	0.5894	0.4498	0.8545	0.8658
Decision Tree	0.750	0.5370	0.5067	0.5711	0.8890
K-Nearest Neighbors	0.736	0.3871	0.2972	0.5550	0.8020
Naive Bayes	0.776	0.3499	0.2277	0.7554	0.6837

Observaciones Clave

Mejor rendimiento global: XGBoost

- Logra el **mejor equilibrio entre recall y precisión para la clase positiva**.
- F1 Score más alto (0.6241)**, reflejando una combinación efectiva de precisión y cobertura.
- Recall sobresaliente (0.88)**: identifica correctamente casi el 90% de los clientes que aceptan la campaña.
- Excelente **AUC (0.945)**, lo que confirma una fuerte capacidad de discriminación.

Random Forest: Precisión elevada, pero menor cobertura

- Aunque su **precisión (0.60)** supera a XGBoost, su **recall (0.56)** es considerablemente menor, por lo que **pierde muchas oportunidades de identificar clientes interesados**.
- Su **F1 score (0.5794)** y **accuracy general (0.91)** lo convierten en una opción muy sólida si se prioriza **minimizar falsos positivos**.

Modelos base (KNN, Naive Bayes) con desempeño débil

- Ambos muestran **precisión muy baja y fuerte sesgo hacia la clase mayoritaria**, lo que los hace poco adecuados en este contexto.
- El Naive Bayes obtiene una F1 de apenas **0.3499**, y KNN **0.3871**, con una AUC muy inferior a los mejores modelos.

Modelos base (KNN, Naive Bayes) con desempeño débil

- Ambos muestran **precisión muy baja y fuerte sesgo hacia la clase mayoritaria**, lo que los hace poco adecuados en este contexto.
- El Naive Bayes obtiene una F1 de apenas **0.3499**, y KNN **0.3871**, con una AUC muy inferior a los mejores modelos.

Regresión Logística: buen recall, pero precisión limitada

- Presenta un **recall competitivo (0.8545)**, cercano al de XGBoost, pero su **precisión (0.45)** es inferior.
- Puede ser útil como benchmark, pero **genera una mayor cantidad de falsos positivos (970)**, lo cual implica costos adicionales para el área de negocio.

Conclusión Final

- XGBoost emerge como el modelo más equilibrado y eficaz**, especialmente si se busca **maximizar la identificación de clientes interesados sin comprometer demasiado la precisión**.
- Random Forest** es también una opción robusta si el objetivo es **priorizar la precisión y reducir falsos positivos**, aunque a costa de perder parte de los clientes potenciales.
- Modelos simples como Naive Bayes y KNN no son competitivos** para esta tarea, incluso con ajustes al desbalance de clases.

En función del objetivo del negocio, la elección del modelo ideal dependerá de si se desea **maximizar cobertura (recall)** o **precisión operacional**. No obstante, XGBoost ofrece el mejor **compromiso general** entre ambas dimensiones, y es el candidato preferente para la implementación final.

7.2 Selección del modelo con mejor balance interpretabilidad-rendimiento

Justificación de la selección del modelo

Luego de analizar comparativamente el rendimiento de todos los modelos evaluados, se selecciona **XGBoost con hiperparámetros optimizados mediante GridSearchCV** como el modelo final para la predicción de aceptación de campañas de marketing.

Criterios de selección

La elección se fundamenta en un conjunto de métricas y aspectos clave:

- F1 Score = 0.6454**: el más alto entre todos los modelos probados, lo que refleja el mejor equilibrio entre precisión y recall en el contexto de clases desbalanceadas.
- Recall = 0.8071**: el modelo logra capturar más del 80% de los clientes que efectivamente aceptan la oferta, lo cual es fundamental si se busca maximizar la tasa de respuesta.
- AUC = 0.9495**: Excelente capacidad de discriminación, lo que confirma una fuerte capacidad de discriminación.

Criterios de selección

La elección se fundamenta en un conjunto de métricas y aspectos clave:

- **F1 Score = 0.6454**: el más alto entre todos los modelos probados, lo que refleja el mejor equilibrio entre precisión y recall en el contexto de clases desbalanceadas.
- **Recall = 0.8071**: el modelo logra capturar más del 80% de los clientes que efectivamente aceptan la oferta, lo cual es fundamental si se busca maximizar la tasa de respuesta.
- **AUC = 0.9485**: confirma una capacidad de discriminación sobresaliente entre las clases positivas y negativas.
- **Precision = 0.5377**: aunque moderada, se considera aceptable en el contexto del marketing directo, donde el costo de contactar falsos positivos es relativamente bajo frente al beneficio de captar nuevos clientes.
- **Accuracy = 0.90**: alto nivel de aciertos globales, aunque este valor se interpreta con cautela debido al desbalance de clases.
- **Robustez y estabilidad**: el modelo mostró un rendimiento estable tanto en validación cruzada como en test, sin signos evidentes de sobreajuste.
- **Interpretabilidad**: mediante herramientas como SHAP y LIME se logró una buena comprensión del funcionamiento interno del modelo, fortaleciendo su confiabilidad.

Comparación con otros modelos

Aunque modelos como **Random Forest** y **Regresión Logística** ofrecieron buenos resultados, no lograron superar al modelo XGBoost en términos de F1 Score y recall, métricas críticas para este caso de uso.

En contraste, modelos como **Naive Bayes** o **KNN** mostraron un desempeño significativamente inferior y fueron descartados.

Conclusión

El modelo XGBoost tuneado se posiciona como la solución más **eficiente, equilibrada y robusta** para predecir la aceptación de campañas de marketing en el presente caso, y será utilizado como base para la implementación final.

Por lo tanto, se selecciona el modelo **XGBoost con balanceo, escalado y selección automática de variables** como el modelo final del presente trabajo, al maximizar la capacidad del sistema para identificar clientes que efectivamente aceptarán una campaña, favoreciendo así la eficiencia en la gestión comercial del banco.

8- PRODUCTIVIZACIÓN DEL MODELO

La productivización del modelo es el proceso de preparar y desplegar el modelo entrenado para su uso en entornos reales, garantizando que funcione de manera eficiente, escalable y mantenible.

En este TFM, se busca que el modelo predictivo pueda integrarse en un sistema que reciba datos nuevos, aplique el preprocesamiento adecuado y entregue predicciones automáticas. Para ello, se guarda el pipeline completo (preprocesamiento y modelo) usando herramientas como **joblib**, facilitando su carga y uso en producción.

Este paso es clave para transformar el trabajo de investigación en una solución práctica que aporte valor al banco, permitiendo la toma de decisiones basada en datos en tiempo real.

Primero se comienza por replicar el **preprocesamiento** utilizado en las etapas de desarrollo de este Notebook que permitirá transformar los datos ingresados en el chatbot del proyecto.

```
# =====
# 1. Clase de preprocesamiento manual personalizado
# =====

class PreprocessingTransformer(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()

        # Reglas personalizadas
        X.loc[(X['age'] > 60) & (X['job'] == 'unknown'), 'job'] = 'retired'
        X.loc[(X['education'] == 'unknown') & (X['job'] == 'management'), 'education'] = 'university.degree'
        X.loc[(X['education'] == 'unknown') & (X['job'] == 'services'), 'education'] = 'high.school'
        X.loc[(X['education'] == 'unknown') & (X['job'] == 'housemaid'), 'education'] = 'basic.4y'
        X.loc[(X['job'] == 'unknown') & (X['education'] == 'basic.4y'), 'job'] = 'blue-collar'
        X.loc[(X['job'] == 'unknown') & (X['education'] == 'basic.6y'), 'job'] = 'blue-collar'
        X.loc[(X['job'] == 'unknown') & (X['education'] == 'basic.9y'), 'job'] = 'blue-collar'
        X.loc[(X['job'] == 'unknown') & (X['education'] == 'professional.course'), 'job'] = 'technician'

        # Transformaciones simples
        X['pdays'] = X['pdays'].apply(lambda x: 0 if x == 999 else x)
```



```
# Transformaciones simples
X['pdays'] = X['pdays'].apply(lambda x: 0 if x == 999 else x)

X['month'].replace(
    ('jan', 'feb', 'mar', 'apr', 'may', 'jun', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec'),
    range(1, 13), inplace=True)

X['day_of_week'].replace(
    ('mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun'),
    range(1, 8), inplace=True)

# Binning de edad
bins = [0, 25, 40, 60, 140]
labels = ['young', 'lower middle aged', 'middle aged', 'senior']
X['age_binned'] = pd.cut(X['age'], bins=bins, labels=labels, right=True, include_lowest=True)

# Agrupamientos
job_map = {
    'admin': 'White-collar', 'management': 'White-collar', 'technician': 'White-collar',
    'blue-collar': 'Blue-collar', 'services': 'Blue-collar', 'housemaid': 'Blue-collar',
    'entrepreneur': 'Self-employed', 'self-employed': 'Self-employed',
    'retired': 'Non-active', 'student': 'Non-active', 'unemployed': 'Non-active',
    'unknown': 'Other'
}
X['job_grouped'] = X['job'].map(job_map)

education_map = {
    'basic.9y': 'Basic', 'basic.4y': 'Basic', 'basic.6y': 'Basic',
    'high.school': 'Middle', 'professional.course': 'Middle',
    'university.degree': 'Superior', 'unknown': 'Other', 'illiterate': 'Other'
}
X['education_grouped'] = X['education'].map(education_map)

# Nuevas variables combinadas
X['contacted_previously'] = (X['previous'] >= 1).astype(int)
X['life_stage'] = X['age_binned'].astype(str) + ' & ' + X['marital']
X['socio-economic'] = X['job'].astype(str) + ' & ' + X['education']

# Drop de columnas redundantes
X.drop(columns=['nr.employed'], inplace=True)

return X
```

Luego se continúa por establecer el mecanismo por el cual se **seleccionan solo un conjunto determinado de features**, las cuales coinciden con las features con las que se entrenó el modelo final optimizado.

```
# =====
# 2. Clase FeatureSelector personalizada
# =====

class FeatureSelector(BaseEstimator, TransformerMixin):
    def __init__(self, feature_names):
        self.feature_names = feature_names
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        return X[self.feature_names]
```

Posteriormente, se reproduce un proceso de carga de datos, a los cuales se les quita la variable objetivo y se obtienen sus variables según sus tipologías específicas.

```
# =====
# 3. Preparación de Los datos
# =====

#Efectúo la Lectura del csv y guardo todo en un dataframe de Pandas con nombre df
X = pd.read_csv('bank-additional-full.csv', sep=';')

y = X['y'].replace(['no': 0, 'yes': 1])
X = X.drop(columns=['y'])

# Cargar las features seleccionadas previamente (después del encoding)
selected_features_rfecv = joblib.load('selected_features_rfecv.pkl')

model_final = joblib.load("modelo_final_xgb.pkl")

# Aplicar PreprocessingTransformer para generar nuevas variables
X_temp = PreprocessingTransformer().fit_transform(X)

# Detectar tipo de variables sobre el resultado del paso 1 (antes de codificar)
numeric_cols = X_temp.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = X_temp.select_dtypes(include=['object', 'category']).columns.tolist()
```

Para manejar las variables categóricas en el pipeline de preprocesamiento, vuelvo a utilizar la técnica de **codificación One Hot Encoding**.

```
]: # =====  
# 4. OneHotEncoder personalizado como Transformer  
# =====  
  
# Clase que implementa un transformer para codificar variables categóricas mediante One Hot Encoding, integrable en pipelines de scikit-learn.  
class OneHotEncoderTransformer(BaseEstimator, TransformerMixin):  
    def __init__(self, categorical_cols):  
        self.categorical_cols = categorical_cols  
        self.encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')  
  
    def fit(self, X, y=None):  
        X_ = X[self.categorical_cols].astype(str)  
        self.encoder.fit(X_)  
        self.feature_names_out = self.encoder.get_feature_names_out(self.categorical_cols)  
        return self  
  
    def transform(self, X):  
        X = X.copy()  
        X_cat = X[self.categorical_cols].astype(str)  
        encoded = self.encoder.transform(X_cat)  
        df_encoded = pd.DataFrame(encoded, columns=self.feature_names_out, index=X.index)  
        X.drop(columns=self.categorical_cols, inplace=True)  
        X = pd.concat([X, df_encoded], axis=1)  
        return X
```

Ahora, con la siguiente clase `ManualScaler` implemento un transformer compatible con scikit-learn que aplica `MinMaxScaler` únicamente a las columnas numéricas especificadas.

```
: # =====  
# 5. Transformador personalizado para escalado manual de variables numéricas  
# =====  
  
# Transformer personalizado que aplica MinMaxScaler únicamente a las columnas numéricas indicadas.  
class ManualScaler(BaseEstimator, TransformerMixin):  
    def __init__(self, numeric_cols):  
        self.numeric_cols = numeric_cols  
        self.scaler = MinMaxScaler()  
  
    def fit(self, X, y=None):  
        self.scaler.fit(X[self.numeric_cols])  
        return self  
  
    def transform(self, X):  
        X = X.copy()  
        X[self.numeric_cols] = self.scaler.transform(X[self.numeric_cols])  
        return X
```

En esta siguiente etapa se **construye el pipeline completo** que integra todas las fases del preprocesamiento personalizado, la codificación de variables categóricas, la selección de características, el escalado de variables numéricas y el modelo final resultante.

El pipeline se entrena con los datos disponibles y, una vez ajustado, se guarda en disco utilizando `joblib`. Esto permite reutilizar el pipeline entrenado para realizar predicciones sobre datos nuevos sin necesidad de repetir todo el proceso desde cero.

```
# =====  
# 6. Construcción, entrenamiento y guardado del pipeline final  
# =====  
  
# Reutilizo las clases que generé antes  
pipeline_final = Pipeline([  
    ('manual_preprocessing', PreprocessingTransformer()),  
    ('encoding', OneHotEncoderTransformer(categorical_cols=categorical_cols)),  
    ('feature_selection', FeatureSelector(selected_features_rfecv)),  
    ('scaling', ManualScaler(numeric_cols=numeric_cols)),  
    ('modelo', model_final)  
)  
  
# Ajusto el pipeline  
pipeline_final.fit(X, y)  
  
# Guardado para producción  
joblib.dump(pipeline_final, 'pipeline_modelo_completo.pkl')
```

Una vez guardado el pipeline completo (que incluye todas las transformaciones y el modelo entrenado), es posible cargarlo para su uso posterior sin necesidad de reentrenar.

La celda de código a continuación, **inspecciona sus componentes internos para verificar las etapas y objetos que contiene el pipeline completo**.

```
# =====  
# 7. Carga del pipeline guardado y acceso/visión de sus componentes  
# =====  
  
# Cargo el pipeline completo previamente guardado, que incluye preprocesamiento y modelo  
modelo = joblib.load("pipeline_modelo_completo.pkl")  
# Visualización de los pasos del pipeline para verificar su estructura y componentes  
modelo.named_steps
```

Desglose de los componentes del pipeline cargado

Al cargar el pipeline completo guardado, podemos inspeccionar sus etapas internas con `named_steps`.

El output muestra que el pipeline contiene las siguientes fases:

- **manual_preprocessing**: Transformer personalizado para aplicar reglas manuales de preprocesamiento.
- **encoding**: Transformer que realiza One Hot Encoding sobre las columnas categóricas indicadas.

Desglose de los componentes del pipeline cargado

Al cargar el pipeline completo guardado, podemos inspeccionar sus etapas internas con `named_steps`.

El output muestra que el pipeline contiene las siguientes fases:

- **manual_preprocessing**: Transformer personalizado para aplicar reglas manuales de preprocesamiento.
- **encoding**: Transformer que realiza One Hot Encoding sobre las columnas categóricas indicadas.
- **feature_selection**: Selector de características basado en la lista de features seleccionadas tras análisis previo.
- **scaling**: Escalador personalizado que aplica MinMaxScaler solo a las variables numéricas especificadas.
- **modelo**: El modelo entrenado final, en este caso un `XGBClassifier` con parámetros ajustados.

Este desglose permite verificar que todas las etapas necesarias para el procesamiento y predicción están correctamente integradas en el pipeline, facilitando su uso y mantenimiento.

Detalle de implementación productiva final

Finalmente, se señala que en el repositorio de GitHub previamente referenciado se incluye el archivo **app.py**, el cual constituye el núcleo del backend de la aplicación productiva desarrollada. Dicho backend, en conjunto con otros módulos y un frontend interactivo, posibilita la integración del modelo predictivo con un chatbot capaz de responder consultas del usuario mediante un LLM. Este LLM se alimenta de información obtenida a través del método RAG, a partir de chunks extraídos del informe final del TFM en formato PDF.

k.2) Archivo de aplicación backend en VisualStudioCode: **app.py**

Enlace Github: https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/app.py


```
C:\Users\PC> TFM > app.py > ...
1 import os
2 import logging
3 from flask import Flask, request, jsonify, render_template
4 from flask_cors import CORS
5 import joblib
6 import pandas as pd
7 from preprocessing import PreprocessingTransformer
8 from feature_selector import FeatureSelector
9 from onehot_transformer import OneHotEncoderTransformer
10 from manual_scaler import ManualScaler
11 from chat_rag_local import responder_pregunta
12 import traceback
13
14 # --- Configuración de logging ---
15 # Se establece el formato y nivel de logging para registrar eventos e información
16 logging.basicConfig(
17     level=logging.INFO,
18     format='%(asctime)s - %(levelname)s - %(message)s'
19 )
20 logger = logging.getLogger(__name__)
21
22 # --- Inicializo la app Flask ---
23 app = Flask(__name__)
24 CORS(app) # Habilitar CORS para permitir peticiones desde otros dominios (útil para frontend externo)
25
26 # --- Cargo el pipeline entrenado ---
27 # Se carga el modelo/pipeline completo previamente entrenado con joblib
28 modelo = joblib.load("pipeline_modelo_completo.pkl")
29
30 # --- Variables numéricas y requeridas ---
31 # Se definen los nombres de las variables que deben ser numéricas
32 numeric_vars = {
33     "age", "duration", "campaign", "pdays", "previous",
34     "emp.var.rate", "cons.price.idx", "cons.conf.idx", "euribor3m", "nr.employed"
35 }
36 # Conjunto de variables que se esperan obligatoriamente para hacer la predicción
37 variables_requeridas = {
38     "age", "job", "marital", "education", "default", "housing", "loan", "contact",
39     "month", "day_of_week", "duration", "campaign", "pdays", "previous", "poutcome",
40     "emp.var.rate", "cons.price.idx", "cons.conf.idx", "euribor3m", "nr.employed"
41 }
42
43 # --- Rutas de la aplicación Flask ---
44 @app.route('/')
45 def home():
46     # Renderiza la página principal con el frontend del chatbot
47     return render_template("chat.html")
48
49 @app.route('/predict', methods=['POST'])
50 def predict():
51     # Recibe los datos enviados en formato JSON desde el frontend
52     datos_usuario = request.json or {}
53     # Verifica que se haya enviado un JSON válido
54     if not datos_usuario:
55         return jsonify({"error": "No se enviaron datos JSON válidos."}), 400
56
57     # Verifica si faltan variables obligatorias
58     faltantes = variables_requeridas - datos_usuario.keys()
59     if faltantes:
60         return jsonify({"error": f"Faltan variables obligatorias: {', '.join(sorted(faltantes))"}"), 400
61
62     # Detecta si se enviaron variables extra no reconocidas
63     extra = datos_usuario.keys() - variables_requeridas
64     if extra:
65         logger.warning(f"Se enviaron variables extra: {extra}")
66
67     # Convierte las variables numéricas a float
68     for var in numeric_vars:
69         if var in datos_usuario:
70             try:
71                 datos_usuario[var] = float(datos_usuario[var])
72             except ValueError:
73                 return jsonify({"error": f"La variable '{var}' debe ser numérica."}), 400
74
75     try:
76         # Crea un DataFrame con los datos del usuario para pasarlos al pipeline
77         df = pd.DataFrame([datos_usuario])
78         logger.info(f"Predicción recibida con columnas: {df.columns.tolist()}")
79         # Realiza la predicción y obtiene la probabilidad de la clase positiva
80         prediccion = int(modelo.predict(df)[0])
81         probabilidad = float(modelo.predict_proba(df)[0][1])
82         # Devuelve la predicción y la probabilidad al frontend
83         return jsonify({
84             "prediccion": prediccion,
85             "probabilidad": round(probabilidad, 4)
86         })
87
```

```
C:\Users\PC> TFM > app.py > ...
50 def predict():
51     try:
52         # Crea un DataFrame con los datos del usuario para pasarlos al pipeline
53         df = pd.DataFrame([datos_usuario])
54         logger.info(f"Predicción recibida con columnas: {df.columns.tolist()}")
55         # Realiza la predicción y obtiene la probabilidad de la clase positiva
56         prediccion = int(modelo.predict(df)[0])
57         probabilidad = float(modelo.predict_proba(df)[0][1])
58         # Devuelve la predicción y la probabilidad al frontend
59         return jsonify({
60             "prediccion": prediccion,
61             "probabilidad": round(probabilidad, 4)
62         })
63     except Exception:
64         # Captura errores en la predicción y los registra en el log
65         logger.error("Error en predicción:\n" + traceback.format_exc())
66         return jsonify({"error": "Error al procesar la predicción. Verifique los datos enviados."}), 500
67
68 @app.route('/rag_chat', methods=['POST'])
69 def rag_chat():
70     # Recibe la pregunta enviada desde el frontend para el sistema RAG
71     pregunta = request.json.get("pregunta", "").strip()
72     if not pregunta:
73         return jsonify({"error": "No se recibió una pregunta válida."}), 400
74
75     try:
76         # Llama a la función de RAG que genera la respuesta basada en los documentos indexados
77         respuesta = responder_pregunta(pregunta)
78         return jsonify({"pregunta": pregunta, "respuesta": respuesta})
79     except Exception:
80         # Captura errores en el procesamiento RAG
81         logger.error("Error en RAG:\n" + traceback.format_exc())
82         return jsonify({"error": "Ocurrió un error interno al procesar la pregunta."}), 500
83
84 # --- Ejecución con Waitress (compatible con Windows y producción) ---
85 if __name__ == "__main__":
86     from waitress import serve
87     port = int(os.environ.get("PORT", 5000)) # Permite configurar el puerto vía variable de entorno
88     serve(app, host="0.0.0.0", port=port) # Inicia la app usando Waitress, adecuada para producción
```

[k.3\) Archivo de generación de índice RAG en VisualStudioCode: build_rag_index.py](#)

Enlace Github:

https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/build_rag_index.py

```
C: > Users > PC > TFM > build_rag_index.py > ...
1 import os
2 import json
3 import faiss
4 import numpy as np
5 from pathlib import Path
6 from sentence_transformers import SentenceTransformer
7 from PyPDF2 import PdfReader
8
9 # =====
10 # CONFIG
11 # =====
12 # Rutas y parámetros principales
13 PDF_PATH = "TFM_Mauro_Alexis_Fernandez.pdf" # PDF origen para construir el índice
14 CHUNK_SIZE = 700 # Cantidad de caracteres por fragmento (chunk)
15 CHUNK_OVERLAP = 100 # Superposición entre fragmentos para no perder contexto
16 INDEX_NAME = "rag_index.faiss" # Nombre del archivo donde se guardará el índice FAISS
17 METADATA_NAME = "rag_metadata.json" # Archivo para guardar texto de cada chunk
18 MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2" # Modelo de embeddings
19
20 # =====
21 # FUNCIONES
22 # =====
23 def smart_chunk(text, chunk_size=700, overlap=100):
24     """Divide un texto largo en chunks con solapamiento"""
25     if len(text) <= chunk_size:
26         return [text]
27     chunks = []
28     start = 0
29     while start < len(text):
30         # Genera el fragmento desde start hasta start+chunk_size
31         end = min(len(text), start + chunk_size)
32         chunks.append(text[start:end])
33         # Avanza al siguiente fragmento restando el solapamiento
34         start += chunk_size - overlap
35     return chunks
36
37 def extract_chunks_from_pdf(path):
38     reader = PdfReader(path)
39     full_text = ""
40     for page in reader.pages:
41         # Concatena texto de cada página
42         full_text += page.extract_text() + "\n"
43     return smart_chunk(full_text)
44
45 # =====
46 # MAIN
```

```
C: > Users > PC > TFM > build_rag_index.py > ...
37 def extract_chunks_from_pdf(path):
43     return smart_chunk(full_text)
44
45 # =====
46 # MAIN
47 # =====
48 print("✅ Cargando modelo de embeddings...")
49 model = SentenceTransformer(MODEL_NAME) # Carga el modelo de transformadores para embeddings
50
51 print("📄 Extrayendo y dividiendo texto del PDF...")
52 chunks = extract_chunks_from_pdf(PDF_PATH) # Lee y chunkifica el PDF
53 print(f"🔗 Total de chunks generados desde el PDF: {len(chunks)}")
54
55 print("📊 Generando embeddings...")
56 embeddings = model.encode(chunks, show_progress_bar=True) # Convierte cada chunk a un vector
57
58
59 print("💾 Guardando índice FAISS y metadatos...")
60 d = embeddings.shape[1] # Dimensión de los vectores
61 index = faiss.IndexFlatL2(d) # Crea índice plano (L2)
62 index.add(np.array(embeddings)) # Agrega todos los vectores al índice
63 faiss.write_index(index, INDEX_NAME) # Guarda índice en disco
64
65 # Guarda también el texto original de cada chunk para referencia
66 with open(METADATA_NAME, "w", encoding="utf-8") as f:
67     json.dump({"chunks": chunks}, f, ensure_ascii=False, indent=2)
68
69 print("✅ ¡Índice RAG creado con éxito desde el PDF!")
70
```

[k.4\) Archivo de chatbot con recuperación aumentada de información \(RAG\) en VisualStudioCode: chat_rag_local.py](#)

Enlace Github:

https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/chat_rag_local.py

```
C: > Users > PC > TFM > chat_rag_local.py > ...
1  import json
2  import faiss
3  import numpy as np
4  from sentence_transformers import SentenceTransformer
5  from transformers import pipeline
6  import torch
7  import os
8  from transformers import logging
9  logging.set_verbosity_info()
10
11 # =====
12 # CONFIG
13 # =====
14 # Modelos utilizados:
15 # - EMBED_MODEL: para convertir preguntas y documentos en vectores numéricos (embeddings).
16 # - QA_MODEL: modelo extractivo para responder preguntas en español a partir de un contexto.
17 EMBED_MODEL = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
18 QA_MODEL = "mrms8488/bert-base-spanish-wwm-cased-finetuned-spa-squad2-es" # modelo extractivo en español
19
20 # =====
21 # CARGA DE EMBEDDINGS
22 # =====
23 print("🔍 Cargando modelo de embeddings...")
24 embedding_model = SentenceTransformer(
25     EMBED_MODEL,
26     device='cuda' if torch.cuda.is_available() else 'cpu' # Usa GPU si está disponible
27 )
28 print("✅ Embeddings cargados")
29
30 # =====
31 # CARGA DEL MODELO QA
32 # =====
33 print(f"🔍 Cargando modelo de QA: {QA_MODEL}...")
34 qa_pipeline = pipeline(
35     "question-answering", # Pipeline de preguntas y respuestas (extractivo)
36     model=QA_MODEL,
37     tokenizer=QA_MODEL
38 )
39 print("✅ Pipeline QA listo")
40
41 # =====
42 # CARGA DEL ÍNDICE RAG
43 # =====
44 # Lee el índice FAISS generado previamente y los metadatos (texto chunkificado)
45 print("📖 Cargando índice RAG...")
46 index = faiss.read_index("rag_index.faiss")
```

```
C: > Users > PC > TFM > chat_rag_local.py > ...
40
41 # =====
42 # CARGA DEL ÍNDICE RAG
43 # =====
44 # Lee el índice FAISS generado previamente y los metadatos (texto chunkificado)
45 print("📖 Cargando índice RAG...")
46 index = faiss.read_index("rag_index.faiss")
47 with open("rag_metadata.json", "r", encoding="utf-8") as f:
48     metadatos_json = json.load(f)
49     chunks = metadatos_json.get("chunks", [])
50 print(f"✅ Índice y metadatos cargados ({len(chunks)} chunks)")
51
52 # =====
53 # FUNCIONES RAG
54 # =====
55 def recuperar_contexto(pregunta, k=3):
56     """Recupera los k chunks más relevantes usando embeddings + FAISS"""
57     # Convierte la pregunta en vector de embeddings
58     pregunta_emb = embedding_model.encode(pregunta)
59     # Busca los k más cercanos en el índice FAISS
60     _, indices = index.search(np.array([pregunta_emb]), k)
61     # Recupera el texto original de cada índice encontrado
62     contextos = [chunks[i] for i in indices[0] if 0 <= i < len(chunks)]
63     return "\n\n".join(contextos)
64
65 def responder_pregunta(pregunta):
66     """
67     Usa el pipeline de QA (extractivo) para responder
68     """
69     contexto = recuperar_contexto(pregunta)
70     if not contexto.strip():
71         return "⚠️ No encontré contexto relevante en los documentos."
72     # Pasa la pregunta y el contexto al modelo de QA extractivo
73     resultado = qa_pipeline({
74         "question": pregunta,
75         "context": contexto
76     })
77
78     return resultado["answer"]
79
```

[k.5\) Archivo con clase de selección de características en VisualStudioCode:](#)

[feature_selector.py](#)

[Enlace Github:](#)

https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/feature_selector.py

```
C:\> Users > PC > TFM > feature_selector.py > ...
1 # =====
2 # Clase FeatureSelector personalizada
3 # =====
4 # Esta clase permite seleccionar el subconjunto específico necesario de features.
5 # Es útil como paso del pipeline para mantener únicamente las columnas relevantes que requiere el modelo entrenado para predecir.
6
7
8 # Importo las librerías necesarias
9 import pandas as pd
10 from sklearn.base import BaseEstimator, TransformerMixin
11
12 class FeatureSelector(BaseEstimator, TransformerMixin):
13     def __init__(self, selected_features):
14         # Lista de nombres de columnas que se desean conservar
15         self.feature_names = selected_features
16
17     def fit(self, X, y=None):
18         # No requiere ajuste (fit) ya que la selección está basada en nombres conocidos
19         return self
20
21     def transform(self, X):
22         print("📄 Columnas recibidas en X (shape):", X.shape)
23         print("📄 Número de features seleccionadas:", len(self.feature_names))
24
25         # Si no es DataFrame, intento convertir
26         if not isinstance(X, pd.DataFrame):
27             try:
28                 # Si no lo es (ej., una sparse matrix), intenta convertirlo a DataFrame
29                 X = pd.DataFrame(X.toarray(), columns=self.feature_names)
30                 print("✅ Se reconstruyó el DataFrame desde matriz dispersa.")
31             except Exception as e:
32                 # Si falla la reconstrucción, da un mensaje de error explícito
33                 raise ValueError(
34                     "❌ No se pudo reconstruir el DataFrame desde sparse matrix.\n"
35                     "Es posible que los nombres en self.feature_names no coincidan con las columnas reales."
36                 ) from e
37
38         # Devuelvo solo las columnas seleccionadas
39         try:
40             return X[self.feature_names]
41         except KeyError as e:
42             # Si alguna columna no está presente, muestra cuáles faltan
43             raise ValueError(
44                 f"❌ Algunas columnas de self.feature_names no están presentes en X.\n"
45                 f"Columnas faltantes: {set(self.feature_names) - set(X.columns)}"
46             ) from e
```

[k.6\) Archivo con clase de escalado de variables en VisualStudioCode: manual_scaler.py](#)

Enlace Github:

https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/manual_scaler.py

```
C:\> Users > PC > TFM > manual_scaler.py > ...
1 # =====
2 # Escalador personalizado con MinMaxScaler
3 # =====
4 # Esta clase aplica el escalado Min-Max sobre columnas numéricas especificadas para usar dentro de pipeline con app.py y así replicar lo realizado en el Notebook.
5
6
7 # Importo las librerías necesarias.
8 from sklearn.base import BaseEstimator, TransformerMixin
9 from sklearn.preprocessing import MinMaxScaler
10
11
12 class ManualScaler(BaseEstimator, TransformerMixin):
13     def __init__(self, numeric_cols):
14         # Lista de columnas numéricas a escalar
15         self.numeric_cols = numeric_cols
16         # Inicializa el escalador MinMax (escalado al rango [0, 1])
17         self.scaler = MinMaxScaler()
18
19     def fit(self, X, y=None):
20         # Ajusta el escalador sólo sobre las columnas numéricas
21         self.scaler.fit(X[self.numeric_cols])
22         return self # Retorna self para integrarse en pipelines
23
24     def transform(self, X):
25         X = X.copy()
26         # Aplica el escalado Min-Max sobre las columnas numéricas
27         X[self.numeric_cols] = self.scaler.transform(X[self.numeric_cols])
28         return X # Retorno el DataFrame transformado
```

[k.7\) Archivo con clase de codificación One-Hot en VisualStudioCode:](#)
onehot_transformer.py

Enlace Github:

https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/onehot_transformer.py

```
C:\Users\PC> TFM > onehot_transformer.py > ...
1  # =====
2  # Codificador personalizado con OneHotEncoder
3  # =====
4  # Esta clase replica la codificación One-Hot sobre columnas categóricas especificadas (como las usadas en el Notebook), y la integra como paso del pipeline para app.py
5
6
7  #Importo las librerías necesarias.
8  from sklearn.base import BaseEstimator, TransformerMixin
9  from sklearn.preprocessing import OneHotEncoder
10 import pandas as pd
11
12 # Clase personalizada para codificación One-Hot
13 class OneHotEncoderTransformer(BaseEstimator, TransformerMixin):
14     def __init__(self, categorical_cols):
15         # Lista de columnas categóricas a codificar
16         self.categorical_cols = categorical_cols
17         # Inicializo el codificador OneHotEncoder (devuelve matriz densa y evita errores con categorías desconocidas)
18         self.encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
19
20
21     def fit(self, X, y=None):
22         # Convierte las columnas categóricas a string (por seguridad, en caso de que tengan tipos mixtos)
23         X_ = X[self.categorical_cols].astype(str)
24         # Ajusta el codificador a los datos
25         self.encoder.fit(X_)
26         # Guarda los nombres de las nuevas columnas codificadas
27         self.feature_names_out = self.encoder.get_feature_names_out(self.categorical_cols)
28         return self # Retorna self para cumplir con la interfaz de scikit-learn
29
30
31     def transform(self, X):
32         X = X.copy()
33         # Selecciona y convierte las columnas categóricas a string
34         X_cat = X[self.categorical_cols].astype(str)
35         # Aplica la transformación One-Hot
36         encoded = self.encoder.transform(X_cat)
37         # Convierte el resultado a DataFrame con los nombres de las columnas codificadas
38         df_encoded = pd.DataFrame(encoded, columns=self.feature_names_out, index=X.index)
39         # Elimina las columnas categóricas originales del DataFrame
40         X.drop(columns=self.categorical_cols, inplace=True)
41         # Concatena las nuevas columnas codificadas con el resto del DataFrame
42         X = pd.concat([X, df_encoded], axis=1)
43         return X # Retorno el DataFrame transformado
```

[k.8\) Archivo con clase de procesamientos manuales en VisualStudioCode:](#)
preprocessing.py

Enlace Github:

https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/preprocessing.py


```
C:\Users\> PC > TFM > preprocessing.py > ...
1  # =====
2  # Preprocesamiento manual personalizado como el efectuado en el Notebook para obtener el modelo
3  # =====
4  # Este transformador replica el preprocesamiento del notebook original para usarlo junto al archivo app.py
5  # Aplica las reglas manuales, binning, agrupaciones y crea las nuevas variables como en el Notebook.
6
7  #Importo las librerías necesarias
8  import pandas as pd
9  from sklearn.base import BaseEstimator, TransformerMixin
10
11
12 # Clase personalizada compatible con scikit-learn
13 class PreprocessingTransformer(BaseEstimator, TransformerMixin):
14     def fit(self, X, y=None):
15         return self
16     # Método transform: aplica todas las transformaciones al dataset
17     def transform(self, X):
18         X = X.copy()
19
20         # Reglas personalizadas para imputación
21         # Corrige valores 'unknown' en función de otras columnas
22         X.loc[(X['age'] > 60) & (X['job'] == 'unknown'), 'job'] = 'retired'
23         X.loc[(X['education'] == 'unknown') & (X['job'] == 'management'), 'education'] = 'university.degree'
24         X.loc[(X['education'] == 'unknown') & (X['job'] == 'services'), 'education'] = 'high.school'
25         X.loc[(X['education'] == 'unknown') & (X['job'] == 'housemaid'), 'education'] = 'basic.4y'
26         # Reglas cruzadas para asignar 'job' en base a 'education'
27         X.loc[(X['job'] == 'unknown') & (X['education'] == 'basic.4y'), 'job'] = 'blue-collar'
28         X.loc[(X['job'] == 'unknown') & (X['education'] == 'basic.6y'), 'job'] = 'blue-collar'
29         X.loc[(X['job'] == 'unknown') & (X['education'] == 'basic.9y'), 'job'] = 'blue-collar'
30         X.loc[(X['job'] == 'unknown') & (X['education'] == 'professional.course'), 'job'] = 'technician'
31
32         # ==== Limpieza de valores especiales ====
33         # pdays = 999 significa "no contactado antes", lo pasamos a 0
34         X['pdays'] = X['pdays'].apply(lambda x: 0 if x == 999 else x)
35
36         # Reemplazo de valores de texto por números (orden cronológico)
37         X['month'].replace(
38             ('jan', 'feb', 'mar', 'apr', 'may', 'jun', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec'),
39             range(1, 13), inplace=True)
40
41         X['day_of_week'].replace(
42             ('mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun'),
43             range(1, 8), inplace=True)
44
45         # Binning de edad
46         # Crea la variable categórica 'age binned' a partir de intervalos
```

```
44
45 # Binning de edad
46 # Crea la variable categórica 'age_binned' a partir de intervalos
47 bins = [0, 25, 40, 60, 140]
48 labels = ['young', 'lower middle aged', 'middle aged', 'senior']
49 X['age_binned'] = pd.cut(X['age'], bins=bins, labels=labels, right=True, include_lowest=True)
50
51 # Agrupamientos
52 # Agrupa tipos de trabajo en categorías más amplias
53 job_map = {
54     'admin.': 'White-collar', 'management': 'White-collar', 'technician': 'White-collar',
55     'blue-collar': 'Blue-collar', 'services': 'Blue-collar', 'housemaid': 'Blue-collar',
56     'entrepreneur': 'Self-employed', 'self-employed': 'Self-employed',
57     'retired': 'Non-active', 'student': 'Non-active', 'unemployed': 'Non-active',
58     'unknown': 'Other'
59 }
60 X['job_grouped'] = X['job'].map(job_map)
61
62 # Agrupa niveles educativos en categorías más amplias
63 education_map = {
64     'basic.9y': 'Basic', 'basic.4y': 'Basic', 'basic.6y': 'Basic',
65     'high.school': 'Middle', 'professional.course': 'Middle',
66     'university.degree': 'Superior', 'unknown': 'Other', 'illiterate': 'Other'
67 }
68 X['education_grouped'] = X['education'].map(education_map)
69
70 # Nuevas variables combinadas
71 # Variable binaria: si fue contactado previamente o no
72 X['contacted_previously'] = (X['previous'] >= 1).astype(int)
73 # Crea una combinación de edad binned y estado civil
74 X['life_stage'] = X['age_binned'].astype(str) + ' & ' + X['marital']
75 # Crea una combinación de trabajo y educación (como proxy socioeconómico)
76 X['socio-economic'] = X['job'].astype(str) + ' & ' + X['education']
77
78 # Drop columnas redundantes
79 # 'nr.employed' es altamente correlacionada con otras variables y fue descartada en el modelo
80 X.drop(columns=['nr.employed'], inplace=True)
81
82 return X # Retorna el DataFrame transformado
```

[k.9\) Archivo de interfaz de usuario \(frontend\) en VisualStudioCode: chat.html](#)

[Enlace Github:](#)

https://github.com/MauroAlexisFernandez/TFM_UCM_DataScience/blob/main/chat_rag_local.py

```
C:\Users\PC> TFM > templates > chat.html > html > head > style > #chatbox
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4 <meta charset="UTF-8" />
5 <title>Chatbot Bank Marketing Mejorado</title>
6 <!-- Bootstrap para estilos y componentes responsivos -->
7 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet" />
8 <style>
9 /* Contenedor principal del chat */
10 #chatbox {
11   border: 1px solid #ccc;
12   padding: 15px;
13   height: 350px;
14   overflow-y: scroll; /* Permite desplazamiento vertical */
15   margin-bottom: 15px;
16   background: #f9f9f9;
17   font-family: Arial, sans-serif;
18   border-radius: 10px;
19   display: flex;
20   flex-direction: column;
21   gap: 10px;
22 }
23 /* Burbujas de mensaje */
24 .bubble {
25   max-width: 70%;
26   padding: 10px 15px;
27   border-radius: 20px;
28   word-wrap: break-word;
29   box-shadow: 0 1px 3px rgba(0,0,0,0.2);
30 }
31 /* Estilo de mensajes del bot */
32 .bot {
33   background-color: #0d6efd;
34   color: white;
35   align-self: flex-start;
36   border-bottom-left-radius: 0;
37 }
38 /* Estilo de mensajes del usuario */
39 .user {
40   background-color: #198754;
41   color: white;
42   align-self: flex-end;
43   border-bottom-right-radius: 0;
44 }
45 /* Barra de progreso para mostrar la probabilidad */
46 .progress-bubble {
```

```

45  /* Barra de progreso para mostrar la probabilidad */
46  .progress-bubble {
47    background-color: rgba(255, 255, 255, 0.2);
48    border-radius: 10px;
49    height: 12px;
50    width: 100%;
51    margin-top: 5px;
52    overflow: hidden;
53  }
54  .progress-bar-bubble {
55    height: 100%;
56    width: 0%;
57    transition: width 0.5s ease-in-out;
58  }
59  </style>
60  </head>
61  <body class="d-flex flex-column min-vh-100">
62  <div class="container my-4 flex-grow-1 d-flex flex-column">
63    <h1 class="mb-4 text-center">Chatbot para Predicción de la suscripción a depósitos a plazo</h1>
64    <!-- Área de mensajes -->
65    <div id="chatbox" class="mb-3 flex-grow-1"></div>
66    <!-- Área de entrada dinámica (inputs o selects) -->
67    <div id="inputArea" class="mb-3"></div>
68    <!-- Botón para enviar respuestas -->
69    <div class="d-flex justify-content-center mb-4">
70      <button class="btn btn-primary" onclick="enviarRespuesta()">Enviar</button>
71    </div>
72    <!-- Área para preguntas RAG sobre el modelo -->
73    <div id="ragArea" class="mt-4" style="display: none;">
74      <h3>¿Tienes una pregunta sobre el modelo o el TFM?</h3>
75      <div class="input-group mb-3">
76        <input type="text" id="preguntaRAG" class="form-control" placeholder="Ej: ¿Cuál es el modelo más efectivo?" />
77        <button class="btn btn-secondary" onclick="enviarPreguntaRAG()">Preguntar</button>
78      </div>
79      <div id="respuestaRAG" style="white-space: pre-wrap;"></div>
80      <div id="loadingRAG" class="d-none align-items-center mt-2">
81        <div class="spinner-border spinner-border-sm text-primary me-2" role="status" aria-hidden="true"></div>
82        <span>Cargando respuesta...</span>
83      </div>
84    </div>
85  </div>
86  <!-- Footer -->
87  <footer class="bg-light text-center py-3 mt-auto border-top">

```

```

86  <!-- Footer -->
87  <footer class="bg-light text-center py-3 mt-auto border-top">
88    <small>Proyecto realizado en la <strong>Universidad Complutense de Madrid</strong></small>
89  </footer>
90  <!-- Bootstrap JS -->
91  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
92  <script>
93    /* =====
94    VARIABLES Y CATEGORÍAS
95    ===== */
96    const variables = [
97      "age", "job", "marital", "education", "default", "housing", "loan", "contact",
98      "month", "day_of_week", "duration", "campaign", "pdays", "previous", "poutcome",
99      "emp.var.rate", "cons.price.idx", "cons.conf.idx", "euribor3m", "nr.employed"
100    ];
101
102    const categorias = {
103      job: ["admin.", "blue-collar", "entrepreneur", "housemaid", "management", "retired", "self-employed", "services", "student", "technician", "unemployed", "unknown"],
104      marital: ["married", "single", "divorced", "unknown"],
105      education: ["basic.4y", "basic.6y", "basic.9y", "high.school", "illiterate", "professional.course", "university.degree", "unknown"],
106      default: ["yes", "no", "unknown"],
107      housing: ["yes", "no", "unknown"],
108      loan: ["yes", "no", "unknown"],
109      contact: ["cellular", "telephone", "unknown"],
110      month: ["jan", "feb", "mar", "apr", "may", "jun", "jul", "aug", "sep", "oct", "nov", "dec"],
111      day_of_week: ["mon", "tue", "wed", "thu", "fri"],
112      poutcome: ["failure", "nonexistent", "success"]
113    };
114
115    const variablesNumericas = [
116      "age", "duration", "campaign", "pdays", "previous",
117      "emp.var.rate", "cons.price.idx", "cons.conf.idx", "euribor3m", "nr.employed"
118    ];
119
120    let respuestas = {}; // Almacena respuestas del usuario
121    let indicePregunta = 0; // Índice de la variable actual a preguntar
122    const chatbox = document.getElementById('chatbox');
123    const inputArea = document.getElementById('inputArea');
124
125    /* =====
126    FUNCIONES DE INTERFAZ
127    ===== */
128    // Muestra un mensaje en la burbuja de chat

```

```
125  /* =====
126  | FUNCIONES DE INTERFAZ
127  | ===== */
128  // Muestra un mensaje en la burbuja de chat
129  function mostrarMensaje(texto, quien){
130    const div = document.createElement('div');
131    div.className = 'bubble ' + (quien==='Bot'? 'bot': 'user');
132    div.textContent = texto;
133    chatbox.appendChild(div);
134    chatbox.scrollTop = chatbox.scrollHeight;
135  }
136  // Muestra el input apropiado para la variable actual
137  function mostrarInput(){
138    inputArea.innerHTML='';
139    if(indicePregunta>=variables.length) return;
140    const varActual = variables[indicePregunta];
141    mostrarMensaje("Por favor ingresa el dato para " + varActual + ":", "Bot");
142    // Si la variable es categórica: desplegable (select)
143    if(categorias[varActual]){
144      const select=document.createElement('select');
145      select.id='respuestaInput';
146      select.className="form-select";
147      categorias[varActual].forEach(opcion=>{
148        const opt=document.createElement('option');
149        opt.value=opcion;
150        opt.text=opcion;
151        select.appendChild(opt);
152      });
153      inputArea.appendChild(select);
154    }
155    // Si es numérica: input de texto
156    else {
157      const input=document.createElement('input');
158      input.type='text';
159      input.id='respuestaInput';
160      input.placeholder='Ingresa un número';
161      input.className='form-control';
162      inputArea.appendChild(input);
163    }
164  }
165
```

```
166  /* =====
167  |   ENVÍO DE RESPUESTAS
168  |   ===== */
169  function enviarRespuesta(){
170      const inputElem=document.getElementById('respuestaInput');
171      if(!inputElem) return;
172      let valor=inputElem.value.trim();
173      if(!valor){ alert("Por favor ingresa un valor válido."); return; }
174
175      const varActual = variables[indicePregunta];
176      // Validaciones para variables numéricas
177      if(variablesNumericas.includes(varActual)){
178          if(isNaN(valor)){ alert("Debes ingresar un número válido para " + varActual); return; }
179          valor = parseFloat(valor);
180
181          // Reglas específicas por variable
182          if(varActual=="age"){
183              if(valor<17 || valor>110){ alert("La edad debe estar entre 17 y 110 años."); return; }
184          }
185          if(["duration","nr.employed","pdays","previous"].includes(varActual) && valor<0){
186              alert(varActual+" no puede ser menor a 0."); return; }
187          if(varActual=="campaign" && valor<1){ alert("campaign debe ser mayor o igual a 1."); return; }
188      }
189      // Guarda respuesta y pasa a la siguiente variable
190      respuestas[varActual]=valor;
191      mostrarMensaje(valor,"Usuario");
192      indicePregunta++;
193      if(indicePregunta<variables.length){ mostrarInput(); }
194      else{
195          mostrarMensaje("Gracias! Estoy procesando tu predicción...", "Bot");
196          enviarAlBackend();
197          inputArea.innerHTML='';
198      }
199  }
200
201  /* =====
202  |   LLAMADAS AL BACKEND
203  |   ===== */
204  // Envía las respuestas al endpoint /predict
205  function enviarAlBackend(){
206      fetch('/predict',{
207          method:'POST',
```

```

201  /* ===== */
202  LLAMADAS AL BACKEND
203  ===== */
204  // Envía las respuestas al endpoint /predict
205  function enviarAlBackend(){
206      fetch('/predict',{
207          method:'POST',
208          headers:{'Content-Type':'application/json'},
209          body:JSON.stringify(respuestas)
210      })
211      .then(res=>res.json())
212      .then(data=>{
213          if(data.error){
214              mostrarMensaje("Error: "+data.error,"Bot");
215          } else {
216              // Muestra resultado y probabilidad con barra de progreso
217              const prob=(data.probabilidad*100).toFixed(2);
218              const div=document.createElement('div');
219              div.className='bubble bot';
220              div.innerHTML = "Predicción: "+(data.prediccion===1?"Sí contratará el depósito a plazo":"No contratará el depósito a plazo")+" (Probabilidad: "+prob+"%)";
221
222              const progress=document.createElement('div');
223              progress.className='progress-bubble';
224              const progressBar=document.createElement('div');
225              progressBar.className='progress-bar-bubble';
226              progressBar.style.width = prob + '%';
227
228              // Color según porcentaje
229              if(prob<40){ progressBar.style.backgroundColor='red'; }
230              else if(prob<70){ progressBar.style.backgroundColor='yellow'; progressBar.style.color='black'; }
231              else{ progressBar.style.backgroundColor='green'; }
232
233              progress.appendChild(progressBar);
234              div.appendChild(progress);
235              chatbox.appendChild(div);
236              chatbox.scrollTop = chatbox.scrollHeight;
237              // Activa sección RAG
238              document.getElementById("ragArea").style.display="block";
239          }
240      })
241      .catch(()=>mostrarMensaje("Error de conexión con el servidor","Bot"));
242  }
243
244  // Envía una pregunta al endpoint /rag_chat
245  function enviarPreguntaRAG(){
246      const pregunta=document.getElementById("preguntaRAG").value.trim();
247      if(!pregunta){ alert("Por favor escribe una pregunta."); return; }
248
249      const loadingElem=document.getElementById("loadingRAG");
250      const respuestaElem=document.getElementById("respuestaRAG");
251
252      loadingElem.classList.remove("d-none");
253      respuestaElem.textContent="";
254
255      fetch("/rag_chat",{
256          method:"POST",
257          headers:{'Content-Type':'application/json'},
258          body:JSON.stringify({pregunta:pregunta})
259      })
260      .then(res=>res.json())
261      .then(data=>{
262          loadingElem.classList.add("d-none");
263          if(data.error) respuestaElem.textContent="✗ "+data.error;
264          else respuestaElem.textContent="🤖 "+data.respuesta;
265      })
266      .catch(()=>{
267          loadingElem.classList.add("d-none");
268          respuestaElem.textContent="✗ Error de conexión con el servidor.";
269      });
270  }
271
272  /* Inicia el flujo mostrando la primera pregunta */
273  mostrarInput();
274  </script>
275  </body>
276  </html>
277

```